Syrian Arab Republic
Ministry of Higher Education and Scientific Research
Manara University-Faculty of Engineering
Robotic and Intelligent Systems Department

Simulation of a Mobile Manipulator for Pick-and-Place Tasks in Static Environments

Prepared by:

Beilassan Hdewa
Lana Alwazzeh
Zain Alabidin Shbani

Supervised by

PhD. Mohamad Kheir Abdullah Mohamad, PhD. Essa Alghannam

2025-2026

الإهداء

إهداء بيلسان هديوه

إلى أمي الغالية،

إلى من كانت دعواتها سندًا لي في كل خطوة، ونورًا يضيء طريقي في لحظات التعب والسعي.

إلى القلب الذي منحني الحب بلا حدود، والصبر بلا مقابل، والإيمان بقدرتي في كل وقت وحين.

إليكِ يا أمي، أهدي ثمرة هذا الجهد، عرفانًا بكل ما قدمتهِ من تضحيات، وبكل كلمة تشجيع كانت سببًا في استمراري.

أسأل الله أن يحفظكِ، ويديم عليكِ الصحة والسعادة، وأن يجعل هذا الإنجاز سببًا لفخركِ كما كنتِ دائمًا سببًا في نجاحي.

إليكِ يا أمي... كل الشكر، وكل الحب، وكل هذا الإنجاز.

إلى أبي الغالي،

إلى من علّمني معنى القوة والصبر، وكان سندي الثابت في كل مراحل حياتي.

إلى من لم يبخل يومًا بدعمه، وسعى بكل جهده ليؤمّن لي طريق النجاح والكرامة.

إليك يا أبي، أهدي هذا الإنجاز عرفانًا بفضلك الكبير، وتقديرًا لتضحياتك التي لا تُعد ولا تُحصى، والتي كانت دائمًا خلف كل خطوة وصلتُ إليها.

أسأل الله أن يحفظك لي، وأن يمدّك بالصحة والعافية، وأن يجعل هذا النجاح فرحة في قلبك كما كنتَ دائمًا مصدر فخري وأماني.

إليك يا أبي كل الحب والامتنان.

إلى دانيال،

إلى من كان لي السند والعضد، وشعلة الأمل التي تنير دربي.. إلى أخي الغالي، الذي لم يبخل عليّ يوماً بالنصيحة والتشجيع. أهدي إليك ثمرة هذا الجهد، راجيةً من الله أن نصل معاً إلى أعلى المراتب.

وإلى علي،

إلى أخي وتوءم روحي.. رفيق الدرب الذي رأى في نجاحي طموحاً، وفي تعبي إصراراً. أهدي إليك هذا المشروع، الذي ما كان ليكتمل لولا وقوفك الدائم إلى جانبي.

لكما معًا كل الحب والتقدير.

إلى خالتي العزيزة فاتن،

يقولون إن الحنان لا يتكرر، لكنكِ كذّبتِ هذا القول، إذ وجدتُ فيكِ قلبًا لا يختلف عن قلب أمي في طيبته وسعة عطائه، إليكِ أهدي ثمرة هذا الجهد، عرفانًا بحبٍّ منحتِهِ دون حساب.

إلى أساتذتي،

إلى الذين حملوا أسمى رسالة في الوجود، إلى ينابيع المعرفة والشعلة التي تنير دروبنا، إلى أساتذتي الأفاضل الذين لم يبخلوا علينا بعلمهم وتوجيهاتهم طوال مسيرتنا الجامعية.. نُهدي ثمرة جهدنا هذا

إلى عمّتي الغالية ثناء،

بعض الناس لا تحتاج أن تقول الكثير، حضورهم وحده يمنحك الطمأنينة، وابتسامتهم تكفي لتشعر أن كل شيء سيكون بخير، كنتِ دائمًا قريبة بطريقتك الهادئة التي تسكن القلب، ولم أكن يومًا بحاجة إلى كلمات كثيرة لأعرف أنكِ معي.

إلى هيا،

إلى رفيقة العمر والروح، من كانت ولم تزل سنداً حقيقياً وملجأً دافئاً في كل الأوقات. أهدي إليكِ ثمرة تعبي وجهدي، امتناناً لكل لحظة صدق أهديتني إياها، ولكل كلمة طيبة دفعتني للأمام.

إلى لانا،

إلى رفيقة العمر والمسيرة، من أضاءت عتمة الأيام بابتسامتها، وحوّلت مشاق البحث والدراسة إلى ذكريات لا تُنسى. إلى صديقتي وجميلتي.. أهدي ثمرة هذا الجهد، فنجاحي لا يكتمل إلا بوجودكِ.

إهداء زين العابدين شباني

إلى أبي الحبيب،

ذلك الرجل الذي كان ولا يزال السند الحقيقي في كل خطوة، والذي علّمني أن الطريق إلى النجاح لا يُعبد إلا بالصبر والعمل والإرادة. إلى من حمل عني أثقال الطريق بصمته، ومنحني القوة حين ضعفت، أُهدي هذا الإنجاز عرفانًا بما قدمه من دعم وتضحيات لا تُقدّر بثمن.

إلى أمي الغالية،

إلى القلب الذي لم يعرف يومًا التعب، وإلى العين التي سهرت والدعاء الذي لم ينقطع. إلى من كانت قوتي في ضعفي، ونوري في عتمة الطريق، وسببًا في كل خطوة وصلت إليها. كلمات الشكر تعجز أمام ما قدمتِه من حب واحتواء وتضحية، فأنتِ الأساس في كل نجاح، وأنتِ الحكاية التي لا تُختصر. هذا العمل لكِ أولًا، لأنكِ كنتِ وراء كل ما وصلت إليه.

إلى أختي الدكتورة آية،

شريكتي في الطموح والسعي، التي أثبتت أن الاجتهاد طريقٌ حقيقي نحو التميز، وأن الإصرار يصنع الفرق. كان لوجودك أثر جميل في هذه الرحلة، دعمًا وتشجيعًا في اللحظات التي احتجت فيها إلى من يذكرني بأن الاستمرار هو الطريق الوحيد للوصول. كنتِ دائمًا مصدر فخر واعتزاز، وشاهدة على هذه المسيرة بكل تفاصيلها.

إلى أخي حيدرة،

الذي كان حضوره وحده كافيًا ليمنحني القوة، وابتسامته البسيطة قادرة على أن تخفف عني ثقل الأيام والتعب. رغم صغر سنه، كان له أثر كبير في نفسي، وكان وجوده دافعًا صامتًا للاستمرار والتقدم. ربما لا يدرك حجم تأثيره، لكنه كان دائمًا سببًا في أن أستعيد طاقتي وأمضي قدمًا.

إلى ابن عمي حميد،

الذي لم يكن يومًا مجرد ابن عم، بل أخًا حقيقيًا تربّيت معه وشاركني تفاصيل العمر. كنتَ دائمًا الرفيق الأقرب والسند الثابت، فلك مني كل المحبة والتقدير.

إلى أساتذتي ودكاترتي الكرام،

الذين كانوا مشاعل علمٍ أنارت دربي منذ بداية رحلتي الدراسية وحتى اليوم، ولم يبخلوا بعلمهم وتوجيههم. لكم مني خالص الشكر والتقدير.

إلى أصدقائي الأعزاء،

الذين كانوا جزءًا لا يتجزأ من هذه الرحلة، وشاركوا معي لحظات التعب قبل الفرح، وقدموا لي الدعم والمساندة في أصعب الأوقات. إلى من كانوا دائمًا مصدر طاقة إيجابية وتشجيع مستمر، وكان لوجودهم أثر كبير في استمراري وتحقيقي لهذا الإنجاز.

كنان منى، محمد تنزكلي، بلال غانم، ماهر جبيلي، ماهر سعيد، يوسف عمار، حسين اسماعيل، مطر حنا، يقين حموي، مضر بدور، محمد محمد، مقداد طراف، حسن جعفر.

إلى أصدقائي،

الذين كانت لهم بصمة جميلة في هذه الرحلة، وشاركوني بعضًا من لحظاتها بتشجيعهم ودعمهم الصادق. وجودكم، رغم بساطته، كان له أثر طيب في نفسي، فلكم مني جزيل الشكر والتقدير.

دانيال حنا، منصور سليمان، شادي السليم، سليم الخطيب، يائيل الحسن،حنين شاهين، محمد ابراهيم، اياس سلطان.

إلى صديقتي المهندسة بانه،

رفيقة أيّام المدرسة وبدايات الطريق، التي بقي حضورها جميلًا رغم تغيّر السنوات. كانت صداقتك ذكرى طيبة ودعمًا صادقًا في مراحل مختلفة، فلكِ مني كل الشكر والتقدير.

إليكم جميعًا،

أُهدي هذا المشروع المتواضع، الذي هو ثمرة سنوات من الجهد والتعب، وتعبير صادق عن امتناني لكل من كان له أثر في هذه الرحلة.

إهداء لانا الوزه

إلى أمي،

إلى القلب الذي احتواني في كل عاصفة، وإلى الحضن الذي كان أمانًا حين ضاقت بي الأيام. وقوفكِ إلى جانبي كان حبًا عميقًا لا تستطيع الكلمات أن تفيه حقه، وحنانكِ كان قوةً تسندني في أصعب اللحظات. حتى حزنكِ الخفي خلف ابتسامتكِ كان شاهدًا على صبركِ العظيم، وعلى قدرتكِ الدائمة على منحي الطمأنينة حين كنتُ بأمسّ الحاجة إليها. هذا التخرج ليس إنجازي وحدي، بل هو جزء من تعبكِ، ودعائكِ، وحبكِ الذي رافقني في كل خطوة.

إلى أبي،

إلى الرجل الذي كان حبّه حاضرًا في العطاء أكثر من الكلمات، وفي التضحية أكثر من الوعود. صمتك كان قوة، وصبرك كان سندًا، ودعمك الهادئ كان نورًا يرافقني في هذه الرحلة. في هذا الإنجاز أثر من تعبك، ومن إيمانك بي، ومن حبك الذي لم يكن يومًا بحاجة إلى شرح. لك في هذا النجاح نصيب كبير، لأن وجودك كان دائمًا مصدر أمان وثبات.

إلى أختي سالي،

إلى من تفهمني دون شرح، وتقرأ ما في قلبي قبل أن تنطق الكلمات. قربكِ كان ملاذًا آمنًا، وفهمكِ كان هدية لا تُقدّر بثمن. معرفتكِ بالفرق بين ضحكتي الصادقة وابتسامتي التي كنت أختبئ خلفها كانت دليلًا على عمق قربكِ من روحي. وجودكِ في حياتي كان طمأنينة، وحبكِ كان من الأسباب التي منحتني القوة في اللحظات الصعبة.

إلى أختي رغد،

إلى أختي الكبرى، وسندي، ومصدر قوتي. وجودكِ إلى جانبي كان حمايةً وتشجيعًا وحبًا لا يعرف التردد. دعمكِ في كل مرحلة كان سببًا في شعوري أنني لست وحدي، وحرصكِ على أن يحمل هذا اليوم فرحته الخاصة كان من أجمل ما رافق هذا الإنجاز. شكرًا لقلبكِ الذي كان دائمًا قريبًا، ولحبكِ الذي ذكّرني بأنني أستحق الفرح بكل خطوة تعبت كثيرًا حتى أصل إليها.

إلى دانيال،

إلى صغير عائلتنا، وإلى الوجه الملائكي الذي دخل حياتنا فزادها حبًا ودفئًا. رغم صغر عمرك، كان لوجودك أثر كبير في قلبي؛ فابتسامتك البريئة كانت قادرة على أن تخفف ثقل الأيام، ونظراتك الصغيرة كانت تزرع في داخلي فرحًا لا يشبه سواه. كل لحظة أراك فيها كانت تذكيرًا بأن الفرح قد يولد من قلبٍ صغير، وأن وجود طفلٍ في العائلة قادر على أن يمنحها حياةً وأملًا جديدين.

إلى أختي روزيت،

إلى من علّمتني أن القوة ليست في قسوة القلب، بل في الصبر والتضحية والاستمرار. وجودكِ في حياتي قدوة أعتز بها، ومصدر إلهام يرافقني في كل مرحلة. أرى فيكِ أختًا قويةً، معطاءةً، وصادقةَ القلب، وأتمنى أن أحمل في داخلي شيئًا من صبركِ وثباتكِ وجمال روحكِ.

إلى صديقتي بيلسان،

وجودكِ في أغلب مشاريعي الجامعية كان جزءًا من جمال هذه الرحلة، وتعاونكِ كان سببًا في أن يصبح العمل أخفّ وأجمل وأكثر قيمة. لكِ في هذه الرحلة مكانة خاصة، لأنكِ كنتِ شريكةً مخلصة، وصديقةً رقيقة، ورفيقةً جعلت للتعب معنى أجمل.

إلى صديقتي رنيم،

اجتهادكِ كان دافعًا لي في لحظات التعب، وكلماتكِ اللطيفة كانت تصلني في الوقت الذي أحتاج فيه إلى من يرفع معنوياتي ويذكرني بما أملك من قوة. شكرًا لأنكِ كنتِ ترين في داخلي ما كنت أعجز أحيانًا عن رؤيته.

إلى صديقتي آلاء،

إلى من سبقتني بخطوة في هذا الاختصاص، وكانت بحضورها وحيويتها سببًا في أن أرى هذا الطريق بصورة أجمل. شكرًا لأنكِ كنتِ مصدر حماس وإلهام، ولأن وجودكِ ترك في بداياتي أثرًا لا أنساه.

إلى أساتذتي الأجلاء،

إلى الذين غرسوا فينا حب التميز والبحث العلمي، وكانوا السند العلمي والمعنوي.. أهديكم مشروع تخرجي هذا، عربون محبة ووفاء، سائلين المولى عز وجل أن يبارك في علمكم وجهودكم.

إليكم جميعًا،

هذا العمل إهداءٌ محمّل بكل ما في قلبي من حب وامتنان. لكل واحد منكم أثر لا يُنسى في رحلتي، ولكل واحد منكم مكان لا يشبه غيره في قلبي. ما كان لهذا الإنجاز أن يكتمل معناه، ولا لهذا اليوم أن يحمل فرحته الحقيقية، لولا وجودكم في حياتي وقلبي.

ملخص

يعرض هذا المشروع تصميم نموذج أولي لروبوت متحرك مناور Autonomous Mobile Manipulator مخصص لتنفيذ عمليات الالتقاط والوضع Pick-and-Place ضمن بيئة داخلية ساكنة. يتكوّن التصميم من منصة متحركة رباعية العجلات تعتمد آلية التوجيه التفاضليSkid-Steer ، مدمجة مع ذراع روبوتية ذات أربع درجات حرية 4 DOF-مبنية وفق آلية ربط متوازي .Parallelogram Linkage تم نمذجة النظام باستخدام برنامجSolidWorks ، كما أُجري تحليل الاستقرار الساكن والديناميكي اعتمادًا على نظرية .Zero Moment Point (ZMP) أظهرت نتائج التحليل أن النظام يعمل بأمان ضمن جميع التهيئات المختبرة، مع عامل أمان Safety Factor أكبر من 1.5.

يعتمد إطار التحكم في النظام على التعلم المعزز .Reinforcement Learning (RL) تم تصميم وتدريب وتقييم خوارزميتين هما Proximal Policy Optimization (PPO) و Soft Actor-Critic (SAC)ضمن بيئة ملاحية ثنائية الأبعاد مخصصة. يستقبل عامل التعلم RL Agent متجه حالة مكوّنًا من 14 بُعدًا، يضم زوايا المفاصل، سرعات المفاصل، موضع نهاية الذراعEnd-Effector ، وضعية القاعدة المتحركة، الإزاحة النسبية للهدف، وحالة القابض .Gripper كما ينتج العامل متجه فعل مستمر مكوّنًا من 5 أبعاد يتحكم في سرعات مفاصل الذراع، والسرعة الخطية والزاوية للقاعدة المتحركة. تم بناء دالة مكافأة مكوّنة من ثمانية عناصر، تجمع بين إشارات مكافأة كثيفة Dense Reward Shaping وإشارات مكافأة متقطعة مرتبطة بإنجاز مراحل محددةSparse Milestone Bonuses ، وذلك لتوجيه العامل خلال مهام الوصول، والإمساك، والنقل، والوضع. أظهرت النتائج التجريبية تفوق خوارزمية SAC على PPO من حيث كفاءة استخدام العينات، وقيمة المكافأة النهائية، ونسبة النجاح، وجودة المسار، في حين أظهرت PPO استقرارًا أعلى أثناء المراحل الأولى من الاستكشاف والتدريب.

تم تصميم مسار إدراك بصري قائم على YOLO لدعم اكتشاف الأجسام وتحديد مواقعها. كما يناقش التقرير خيارين لتحديد الموقع البصري: تحديد الموقع ثلاثي الأبعاد باستخدام كاميرا RGB-D بوصفه خيارًا إدراكيًا كاملًا، وتحديد الموقع أحادي الرؤية باستخدام Raspberry Pi Camera Module 3 بوصفه بديلًا منخفض التكلفة. يوضح هذا التكامل إمكانية ربط مخرجات الإدراك البصري مع بنية التحكم القائمة علىReinforcement Learning ، بما يدعم تنفيذ مهام-Pick and-Placeضمن بيئات داخلية ساكنة.

الكلمات المفتاحية:

Mobile Manipulator, Pick-and-Place, Reinforcement Learning, PPO, SAC, YOLO, Skid-Steer Robot, CoppeliaSim, Robotic Arm Kinematics, ZMP Stability Analysis.

Abstract

Design of a prototype of an Autonomous Mobile Manipulator that is designed to perform Pick and Place operations in static indoor environment is demonstrated in this project. This design consists of a four-wheel skid-steer mobile platform integrated with a 4 degrees of freedom parallelogram linkage robotic arm. This system has been modeled in SolidWorks and the static as well as dynamic stability analysis has been performed using Zero Moment Point (ZMP) theory, and it has been found to operate safely in all tested configurations with a safety factor greater than 1.5.

The control framework is built upon reinforcement learning (RL). Two algorithms — Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) — were designed, trained, and evaluated in a custom two-dimensional navigation environment. The RL agent observes a 14-dimensional state vector encoding joint angles, joint velocities, end-effector position, base pose, relative target displacement, and gripper state, and produces a 5-dimensional continuous action vector governing arm joint velocities and base linear and angular velocities. An eight-component reward function combining dense shaping signals with sparse milestone bonuses was formulated to guide the agent through reaching, grasping, transporting, and placing subtasks. Experimental results show that SAC outperforms PPO in sample efficiency, final reward, success rate, and path quality, while PPO offers superior training stability during early exploration

A YOLO-based perception pipeline was designed to support object detection and localization. The report discusses both RGB-D-based localization as a full 3D perception option and Raspberry Pi Camera Module 3-based monocular localization as a low-cost implementation alternative.

Keywords: *Mobile Manipulator, Pick-and-Place, Reinforcement Learning, PPO, SAC, YOLO, Skid-Steer Robot, CoppeliaSim, Robotic Arm Kinematics, ZMP Stability Analysis.*

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS

AI — Artificial Intelligence

CAD — Computer-Aided Design

CoM — Center of Mass

CSV — Comma-Separated Values

DC — Direct Current

DDPG — Deep Deterministic Policy Gradient

D-H — Denavit–Hartenberg

DOF — Degree of Freedom

FK — Forward Kinematics

GPIO — General-Purpose Input/Output

GUI — Graphical User Interface

HC-SR04 — Ultrasonic Distance Sensor Module

ICRA — International Conference on Robotics and Automation

IK — Inverse Kinematics

IoU — Intersection over Union

Li-ion — Lithium-Ion

MDP — Markov Decision Process

MLP — Multilayer Perceptron

NMS — Non-Maximum Suppression

PCA9685 — 16-Channel PWM Servo Driver

PPO — Proximal Policy Optimization

PWM — Pulse Width Modulation

RGB — Red, Green, and Blue

RGB-D — Red, Green, Blue, and Depth

RL — Reinforcement Learning

ROS — Robot Operating System

SAC — Soft Actor-Critic

SF — Safety Factor

URDF — Unified Robot Description Format

YOLO — You Only Look Once

ZMP — Zero Moment Point

1. Introduction

Autonomous robotic systems capable of interacting with physical objects in unstructured or semi-structured environments have become a central research focus in modern robotics. Among such systems, mobile manipulators — platforms that combine a wheeled mobile base with an articulated robotic arm — offer a particularly versatile solution by extending the reachable workspace of fixed-base manipulators to cover large areas. Applications range from warehouse logistics and industrial assembly to assistive robotics and household automation.

Despite significant progress in robotic manipulation, the autonomous execution of complete pick-and-place cycles — encompassing perception, navigation, reaching, grasping, transport, and placement — remains a challenging problem. The difficulty arises from the tight coupling between base motion and arm posture, the need for accurate object localization, and the complexity of designing controllers that are both generalizable and computationally tractable for deployment on resource-constrained hardware.

This project addresses these challenges by developing a simulation-based intelligent mobile manipulator for pick-and-place tasks in static indoor environments. The proposed system combines a low-cost physical prototype with a reinforcement-learning-based control framework, a YOLO-based visual perception pipeline, and a rigorous mechanical and stability analysis. The following sections define the problem, formulate the research questions, and state the aims, objectives, and significance of the study.

1.1. Problem Statement

Existing mobile manipulation platforms capable of performing autonomous pick-and-place tasks in indoor environments typically rely on expensive hardware, high-performance computing, or complex sensor suites that are inaccessible for educational and low-cost research settings. Furthermore, conventional model-based controllers require precise dynamic models that are difficult to derive for underactuated or redundant mobile manipulators subject to nonholonomic constraints. Reinforcement learning offers a promising data-driven alternative; however, training stable and efficient RL policies for combined navigation and manipulation tasks remains an open challenge due to high-dimensional state-action spaces, sparse reward signals, and stability risks inherent to mobile platforms carrying extended arms.

Additionally, integrating vision-based perception with learned control in a coherent and modular architecture — particularly on embedded hardware such as Raspberry Pi — requires careful system co-design. The absence of low-cost, modular, and educationally accessible platforms that demonstrate the full pipeline from visual detection to physical manipulation motivates the present work.

1.2. Research Question

The central research question guiding this project is:

“Can a low-cost mobile manipulator, controlled by a reinforcement learning policy trained in simulation, successfully perform autonomous pick-and-place tasks in static indoor

environments, while maintaining mechanical stability and achieving reliable object detection through an integrated YOLO-based perception pipeline?"

This overarching question is decomposed into the following sub-questions:

- Can PPO and SAC policies independently learn effective navigation behavior for a skid-steer mobile base, and how do they compare in terms of convergence, success rate, and path efficiency?
- Can the mechanical design of the proposed mobile manipulator maintain static and dynamic stability across all operational arm configurations?
- Can a YOLO-based perception module accurately detect and localize target objects in real time on embedded hardware, and can its output be effectively integrated into the RL state representation?
- Can the navigation policy trained in a 2D environment be successfully transferred to a 3D simulation and used to drive a complete pick-and-place sequence?

1.3. Aims of The Project

The primary aim of this project is to design, simulate, and validate an autonomous mobile manipulator system that integrates reinforcement learning control, visual perception, and kinematic modeling into a unified, low-cost platform for pick-and-place operations in static indoor environments. Specifically, the project aims to:

- Develop a physically realizable robotic platform based on a 4-DOF parallelogram-linkage arm mounted on a four-wheel skid-steer base, constructed from affordable and readily available components.
- Establish a rigorous kinematic and stability analysis framework covering forward kinematics, inverse kinematics, static CoM analysis, and dynamic ZMP analysis.
- Design and train reinforcement learning controllers (PPO and SAC) capable of autonomous navigation and pick-and-place task execution in simulation.
- Integrate a YOLO-based visual perception module for real-time object detection and 3D localization, and connect its output to the RL control pipeline.
- Validate the complete system in CoppeliaSim and MATLAB simulation environments, demonstrating end-to-end task completion.

1.4. Objectives

To achieve the stated aims, the following specific objectives were defined:

1. Perform a complete mechanical design of the mobile manipulator in SolidWorks, including mass-properties analysis, bill of materials, and center-of-mass validation.
2. Derive the Denavit-Hartenberg (D-H) parameters and analytical forward and inverse kinematics equations for the 4-DOF parallelogram-linkage arm.
3. Develop the forward and inverse kinematic model of the 4WD skid-steer mobile base, including global odometry integration.
4. Conduct static and dynamic stability analyses for five representative arm configurations, computing safety factors relative to the ZMP stability criterion.

5. Design a custom reinforcement learning environment with a 14-dimensional state space, a 5-dimensional continuous action space, and an eight-component shaped reward function.
6. Train and evaluate PPO and SAC agents in the custom 2D navigation environment and conduct a quantitative comparative analysis of their performance.
7. Design and document a YOLO-based perception pipeline for object detection, including dataset preparation, training configuration, grasp-point estimation, and temporal filtering.
8. Implement a full 3D simulation in CoppeliaSim integrating the navigation policy, manipulation module, vision system, and pick-and-place state machine.
9. Cross-validate the system behavior using a Python-to-MATLAB pipeline with quantitative trajectory, end-effector, and state-machine plots.
10. Construct a functional physical prototype and document the hardware architecture, wiring diagram, and component specifications.

1.5. Significance of the Study

This study contributes to the field of intelligent robotics at multiple levels. From a scientific perspective, it provides a systematic comparative evaluation of PPO and SAC in a mobile manipulation context, contributing empirical evidence to the ongoing discussion on RL algorithm selection for combined locomotion-manipulation tasks. The reward shaping strategy and modular state-action design described herein offer a reusable framework for future RL-based robotic control research.

From an engineering perspective, the project demonstrates that the full pipeline from mechanical design and stability analysis to perception integration and simulation validation can be realized on a low-cost platform (<\$200 in hardware) using open-source tools (ROS-compatible architecture, Python, MATLAB, CoppeliaSim). This makes the platform accessible to educational institutions and research groups with limited resources, addressing an important gap in the robotics research community.

From a practical standpoint, the modular architecture of the system — separating navigation, manipulation, and perception into independently testable modules — provides a clear pathway for incremental development toward fully autonomous, end-to-end vision-based RL control. The work therefore serves as both a validated proof-of-concept and a scalable foundation for future research in mobile manipulation, sim-to-real transfer, and embedded intelligent systems.

2. Literature Review

2.1. Foundational Background and Early Developments

Mobile manipulation has developed as an important field in robotics because it combines the workspace extension provided by a mobile base with the object-interaction capability of a robotic arm. Early research established that a mobile manipulator cannot be treated as a fixed manipulator mounted on a passive vehicle, because base motion, arm posture, nonholonomic constraints, and end-effector accuracy are strongly coupled. Yamamoto and Yun [1] introduced one of the early coordination approaches for mobile manipulators, showing the importance of controlling locomotion and manipulation as a unified system. Seraji [2] further developed a unified motion-control framework for mobile manipulators by considering the nonholonomic constraints of the wheeled base and the redundancy produced by the integration of the mobile platform and robotic arm.

The concept of mobile manipulation as a robotic assistant was also emphasized by Khatib [3], where mobility, manipulation, sensing, and interaction with the environment were considered essential elements of autonomous service robots. Later, Srinivasa et al. [4] presented HERB 2.0 as a complete mobile-manipulation platform for home environments, demonstrating the importance of integrating mechanical design, perception, planning, and manipulation. Xu et al. [5] focused on base-position planning for mobile manipulators performing multiple pick-and-place tasks, showing that the position of the mobile base strongly affects reachability, task efficiency, and robustness.

These foundational studies provide the theoretical basis for mobile manipulation systems. They show that pick-and-place performance depends not only on the manipulator but also on the coordination between the base, arm, perception system, and control strategy.

2.2. Recent Developments in Mobile Manipulation and Skid-Steer Mobility

Recent research has extended mobile robot modeling from ideal kinematic descriptions toward data-driven and probabilistic models that account for uncertainty, wheel slip, and terrain interaction. Trivedi et al. [6] proposed a probabilistic motion model for skid-steer wheeled mobile robots using Gaussian Process Regression and Sigma-Point Transform techniques. Their work is relevant to four-wheel skid-steer platforms because it addresses the difference between ideal kinematic behavior and real motion affected by skidding and slipping. Although the present project focuses on a structured indoor environment, the study supports the importance of modeling skid-steer motion carefully when predicting robot pose and designing navigation behavior.

Recent mobile manipulation studies also show the increasing importance of connecting perception with manipulation. Zhao et al. [7] proposed a YOLO-based learning framework for agricultural robotic manipulation, where object detection and grasp-position estimation are combined to support harvesting operations in a simulated environment. This type of work is relevant to pick-and-place systems because object localization is not sufficient by itself; the detected object must also be transformed into a usable grasping target for the manipulator.

2.3. Learning-Based Control for Robotic Manipulation

Reinforcement learning has become an important approach for robotic manipulation because it can learn control policies in continuous state and action spaces. Han et al. [8] reviewed deep reinforcement learning algorithms for robotic manipulation and discussed value-based methods, policy-gradient methods, actor-critic approaches, reward engineering, and simulation-to-real transfer challenges. This review provides a suitable theoretical foundation for the use of PPO, SAC, and DDPG in robotic manipulation tasks.

Recent studies have also investigated the direct use of PPO and SAC in robotic grasping. Ferreira and Barbosa [9] developed a PyBullet-based robotic grasping environment and implemented SAC and PPO for adaptive grasping and post-grasp manipulation. Their results show that reinforcement learning can be applied to grasping tasks when the reward function includes distance, grasping, and post-grasp pose terms. Shianifar et al. [10] studied hyperparameter optimization for PPO and SAC in robotic arm control, showing that training efficiency can be improved by optimizing learning parameters. These studies support the structure of the reinforcement learning chapters in this project, especially the state representation, action space, reward function, and actor-critic architecture.

2.4. YOLO-Based Perception for Object Detection and Manipulation

Vision-based perception is a central component of autonomous pick-and-place systems because the robot must identify the object before planning the reaching and grasping motion. YOLO-based detectors are commonly used in real-time robotic applications due to their single-stage detection structure and fast inference capability. Ahmed et al. [11] demonstrated the use of YOLOv8 with custom datasets for real-time object detection, which supports the use of a project-specific dataset when standard datasets do not match the target objects. Mekled et al. [12] compared YOLOv5, YOLOv8, and YOLOv9 under varying conditions, showing the importance of evaluating detector performance under different visual environments.

Recent YOLO research also focuses on small-object detection and embedded deployment. Liang et al. [13] proposed an improved YOLOv8 model for small-object detection, which is relevant when target objects occupy only a small region of the camera frame. Bu et al. [14] studied the deployment of YOLOv8 on RK3588-based embedded hardware, supporting the general direction of using edge devices for real-time object detection. Syamsul et al. [15] compared different optimizers for YOLOv8 and YOLOv11 in small-object detection, which is relevant to the selection of training settings in YOLO-based perception pipelines.

2.5. Research Gap and Project Positioning

The reviewed studies show that mobile manipulation requires the integration of mechanical structure, mobile-base modeling, robotic-arm kinematics, perception, and control. Earlier studies provide the theoretical foundation for mobile manipulators and base-arm coordination, while recent studies emphasize learning-based manipulation, YOLO-based perception, embedded vision, and improved training strategies. However, many advanced platforms rely on expensive hardware, complex sensors, or high-performance computing resources.

The present project addresses this gap by developing a low-cost mobile manipulator based on a plexiglass four-wheel chassis, DC gear motors, a servo-driven robotic arm, ultrasonic sensing,

Arduino-based low-level control, Raspberry Pi-based processing, YOLO-based object detection, and reinforcement-learning control concepts. This structure provides an educational and modular platform for studying indoor pick-and-place tasks in static environments.

3. System Overview

The suggested system would be a mobile manipulator comprising a four-wheel skid steer drive chassis and a front end 4-DOF arm with a parallelogram linkage. While the former offers planar mobility, the latter enables local manipulation for indoors pick-and-place operations. Thus, the robot could first get to the target location and then proceed with manipulation without having a complete redundant industrial arm.

The base is rectangular in shape; therefore, it enhances the support area and makes the platform appropriate for manipulation. The robot arm is fixed rigidly to the front and upper part of the robot chassis in such a way that the workspace of the robot remains in front of it.

There are four joints in the arm; base yaw, shoulder flexion-extension, elbow flexion-extension, and wrist pitch. They are all made using the parallel-linkage configuration; the elbow joint is mounted not at the elbow but at the shoulder, and it transfers its force to the forearm via the parallel four-bar linkage system. This system is suitable for indoor structured tasks, since it provides horizontal and vertical positioning and orientation of the end-effector, while the heaviest actuators are located near the base.

The system is considered to be a single mechanism as opposed to two separate sub-systems. The center of mass will shift as a result of any movement of the arm ahead or the base moving. This is why the analysis of the mechanism is carried out in terms of its mechanics, kinematics, dynamics, and stability.

In this respect, the complete control loop consists of three major aspects: perception, motion planning, and execution of the movement. The perception aspect involves the camera sensor that identifies the object, and then the navigation system positions the robot's base to a proper position, while the arm is controlled by the arm controller for grasping through inverse kinematics and joint trajectories.

platform is suitable for autonomous indoor pick-and-place and object delivery tasks.

4. Mechanical Architecture & Kinematic Analysis of the 4-DOF Arm

4.1. Structural Description & Nomenclature

The robotic manipulator under study is classified as a 4-DOF Articulated Robotic Arm with Parallel Linkage (commonly known as a parallelogram-type robot).

4.2. Actuator Distribution & the Parallelogram Mechanism

To minimize the net moment of inertia, this design shifts the heavy weight of the actuators downward, near the rotating base:

- **Shoulder Motor (Joint 2):** operates the first upper-arm link, enabling control of the forward-backward movement and the vertical displacement (flexion-extension).

- **Elbow Motor (Joint 3):** located close to the shoulder motor at its base instead of near the elbow joint. Mechanical torque is remotely transmitted to the forearm by means of a linkage bar system creating a rigid parallelogram.

Advantages of this design from an engineering standpoint:

1. Low inertia: greatly reduces the load on the shoulder motor.
2. High mass-to-payload ratio: directs the maximum torque of the motors towards the lifting of the payload as opposed to overcoming the weight of the arm itself.

4.3. Kinematic Coupling & the Compensation Logic

In either actual or simulated executions, there arises an additional kinematic coupling phenomenon. The rotation of the shoulder joint results into the parallel bars compelling the elbow joint to adopt a new absolute position in the air depending on the position of the forearm irrespective of any movement of the elbow motor command.

To maintain independent control, the internal control system executes a compensation algorithm within its joint actuation loop, governed by the following structural geometric offset:

$$Error = \theta_3 - \theta_2 - \varphi_{passive}$$

4.4. Denavit–Hartenberg (D-H) Parameters

Due to the parallel-linkage constraint, the absolute angle of the forearm remains geometrically decoupled from the shoulder angle relative to the frame horizon. The standardized D-H parameters for this 4-DOF configuration are defined as follows:

Link (i)	Joint Angle (θ_i)	Link Twist (α_i)	Link Length (a_i)	Link Offset (d_i)	Joint Type
1	θ_1 (Base)	90°	0	L_1 (Base Height)	Revolute
2	θ_2 (Shoulder)	0°	L_2 (Upper Arm)	0	Revolute
3	θ_3 (Elbow)	0°	L_3 (Forearm)	0	Revolute
4	θ_4 (Wrist)	0°	L_4 (End-effector)	0	Revolute

Table 4-1: The standardized D-H parameters

where:

Parameter	Description	Measured Value (cm)
L_1	Base Height	10
L_2	Upper Arm	20
L_3	Forearm	15
L_4	End-effector	8

Table 4-2: The description and measured values of the parameters

4.5. Analytical Kinematics

A. Forward Kinematics (FK)

By computing the sequential homogeneous transformation matrices ($T = T_0^1 \cdot T_1^2 \cdot T_2^3 \cdot T_3^4$) and simplifying them according to the closed-loop parallelogram geometry, the exact operational spatial coordinates (X, Y, Z) of the end-effector are determined:

Extended radial reaching distance (R):

$$R = L_2 \cdot \cos(\theta_2) + L_3 \cdot \cos(\theta_2 + \theta_3) + L_4 \cdot \cos(\theta_2 + \theta_3 + \theta_4)$$

Position coordinate X:

$$X = R \cdot \cos(\theta_1)$$

Position coordinate Y:

$$Y = R \cdot \sin(\theta_1)$$

Vertical elevation coordinate Z:

$$Z = L_1 + L_2 \sin(\theta_2) + L_3 \sin(\theta_2 + \theta_3) + L_4 \sin(\theta_2 + \theta_3 + \theta_4)$$

B. Inverse Kinematics (IK)

In order to determine the exact angles that will be used to define the target coordinates (X, Y, Z) for control, the following geometric approach is utilized:

1. Base angle (θ_1): calculated using the horizontal projection directly as $\theta_1 = tan^{-1}(\frac{Y}{X})$.
2. Wrist decoupling: the coordinate of the last wrist center (R', Z') is determined by removing the orientation vector of L_4.
3. Law of cosines: directly applicable to the triangle formed by L_2 and L_3 in the plane to give analytical and collision-free solutions of θ_2 and θ_3 [16] [17].

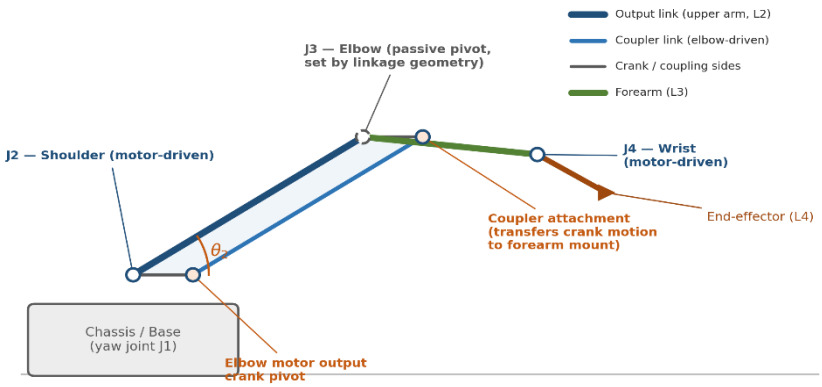

Figure 4-1: Four-bar parallelogram linkage schematic showing the coupler link, output link, and the elbow-motor crank arrangement that keeps the heaviest actuators close to the base.

5. Kinematic Modeling of the 4WD Skid-Steer Mobile Platform

5.1. Introduction & Kinematic Configuration

The mobile platform used for this project is a 4WD Skid Steer Mobile Robot. As opposed to the Ackermann-steering platforms, which include ordinary cars, this type of system does not have any mechanical means to steer. The steering and orientation changes are accomplished only through the differences in velocities of the two-wheel pairs (left and right).

5.2. Kinematic Modeling Parameters

To derive the kinematic equations, the following geometric and operational variables are defined:

- r: the radius of each wheel.
- b: the track width (the lateral distance between the left and right wheels).
- L: the wheelbase (the longitudinal distance between the front and rear axles).
- ω_L: the angular velocity of the left-side motors.
- ω_R: the angular velocity of the right-side motors.
- V_L, V_R: the linear velocities of the left and right sides respectively, where $V_L = r \cdot \omega_L$ and $V_R = r \cdot \omega_R$.

5.3. Forward Kinematic Model (Local Frame)

The forward kinematics transform the angular velocities of the wheels into the linear velocity v_x and angular velocity ω_z of the robot's center of mass in its local reference frame:

Linear velocity along the local X-axis (v_x):

$$v_x = \frac{v_R + v_L}{2} = \frac{r}{2} \cdot (\omega_R + \omega_L)$$

Angular velocity around the vertical Z-axis (ω_z):

$$\omega_z = \frac{v_R - v_L}{b} = \frac{r}{b} \cdot (\omega_R - \omega_L)$$

Lateral velocity (v_y)

Assuming ideal rolling conditions without excessive lateral slip, the local lateral velocity is mathematically constrained to zero: $v_y = 0$.

5.4. Global Coordinate Transformation (Odometry)

To map the robot's motion into a global coordinate system $[X, Y, \theta]^T$, where θ represents the heading angle (orientation) of the platform relative to the global X-axis, the local velocities are integrated using the standard trigonometric transformation:

$$\dot{X} = v_x \cdot \cos(\theta) = (\frac{r}{2})(\omega_R + \omega_L) \cdot \cos(\theta)$$

$$\dot{Y} = v_x \cdot \sin(\theta) = (\frac{r}{2})(\omega_R + \omega_L) \cdot \sin(\theta)$$

$$\dot{\theta} = \omega_z = (\frac{r}{b})(\omega_R - \omega_L)$$

5.5. Inverse Kinematic Model

The inverse kinematics are critical for the control software (e.g., Arduino firmware). They translate the desired global translational velocity (v) and rotational steering velocity (ω) into the specific angular speeds required for the left and right actuators:

$$\omega_R = \frac{(v + \frac{\omega \cdot b}{2})}{r}$$

$$\omega_L = \frac{(v - \frac{\omega \cdot b}{2})}{r}$$

Control logic interpretation for firmware implementation:

- **Straight motion ($v > 0$, $\omega = 0$):** both sides rotate at identical speeds in the same direction ($\omega_R = \omega_L$).
- **Pivot turn / zero-radius turn ($v = 0$, $\omega \neq 0$):** the left and right wheels rotate at equal speeds but in opposite directions ($\omega_R = -\omega_L$), causing the chassis to spin about its geometric center [6] [17].

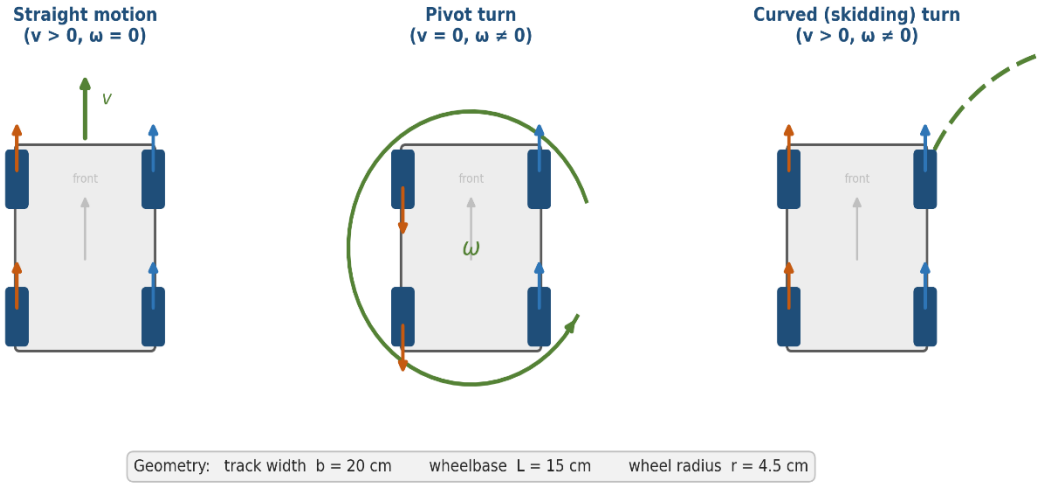

Figure 5-1: 4WD skid-steer base design schematic: straight motion, pivot turn, and curved (skidding) turn, with geometry table (track width b = 20 cm, wheelbase L = 15 cm, wheel radius r = 4.5 cm).

6. Stability Analysis of the Mobile Manipulator

The mobile manipulator being examined here uses a drive base with four wheels that can turn in either direction with an arm consisting of a 4 degrees-of-freedom parallelogram linkage arm on the anterior side (front) of the drive base. This arm consists of a shoulder joint (J2), elbow joint (J3) and wrist joint (J4). In addition, the pivot of the shoulder joint is located above the mounting plate at a distance d1 from it. Since the shoulder joint supports the downstream

kinematic chain (elbow, wrist and gripper), it controls the center of mass forward shift and consequently its tipping potential.

In this project, both the static and dynamic stability limits for the robot have been determined. The static limit has been calculated based on a tipping moment analysis around the front wheel axle with respect to five arm positions, from stowed to fully extended, with payload. On the other hand, the dynamic analysis considers the ZMP translation due to the acceleration and deceleration of the base. In addition to that, the worst-case scenario of maximum reach, maximum payload, and maximum braking has been considered.

Based on the current CAD model, the shoulder height offset is approximated as 0.135 m and is used consistently throughout this analysis.

Modeling point regarding the parallelogram linkage: as the elbow joint is driven using the parallelogram four bar linkage mechanism explained in Section $4.2 - 4.3$, the orientation of the forearm in space does not simply become $\theta_2 + \theta_3$ like it would in a normal serial chain; it depends on the linkage geometry and the passive compensation angle $\varphi_{passive}$. All forward-kinematics and center of mass equations used to start in Section 6.1 use $\theta_2$and θ_3 to represent the effective angles of the forearm, accounting for this passive compensation angle – i.e., the actual orientation of the forearm in space – instead of working out the closed-form equations of the linkage in Section 4.

6.1. Physical System Parameters

Point to Note: While doing the stability analysis, the indexing of arm segments is done with respect to the shoulder reference frame. Hence, the notation $L_1 - L_3$ of section 6 refers to the upper arm, lower arm, and wrist/gripper segment, respectively, which is different from the D-H notation explained in section 4.

Parameter	Symbol	Value	Note/Derivation Source
Vehicle Base Parameters			
Base length	L_b	$0.231\ m$	Direct CAD measurement (longitudinal footprint)
Base width/track	W_b	$0.192\ m$	Direct CAD measurement (lateral footprint)
Base height	H_b	$0.135\ m$	Top of mounting plate above ground level
Half-length (centroid→ front axle)	$half\_L$	$0.1155\ m$	Support-polygon boundary; calculated as $L_b/2$
Half-track width	d_{lat}	$0.096\ m$	Support-polygon boundary; calculated as $W_b/2$
Base mass (incl. electronics)	m_b	$5.0\ kg$	Total mass including onboard electronics
Robotic Arm Parameters			

Upper arm length/mass	L_1/m_1	$0.12\ m/0.8\ kg$	Link 1 physical properties (effective basis)
Forearm length/mass	L_2/m_2	$0.10\ m/0.5\ kg$	Link 2 physical properties (effective basis)
Wrist + gripper length/mass	L_3/m_3	$0.08\ m/0.3\ kg$	Link 3 physical properties (end-effector)
Shoulder height offset	d_1	$0.135\ m$	Approximated from CAD assembly alignment
Derived System Totals			
Total arm mass	m_{arm}	$1.6\ kg$	Calculated summation: $m_1 + m_2 + m_3$
Total system mass	M_{tot}	$6.6\ kg$	Total combined weight: $m_b + m_{arm}$

Table 6-1: The physical parameters used throughout project analysis.

6.2. Forward Kinematics of the Arm (Shoulder-Dominant Chain, Effective Angles)

The planar forward kinematics from the shoulder pivot, projected onto the sagittal (XZ) plane, using the forearm's effective post-compensation angle as described in the modeling note above, are:

$$x(\theta_2, \theta_3) = L_1 \cdot \cos(\theta_2) + L_2 . \cos\,(\theta_2 + \theta_3)$$

$$z(\theta_2, \theta_3) = d_1 + L_1 \cdot \sin(\theta_2) + L_2 . \sin(\theta_2 + \theta_3) + L_3 . \sin(\theta_2 + \theta_3)$$

Since the angle formed by the shoulder joint establishes the common angular displacement between the whole downstream system, the angle itself becomes a determinant factor on whether the arm extends upwards (θ_2 approaching 90°) or stretches out forwards (θ_2 approaching 0°). In worst-case configurations below, the wrist is aligned in collinearity to the forearm.

6.3. Configurations Analyzed

Configuration	Position Code	θ_2/θ_3	Posture Description
A	Stowed	$90°/0°$	Arm vertical, fully retracted inside footprint.
B	Max. Horizontal Reach	$0°/0°$	Arm fully extended forward, horizontal to ground.
C	Max. Reach + Payload	$0°/0°$	As configuration B, with a 100 g object held at tip.
D	Typical Pick Pose	$30°/-60°$	Elbow bent downwards, representative grasp posture.
E	Pick Pose + Payload	$30°/-60°$	As configuration D, with a 100 g object held at tip.

Table 6-2: Arm configurations evaluated.

6.4. Static Stability Analysis

The static stability test involves calculating the torque balance about the axle of the front wheel. This particular point serves as the fulcrum since the arm is positioned precisely above the axle and any change in position of the center of mass causes rotation around it.

6.4.1. Support Polygon and Tipping Fulcrum

The support polygon is the quadrilateral formed by the four contact points between the wheels and ground. The shoulder joint being positioned on the front face means that the front wheel axle is not only the support polygon boundary but also the point of natural tipping around the forward extended arm, and it is positioned at $x = +0.1155\ m$ from the base center of mass.

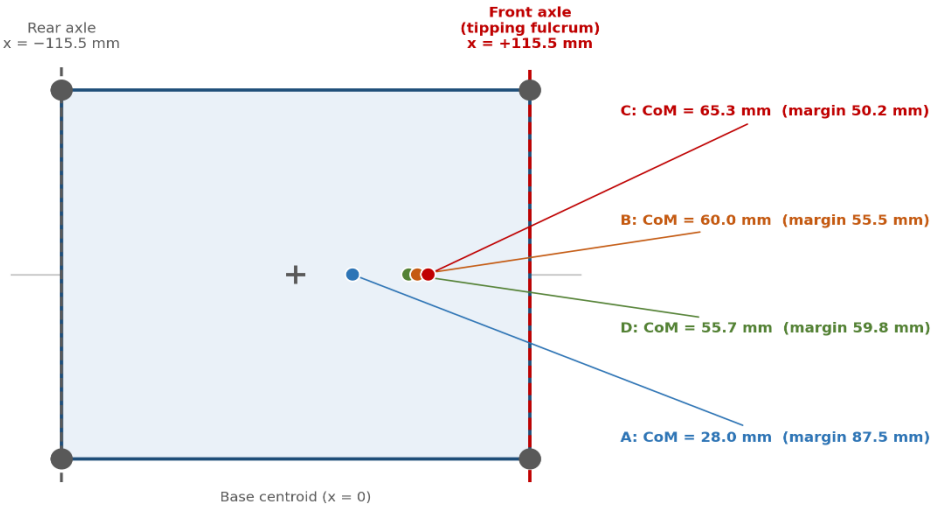

Figure 6-1: Top view of the support polygon, showing the front-mounted shoulder pivot directly above the front axle and the CoM projections for configurations A, B, C, and D.

6.4.2. System Centre-of-Mass Calculation

The center of mass for the entire system is the mass-weighted mean of the base CoM and the three arm links' midpoint CoMs, all relative to the base CoM at ground level:

$$COM_{sys} = \frac{(m_b \cdot r_b + m_1 \cdot r_{mid_1} + m_2 \cdot r_{mid_2} + m_3 \cdot r_{mid_3})}{M_{tot}}$$

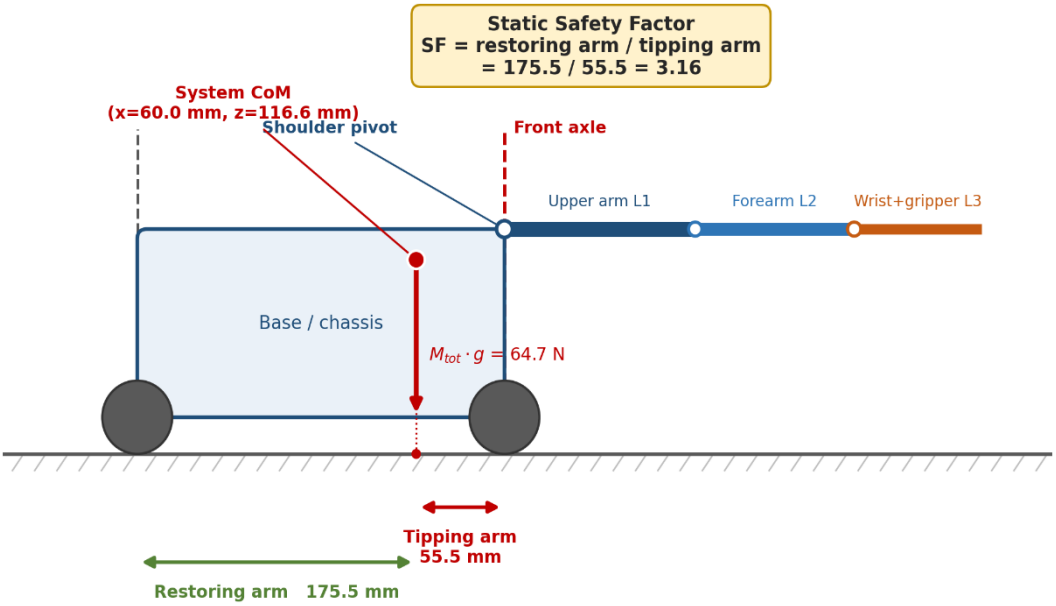

Figure 6-2: Side-view free-body diagram for the worst-case static configuration (B). The tipping arm (55.5 mm, from the CoM projection to the front axle) is compared against the restoring arm (175.5 mm, from the CoM projection to the rear axle). Safety Factor = restoring arm / tipping arm = 175.5 / 55.5 = 3.16.

with the base CoM taken as half the height of the base, and each midpoint calculated using sequential forward kinematics starting from the shoulder pivot point (Section 6.2). Calculating the above equation in the worst-case position (B: maximum horizontal reach) gives an CoM 60.0 mm ahead of the base CoM — well under the 115.5 mm front axle limit.

6.4.3. Static Stability Summary

Configuration	$CoM_x(mm)$	$CoM_z(mm)$	Margin to front axle (mm)	Static SF
A	28.0	148.6	87.5	**1.64**
B	60.0	116.6	55.5	**3.16**
C	65.3	118.9	50.2	**3.60**
D	55.7	122.4	59.8	**2.86**
E	60.5	124.2	55.0	**3.20**

Table 6-3: Static CoM position and safety factor for all evaluated configurations.

The safety factor is defined as the ratio between the restoring moment arm and the tipping moment arm measured about the front axle. Consequently, the safety factor does not scale linearly with the remaining margin to the support-polygon boundary.

6.5. Dynamic Stability Analysis — ZMP Under Acceleration

The static analysis makes an assumption that the acceleration is zero. In practice, the base acceleration/deceleration moves the effective Zero Moment Point (ZMP) from its static CoM projection. The relationship of interest concerning the acceleration ax in the x direction is:

$$ZMP_x = CoM_x - \frac{(CoM_z \cdot a_x)}{g}$$

Braking ($a_x < 0$) shifts the ZMP forward, toward the front axle — the more demanding case for this front-mounted arm. Forward acceleration ($a_x > 0$) shifts the ZMP backward, which is self-stabilizing. The conservative bound adopted for this analysis is a maximum deceleration magnitude of $0.30\ m/s^2$.

6.5.1. Worst-Case Emergency Braking

Evaluating the ZMP shift at the worst-case static configuration (B: maximum reach) under full braking:

$$ZMP_x = 0.0600 - \frac{(0.1166 \times (-0.30))}{9.81} = 0.0600 + 0.0036 = 0.0636\ m$$

This gives a margin of $51.9\ mm$ to the front axle and a dynamic safety factor of approximately 3.45 — the braking deceleration moves the effective tipping point forward by only $3.6\ mm$

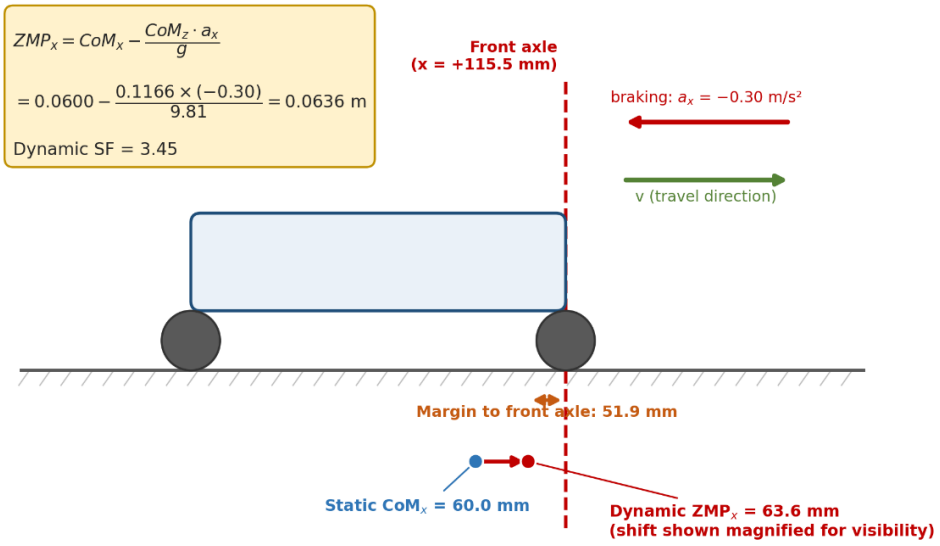

Figure 6-3: Dynamic ZMP shift during emergency braking from the worst-case static configuration. Static CoM x = 60.0 mm → Dynamic ZMP x = 63.6 mm; margin 51.9 mm; Dynamic SF = 3.45.

6.5.2. Combined Worst Case — Maximum Reach, Payload, and Braking

Combining the highest CoM height (full extension), maximum payload, and full braking deceleration simultaneously:

$$ZMP_x = 0.0653 - \frac{(0.1189 \times (-0.30))}{9.81} = 0.0653 + 0.0036 = 0.0689\ m$$

This represents the most demanding operating condition considered in the analysis and still remains comfortably within the prescribed stability limits.

6.5.3. Rotational (Yaw) Dynamics

The base yaw motion performed at the maximum commanded speed ($\omega = 0.5\ rad/s$) will cause centrifugal force acting on the arm's mass. The lateral moment arm for the computation of centrifugal force may be taken as the longitudinal center of mass offset ($60\ mm$):

$$F_{centrifugal} = m_{arm} \cdot \omega^2 \cdot r = 1.6 \times 0.5^2 \times 0.06 = 0.024\ N$$

This force is insignificant in comparison with the total system weight ($m_{total} \cdot g = 64.7\ N$) — only about 0.037% of it. Lateral ZMP shift due to yaw motion does not affect the stability at the maximum angular speed of the designed mechanism [16] [17].

6.5.4. Dynamic Stability Summary

Configuration	Static $CoM_x(mm)$	Dynamic ZMP_x, breaking (mm)	Margin to front axle (mm)	Dynamic SF
A	28.0	32.5	83.0	**1.78**
B	60.0	63.6	51.9	**3.45**
C	65.3	68.9	46.6	**3.96**
D	55.7	59.4	56.1	**3.12**
E	60.5	64.3	51.2	**3.51**

Table 6-4: Dynamic (braking) ZMP position and safety factor for all configurations.

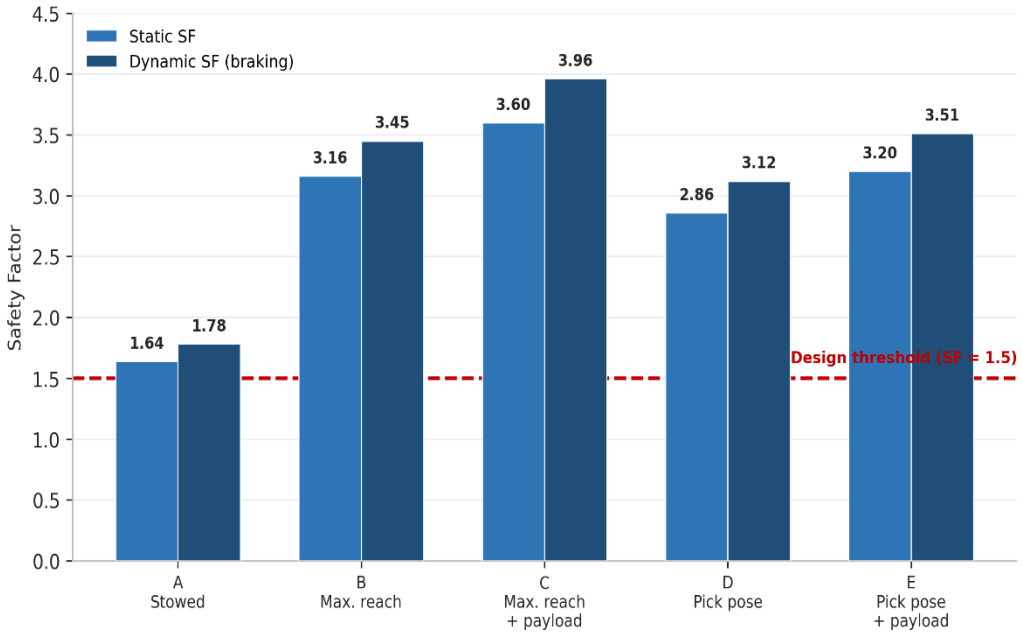

Figure 6-4: Static vs. dynamic (emergency-braking) safety factor for all five evaluated configurations, compared against the 1.5 design threshold. All five configurations exceed the threshold with comfortable safety margins.

6.6. Structural Reasoning — Why the Platform Resists Tip-Over

The combination of three design parameters ensures that the front-mounted 4-DOF parallelogram arm is not sacrificing stability even though it projects itself in front of the support polygon.

- **Heavy base compared to the arm ($3.1:1$ mass ratio):** The heavy base (5.0 kg) overshadows the weighted CoM average. Any contribution of the arm (1.6 kg) in the x direction is reduced fourfold in the CoM average due to the base which does not project in the x direction.
- **Shoulder-based kinematics ensure that the chain remains compact:** since the shoulder defines the angle of the entire chain, the independent variable (along with an additional elbow correction of a relatively small distance, enabled by the parallelogram) determines how much distance the combined mass of the chain can travel. Although the total reach of the chain (0.30 m) is significantly large in comparison with the half-length of the base (0.1155 m), considering the small mass of the links along with the majority of the system mass remaining near the axis, the resultant displacement of the center of mass is just 60 mm (52%).
- **Large four-wheel footprint**: The base dimensions of $231 \times 192\ mm$ provide a large support polygon. A narrower or two-wheel base of the same weight would have a much shorter half-length, decreasing all the safety factors calculated here.

6.7. Active Safety Layer — Controller-Level Safeguards

Although the stability analysis done above proves that the hardware will always be stable within its operating space, the controller itself provides an additional and active safety feature. This is important since, with an untrained or suboptimal control policy, it may be possible to have configurations that are not technically tipping the actual hardware but are undesirable all the same [8] [9] [10] [16].

6.7.1. ZMP / CoM Monitoring

During every control cycle, the center of mass of the entire system is recalculated from the latest joint angles and compared with the conservative stability boundary. The conservative stability boundary (0.18 to $0.24\ m$) used by the robot internally is larger than the support polygon boundary ($0.1155\ m$) determined in this analysis.

6.7.2. Reward Shaping for Stability

The second highest individual penalty in the learning-based control scheme after the placement bonus for completing the task is that of a tip over, and violation of joint limits receives an additional penalty. Stability penalties are thus heavily discouraged as compared to the dense rewards received for reaching and grasping the object.

6.7.3. Action and Joint Constraints

- Base linear velocity is clipped to ±0.3 m/s, bounding the deceleration magnitude used throughout Section 6.5.
- Base angular velocity is clipped to ±0.5 rad/s, bounding the centrifugal loading discussed in Section 6.5.3.

- Joint angles are hard-clipped at their physical limits, with a 5° soft-penalty buffer before saturation.

Safety Layer	Mechanism	Threshold / Value	Function
Static stability	ZMP inside support polygon	SF≥ 1.5 for all reachable poses	Full-system CoM recomputed every control step
Dynamic stability	ZMP inside polygon under braking	SF≥ 1.5 maintained during deceleration	Large termination penalty on boundary violation
Joint limits	Hard clipping + soft penalty	±5° tolerance band before hard limit	Penalty applied per violation
Velocity saturation	Action clipping	$v_x \leq 0.3$ m/s, $\omega \leq 0.5$ rad/s	Applied before actuation
Smooth motion	Action regularization	Quadratic penalty on action magnitude	Limits jerky commands and inertial ZMP shift
Timeout protection	Episode termination	Maximum 200 steps (10 s @ 20 Hz)	Prevents runaway episodes

Table 6-5: Integrated safety mechanisms active during training and deployment.

7. Reinforcement Learning Controller — Theoretical Background

The previous chapters presented the physical system that needed to be controlled, namely, a 4-DOF parallelogram arm attached to a 4-wheel mobile base which performs pick-and-place operations automatically by reaching out to the desired object, picking it up, and placing it at the goal location. In this chapter, the RL-based methodology for designing the controller is discussed; later, in Chapters 8 to 10, the environment, reward function, and neural network architecture are discussed in detail, respectively.

7.1. Introduction to Reinforcement Learning

Reinforcement learning belongs to the class of machine learning where a self-regulating agent learns to perform a sequence of actions based on the evolving state of the environment. While supervised learning involves training using datasets of labeled inputs and outputs, reinforcement learning is achieved through a trial-and-error process, where the agent first perceives the current state of the environment, performs an action, gets a numerical reward feedback, and moves to a new state. Iteration of this process leads to the development of a policy, which is a rule of mapping between states and actions that brings maximum rewards.

RL is appealing for use in robotics because it is capable of working directly with high-dimensional and continuous state-action spaces, complex nonlinear control policies may be easily captured without requiring manual specification, and it is possible to avoid explicitly modeling the dynamics of the environment, thus making RL a potential replacement for classical control approaches in tasks where precise modeling is challenging, like contact-rich grasping tasks.

7.2. Fundamentals of Reinforcement Learning

7.2.1. Core Concepts and Terminology

RL model comprises five components. The agent refers to the decision-making component – in this project, the trained neural network-based controller that governs the behavior of the mobile manipulator. The environment comprises all components outside the agent's system – robot's mechanics, working space, and goal object. The state, s, is a numeric vector representing the current state of the environment. Action, a, denotes the command sent by the agent – in our case, joint velocities for the robot arm and linear/rotational velocities for the robot base. Reward, r, is a scalar value indicating the desirability of the action performed.

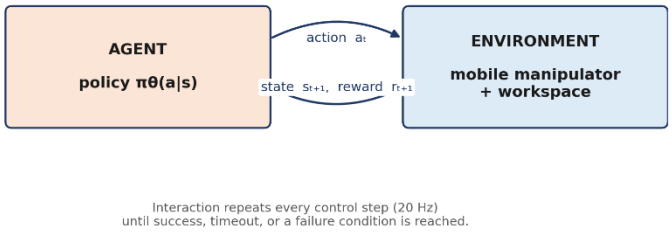

Figure 7-1: The agent–environment interaction loop. At every control step the agent observes the current state, selects an action, and receives the next state and a reward from the environment.

A policy, π, describes the agent's behavior: for each state s it defines either a probability distribution over actions, $\pi(a|s)$, or a deterministic mapping $a = \mu(s)$. Training seeks the optimal policy π^* that maximizes the expected return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1} \quad (2.1)$$

where $\gamma \in [0,1[$ is the discount factor, which balances immediate against future rewards: values close to 1 make the agent far-sighted, while values close to 0 make it myopic.

7.2.2. The Markov Decision Process Framework

RL problems are formally represented as Markov Decision Processes (MDPs), defined by the tuple (S, A, P, R, γ): S is the set of states, A is the set of actions, $P(s'|s,a)$ is the state-transition probability, $R(s,a,s')$ is the reward function, and γ is the discount factor. The Markov property states that the next state depends only on the current state and action, not on the full history that preceded them.

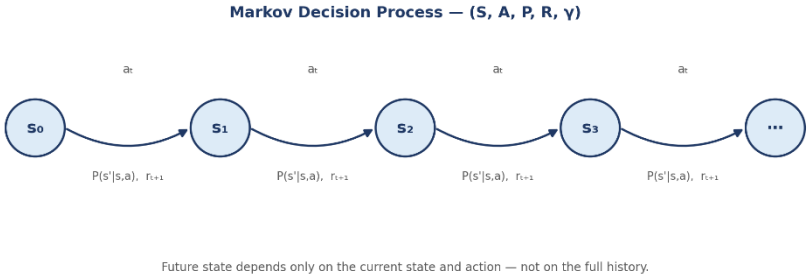

Figure 7-2: The Markov decision process viewed as a chain of states linked by actions, transition probabilities, and rewards. The next state depends only on the current state and action.

Table 7-1 maps each element of the formal MDP onto its concrete meaning for the mobile-manipulator task, as implemented in the environment described in Chapter 8.

Symbol	Meaning	Mobile-Manipulator Mapping
S	State space	Joint angles, joint velocities, end-effector position, base pose, target coordinates, gripper state
A	Action space	Joint-velocity commands for the arm + linear/angular velocity commands for the base
$\boldsymbol{P(s'\|s,a)}$	Transition dynamics	Robot physics (Simulation / real hardware)
$\boldsymbol{R(s,a,s')}$	Reward function	Distance reward + grasp bonus + place bonus + safety penalties
$\boldsymbol{\gamma}$	Discount factor	0.99 (far-sighted; pick-and-place is a long-horizon task)

Table 7-1: Mapping of the MDP framework onto the mobile-manipulator task.

7.2.3. Value Functions and the Bellman Equation

The state-value function $V_\pi(s)$ is the expected return obtained from state s under policy π. The action-value function (Q-function), $Q_\pi(s,a)$, generalizes this by conditioning on the action taken in states. The two are related by:

$$V_\pi(s) = \sum_a \pi(a|s) \cdot Q_\pi(s,a)$$

$$Q_\pi(s,a) = \mathbb{E}[r + \gamma V_\pi(s')] \quad \text{(Bellman equation)}$$

The Bellman equation is the recursive relationship that every RL algorithm uses, in one form or another, to estimate value functions. Temporal-difference (TD) methods — including those underlying PPO and SAC — exploit this recursion to update value estimates from individual transitions, without requiring complete episode trajectories.

7.2.4. Policy Gradient Methods

Policy-gradient methods directly optimize the policy parameters θ by estimating the gradient of the expected return with respect to θ and performing gradient ascent. The policy-gradient theorem gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi[\nabla_\theta log\ \pi_\theta(a|s) \cdot Q_\pi(s,a)]$$

Here $\nabla_\theta log\ \pi_\theta(a|s)$ is the score function of the policy, and $Q_\pi(s,a)$ weights actions according to how much reward they tend to produce. This theorem underlies algorithms such as REINFORCE, PPO, and SAC. To reduce the variance of the gradient estimate, Q_π is typically replaced by the advantage function:

$$A(s,a) = Q(s,a) - V(s)$$

7.2.5. Actor-Critic Architecture

The actor-critic architecture combines policy-gradient learning with value-function approximation. The actor is the policy network $\pi_\theta(a|s)$ that selects actions; the critic is the value network $V_\varphi(s)$ (or $Q_\varphi(s,a)$) that evaluates how good the current state, or state–action pair, is. The critic's estimate is used to compute the advantage function, which in turn guides the actor's parameter updates. This architecture is the foundation of both PPO and SAC — the two algorithms evaluated in this project [8].

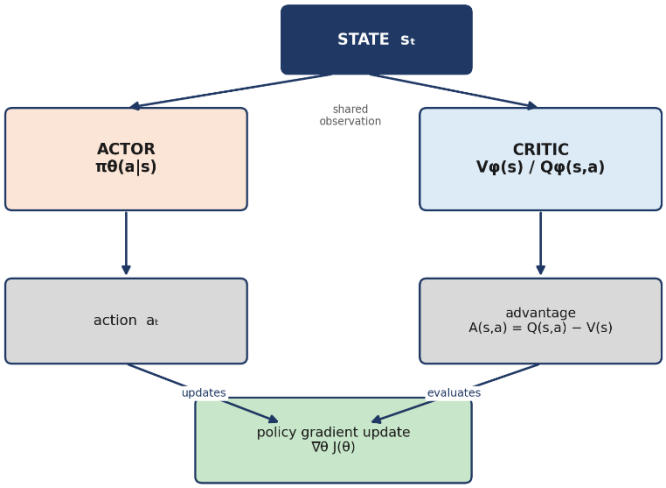

Figure 7-3: Actor-critic architecture: the actor selects an action from the current state while the critic evaluates that state to compute an advantage estimate, which is then used to update the actor.

7.3. RL Algorithms — Comparative Study

7.3.1. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO), proposed by Schulman et al. (2017), is an on-policy policy-gradient algorithm that addresses the training instability of earlier methods such as TRPO by constraining the size of each policy update. PPO optimizes a clipped surrogate objective:

$$L_{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\cdot\hat{A}_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\cdot\hat{A}_t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio between the new and old policy, $\hat{A}_t$ is the estimated advantage, and ε is the clipping parameter (typically 0.1–0.2). Clipping prevents excessively large policy updates that could destabilize training. Because PPO is on-policy, it must collect fresh experience under the current policy before every update, which makes it more sample-intensive than off-policy alternatives but generally more stable [18].

Hyperparameter	Value
Clipping parameter ε	0.2
Discount factor γ	0.99
GAE λ (generalized advantage estimation)	0.95
Mini-batch size	64 transitions

Epochs per update	10
Learning rate (actor / critic)	$3 \times 10^{-4} / 1 \times 10^{-4}$

Table 7-2: PPO key hyperparameters.

7.3.2. Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) by Haarnoja et al. (2018) is an off-policy, maximum-entropy RL algorithm. SAC modifies the conventional reward function with an entropy term H(π(·|s)) to ensure that the policy remains stochastic while maximizing the expected return:

$$J(\pi) = \sum_{t} \mathbb{E}_{(s_t, a_t) \sim \pi} \left[r(s_t, a_t) + \alpha \cdot H(\pi(\cdot \,|s_t)) \right]$$

The α term determines the balance between maximizing rewards and entropy, and it is optimized adaptively based on some target entropy level. SAC utilizes two Q functions to prevent value overestimations, collects samples from experience replay memory, and updates the target Q functions with the use of Polyak averaging. Since SAC reuses samples collected previously using the experience replay mechanism, it is much more sample efficient compared to PPO and thus preferred when simulation cost is the bottleneck [19].

7.3.3. Deep Deterministic Policy Gradient (DDPG)

DDPG is an off-policy, deterministic actor-critic algorithm that extends DQN to continuous action spaces through a deterministic policy $\mu_\theta(s)$ combined with an experience-replay mechanism. In practice, DDPG is known to be sensitive to its hyperparameters and prone to unstable training, particularly in high-dimensional, contact-rich manipulation tasks. For this reason, SAC is preferred over DDPG in this project [20].

7.3.4. Algorithm Selection Rationale

Criterion	PPO	SAC	DDPG
Policy type	Stochastic (on-policy)	Stochastic (off-policy)	Deterministic (off-policy)
Sample efficiency	Low	High (replay buffer)	High (replay buffer)
Training stability	High (clipped updates)	High (entropy regularization)	Low (hyperparameter-sensitive)
Continuous actions	Yes	Yes	Yes
Recommended for manipulation	Yes	Yes (preferred among off-policy)	Not recommended

Table 7-3: Qualitative comparison of PPO, SAC, and DDPG for this task.

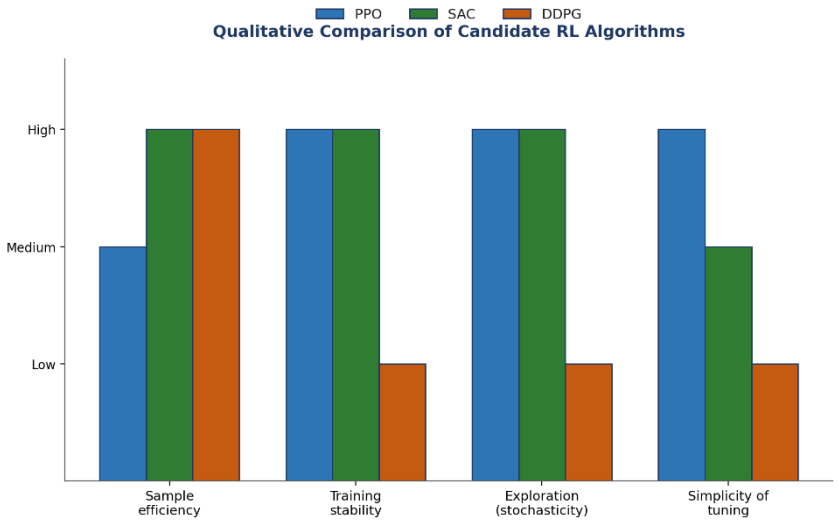

Figure 7-4: Qualitative comparison of PPO, SAC, and DDPG across the criteria most relevant to this project. PPO and SAC dominate DDPG on training stability without sacrificing support for continuous actions.

The choice of PPO as the main algorithm in the work was driven by its stable learning and tuning process, which would be beneficial for the project given the limited number of simulation hours available for the training process. The second algorithm that is used in the work is SAC, which is more sample-efficient compared to the other algorithms that have been tried so far.

8. RL Environment Design

With the algorithm chosen (Chapter 7), this chapter defines the reinforcement learning environment, that is, the state vector available to the agent, the action vector it generates, and when the episode terminates. The reward function will be built on top of the same state and action definitions in Chapter 9.

8.1. State Space

The agent observes a fully observable state vector $s \in \mathbb{R}^{14}$ (partial observability is not considered in this design, consistent with the MDP formulation of Section 7.2.2). All fourteen components are normalized to the range $[-1, +1]$ before being passed to the policy network.

Index	Variable	Symbol	Range	Description
$\boldsymbol{s[1:3]}$	Joint angles	q_1, q_2, q_3	$[-\pi, +\pi]$ rad	Current arm configuration
$\boldsymbol{s[4:6]}$	Joint velocities	$\dot{q}_1, \dot{q}_2, \dot{q}_3$	$[-2, +2]$ rad/s	Arm joint-velocity feedback
$\boldsymbol{s[7:9]}$	End-effector position	p_x, p_y, p_m	$[-1, +1]$ m	EE Cartesian position (base frame)
$\boldsymbol{s[10:11]}$	Base pose	x^b, θ^b	Workspace bounds	Base x-position and yaw angle
$\boldsymbol{s[12:13]}$	Relative target	$\Delta x, \Delta y$	$[-2, +2]$ m	Vector from end-effector to target
$\boldsymbol{s[14]}$	Gripper state	g	$\{0, 1\}$	0 = open, 1 = closed

Table 8-1: Composition of the 14-dimensional state vector.

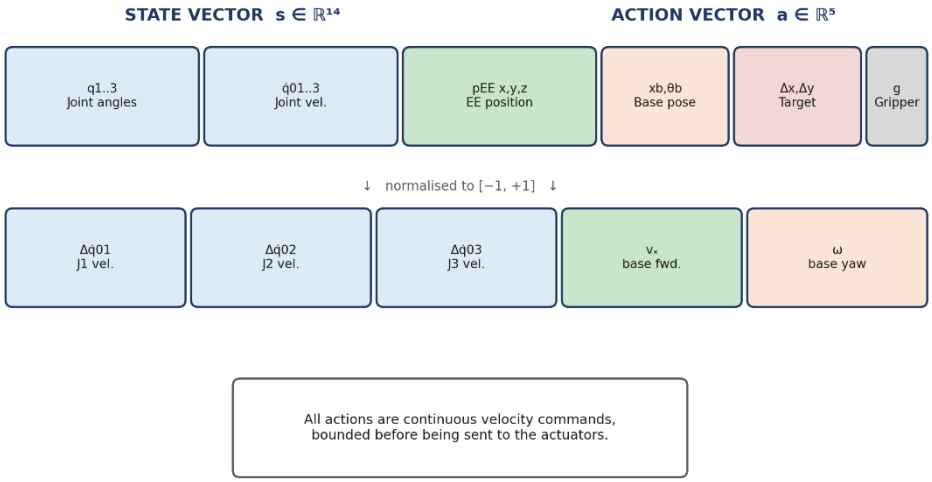

Figure 8-1: Composition of the 14-dimensional state vector and the 5-dimensional action vector. Every component is normalized to [−1, +1] before being seen by the policy network.

The decision was made to use velocity control, rather than position control, at all times due to smoother paths of the end effector and robustness towards modelling errors – a characteristic which becomes relevant when the simulation policy is transferred to the real robot.

8.2. Action Space

The action vector a ∈ $\mathbb{R}^5$ is continuous and expressed in velocities rather than positions, for the same reason given above. Every action is bounded before being sent to the corresponding actuator.

Index	Variable	Symbol	Range	Maps to
$\boldsymbol{a[1]}$	Joint 1 velocity	$\Delta\dot{q}_1$	[−1.5, +1.5] rad/s	Shoulder servo command
$\boldsymbol{a[2]}$	Joint 2 velocity	$\Delta\dot{q}_2$	[−1.5, +1.5] rad/s	Elbow servo command
$\boldsymbol{a[3]}$	Joint 3 velocity	$\Delta\dot{q}_3$	[−2.0, +2.0] rad/s	Wrist servo command
$\boldsymbol{a[4]}$	Base forward velocity	v_x	[−0.3, +0.3] m/s	Base linear velocity
$\boldsymbol{a[5]}$	Base yaw rate	ω	[−0.5, +0.5] rad/s	Base angular velocity

Table 8-2: Composition of the 5-dimensional action vector.

Note that the joint-velocity bounds and link assignments in Table 8-2, match the manipulator joint structure described in Section 4.1 and the actuator configuration described in Section 12.4 and that the base-velocity bounds are consistent with the non-holonomic, two-input $[v_x, \omega]$ control used in the skid-steer kinematic model described in Section 5.3.

8.3. Episode and Termination Conditions

An episode starts off with the robot being posed at some randomly chosen pose in the workspace, while the object of interest is located at some randomly chosen location on the pick table. The episode ends based on any of the following criteria:

- **Success:** the object has been placed within 0.05 m of the centroid of the goal-position.
- **Timeout:** after exceeding 200-time steps (approximately 10 s, at 20 Hz).
- **Failure — Joint Limit Exceeded:** a joint angle surpasses the physical limit of the joint according to the predefined joint-angle limits by more than 5°.

- **Failure — Tip-over risk:** center of mass projected outside the support polygon defined by four wheels.
- **Failure — Collision:** arm links collide with itself or the environment, or the base of the robot collides with the environment or itself.

200-time step episode duration limit equals ten seconds of actual time at the 20 Hz control rate. It's enough time to execute the entire navigating-reaching-grasping-placing defined in the RL task design, which usually takes 80-to-130-time steps to complete [8] [9] [10].

9. Reward Function Design

The reward function is perhaps the most important design component for the whole of the RL setup; an ill-designed reward function might lead to reward hacking, slow convergence, or even the inability of the agent to learn the desired behavior. The reward function used in this project takes the shape of dense shaping with sparse milestone bonuses based on the four stages of the task defined in chapter 8.

9.1. Reward Components

Eight components are combined into the total reward. Dense components are evaluated at every time step and continuously guide the agent; sparse components are awarded once, at a specific milestone.

$$r_{reach} = -\left\|p_{EE} - p_{target}\right\|^2 \quad \text{(dense — minimizes EE-to-object distance)}$$

$$r_{orient} = -0.1 \times \|\theta_{EE} - \theta_{desired}\| \quad \text{(dense — aligns gripper orientation)}$$

$$r_{grasp} = +100.0 \text{ when grasp contact is detected (sparse milestone)}$$

$$r_{carry} = -0.5 \times \left\|p_{EE} - p_{goal}\right\|^2 \quad \text{(dense, after grasp — minimizes EE-to-goal distance)}$$

$$r_{place} = +200.0 \text{ when the object is placed at the goal (sparse — largest reward)}$$

$$r_{limit} = -20.0 \text{ per joint-limit violation (safety penalty)}$$

$$r_{tip} = -50.0 \text{ when the center of mass exits the support polygon (stability penalty)}$$

$$r_{smooth} = -0.005 \times \|a\|^2{}_2 \quad \text{(action regularization — penalizes jerky motion)}$$

The total reward at each step is the sum of all eight components:

$$R(s,a) = r_{reach} + r_{orient} + r_{grasp} + r_{carry} + r_{place} + r_{limit} + r_{tip} + r_{smooth}$$

Component	Value / type	Purpose
$\boldsymbol{r_{reach}}$	$-\left\|p_{EE} - p_{target}\right\|^2$ (dense)	Continuously guides the end-effector toward the object
$\boldsymbol{r_{orient}}$	$-0.1 \times \|\text{angle error}\|$ (dense)	Aligns the gripper for a successful grasp
$\boldsymbol{r_{grasp}}$	$+100$ (sparse)	Strong incentive to attempt grasping at the right moment

r_{carry}	$-0.5 \times \|p_{EE} - p_{goal}\|^2$ (dense)	Guides the robot toward the goal after a successful grasp
r_{place}	+200 (sparse)	Largest bonus — rewards full task completion
r_{limit}	−20 per violation (sparse)	Enforces the joint-angle safety limits of Table 1.10
r_{tip}	−50 per violation (sparse)	Enforces the base-stability constraint of Section 1.4
r_{smooth}	$-0.005 \times \|a\|^2{}_2$ (dense)	Encourages smooth, low-energy, low-wear motion

Table 9-1: Summary of all reward components and their purpose.

9.2. Reward Shaping Strategy

The reward function incorporates a form of shaping similar to that of a curriculum, based on the four phases of the pick-and-place task sequence described in Chapter 8. In early stages of each episode, the high-frequency reaching reward r_{reach} guides the end-effector towards the target. Upon successful reaching of the target, the grasp bonus r_{grasp} provides a large incentive for closing the gripper at the right time. Upon successful grasping, the carry bonus r_{carry} guides the

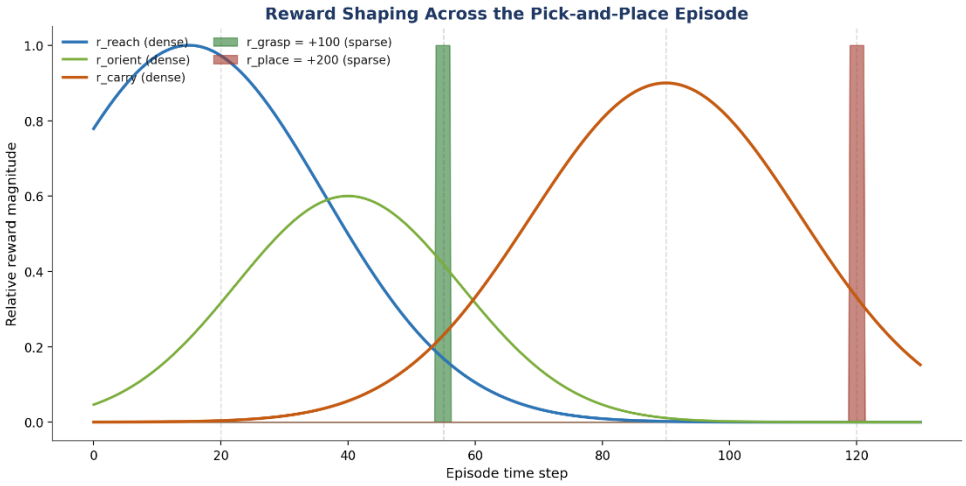

Figure 9-1: Conceptual reward shaping across an episode. Dense rewards (reach, orient, carry) dominate within their respective phases, while the sparse grasp and place bonuses fire as short, large spikes at the corresponding milestones.

robot's actions towards the goal, and finally the place bonus r_{place}, which is the largest term in the reward function, strongly incentivizes completing the whole task [8] [9] [10].

The action-regularization term r_{smooth} penalizes large joint-velocity commands, encouraging smooth, physically plausible motion and discouraging jerky or oscillatory behavior that could damage the hardware structure described in Chapter 12. The tip-over penalty r_{tip} reflects the same stability criterion introduced in Chapter 6: once the projected center of mass leaves the support polygon formed by the four wheels, the episode ends and the agent receives a large negative reward, reinforcing the importance of keeping the arm's reach compatible with the base's stability margin.

10. Neural Network Architecture

Both the actor and the critic are implemented as compact multilayer perceptrons (MLPs) with two hidden layers each. The networks were deliberately kept small so that inference remains fast on the embedded Raspberry Pi 4 processor described in Section 12.5 and Table 12-3, which performs all on-board policy inference for the physical prototype [8] [9] [10].

Network	Input	Hidden layers	Output	Output activation
Actor (PPO)	$s \in \mathbb{R}^{14}$	$64 \rightarrow 64$ (tanh)	$\mu \in \mathbb{R}^5 + log\sigma$	tanh
Critic (PPO)	$s \in \mathbb{R}^{14}$	$64 \rightarrow 64$ (tanh)	$V(s) \in \mathbb{R}$	linear
Actor (SAC)	$s \in \mathbb{R}^{14}$	$256 \rightarrow 256$ (ReLU)	$\mu, \sigma \in \mathbb{R}^5$	tanh-squashed
Q-net 1 (SAC)	$[s, a] \in \mathbb{R}^{19}$	$256 \rightarrow 256$ (ReLU)	$Q(s, a) \in \mathbb{R}$	linear
Q-net 2 (SAC)	$[s, a] \in \mathbb{R}^{19}$	$256 \rightarrow 256$ (ReLU)	$Q(s, a) \in \mathbb{R}$	linear

Table 10-1: Actor and critic network architectures for PPO and SAC.

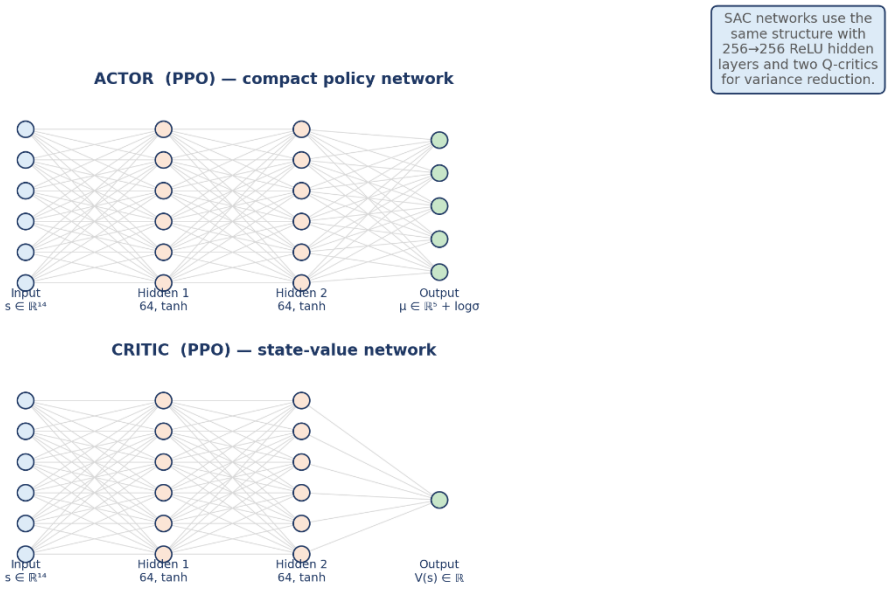

Figure 10-1: Actor and critic network architectures for PPO. Both take the 14-dimensional state vector as input; the actor outputs a Gaussian policy over the 5-dimensional action space, while the critic outputs a single scalar state-value estimate.

The reason that tanh is applied across all the PPO architectures is that it is bounded and smooth – characteristics which help policy stability. The output of the actor is the mean μ and the log standard deviation $\log \sigma$ of each action dimension, creating a Gaussian distribution for sampling actions during training, while at the application stage the deterministic mean μ is used, getting rid of the noise of exploration needed just during training. The networks for SAC have large 256-neuron hidden layers since the off-policy experience replay from Section 7.3.2 provides much more varied input, allowing for a bigger architecture to converge properly.

Initial weight setup is performed following the common procedure for this type of architecture, where the hidden layers are initialized using orthogonal initialization with gain $\sqrt{2}$, and the final output layer is initialized with the smaller gain of 0.01. This ensures that the initial policy remains close to uniform to avoid converging to the wrong local optimum too soon.

11.YOLO-Based Perception Integration for Real-Time Object Detection

11.1. Vision-Based Perception

Autonomous pick-and-place operation requires not only an effective low-level motion controller, but also a reliable perception module capable of locating the target object in the robot workspace. In the previous reinforcement learning environment, the target position was represented by a predefined or randomly generated variable inside the simulation model. Although this formulation is sufficient for controller training and algorithmic validation, it does not fully represent the conditions encountered during real hardware deployment, where the object position must be estimated from sensor data in real time.

For this reason, a YOLO-based vision module has been incorporated into the control structure of the mobile manipulator. The aim of such module is to identify the target object from the frames captured by an RGB camera, calculate its position in 3D space through the corresponding depth image and output its location to the reinforcement learning controller. In such a setting, the YOLO detector acts as a perceptual layer, while the learned reinforcement learning policy is still in charge of decision making and actions.

Consequently, the main goal of incorporating the YOLO model is to replace the pre-defined target locations with those that have been found through object localization in real time. This change enables the mobile manipulator to work in more unstructured settings, where the object can be located differently on the table within the reachable workspace. The final system constitutes a closed-loop of perception and control, where visual inputs are constantly transformed into target positions and further utilized by the RL controller [16].

11.2. Fundamentals of Object Detection

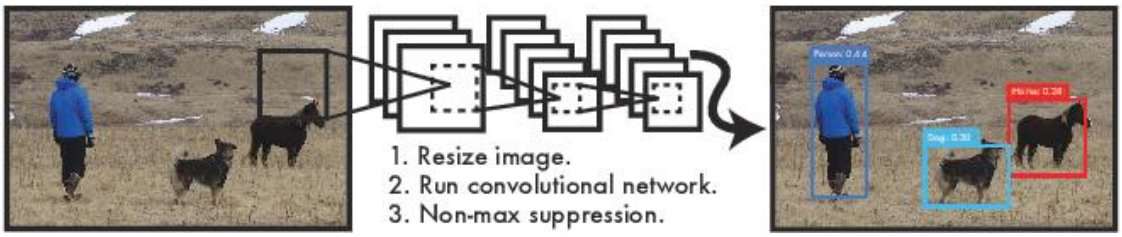

Figure 11-1: The YOLO Detection System.

Object Detection can be explained as a problem of computer vision where certain objects that have been detected are localized inside an image. The difference between image classification and object detection is that while the former gives a class label for an image, the latter gives class predictions as well as spatial localization of these objects. The spatial localization of the object is defined using:

$$b = (u_c, v_c, w, h)$$

where u_c and v_c are the pixel coordinates of the bounding box centre, while w and h are the bounding box width and height in image coordinates. For each detected object, a class label and confidence score are also produced:

$$d_i = (c_i, p_i, b_i)$$

where c_i is the predicted class, p_i is the confidence score, and b_i is the bounding box.

In robotic manipulation, object detection is used to answer three essential questions:

1. whether the target object is visible;
2. where the target object is located in the image plane;
3. whether the detected object is suitable for grasping.

However, a two-dimensional bounding box is not sufficient for robotic control because the manipulator requires metric coordinates in the robot base frame. Therefore, RGB detection must be combined with depth sensing and camera calibration. The role of YOLO is limited to visual detection, while the RGB-D camera and coordinate transformation pipeline are used to convert the 2D detection into a 3D target point [21] [22] [23] [24].

11.3. YOLO Algorithm Overview

YOLO, an abbreviation for "You Only Look Once", is a single-stage object detection framework. The original YOLO formulation frames object detection as a direct regression problem, where bounding boxes and class probabilities are predicted from the full image using a single convolutional neural network. This differs from two-stage detection approaches, which first generate candidate regions and then classify them. The single-stage formulation makes YOLO particularly suitable for real-time robotic applications, where perception latency must remain low [21].

In the YOLO architecture, the input image is divided into grids. The prediction of bounding box and class probabilities is done by each grid cell where the center of the objects lies inside the cell. The objectness score, bounding box coordinates, and class probabilities are estimated for each predicted bounding box. The detection score formula can be given as:

$$S(c, b) = P(object) \times P(c|object)$$

where $P(object)$ represents the probability that an object exists inside the predicted box, and $P(c|object)$ represents the conditional probability of class c given the presence of an object.

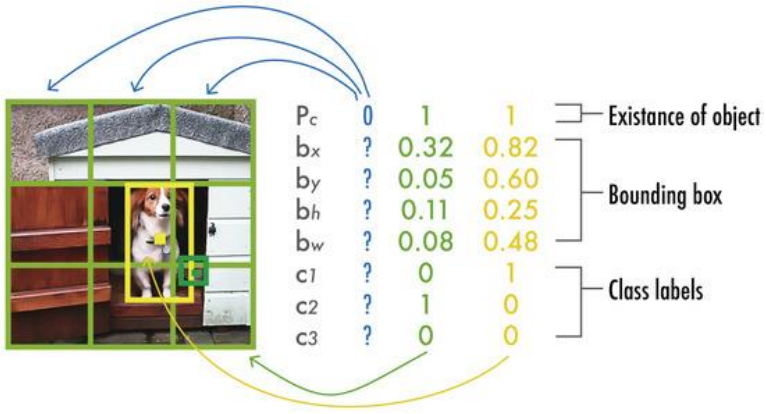

Figure 11-2: YOLO output prediction. The figure depicts a simplified YOLO model with a three-by-three grid, three classes, and a single class prediction per grid element to produce a vector of eight values.

YOLO improvements include anchor boxes, multiple scale feature maps, more efficient backbones, and non-maximum suppression. Anchor boxes enable the network to predict different aspect ratio and sized objects. Multiple scale prediction enables better detection of smaller and larger objects [25] [26]. The non-maximum suppression technique eliminates duplicates by keeping only the most confident bounding box while suppressing those overlapping boxes whose IoU exceeds the chosen threshold.

Figure 11-3: Non-maximum suppression (NMS). (a) Typical output of an object detection model containing multiple overlapping boxes. (b) Output after NMS.

The intersection over union (IoU) of the predicted bounding box B_p and ground truth bounding box B_g is given as:

$$IoU = \frac{Area(B_p \cap B_g)}{Area(B_p \cup B_g)}$$

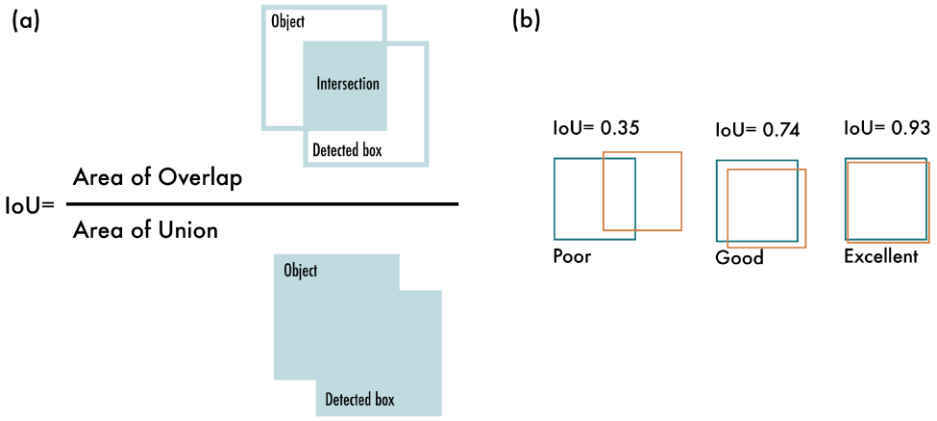

Figure 11-4: Intersection over Union (IoU). (a) The IoU is calculated by dividing the intersection of the two boxes by the union of the boxes; (b) examples of three different IoU values for different box locations.

IoU is used during both training and evaluation. A detection is usually considered correct when its predicted class is correct and the IoU with the corresponding ground-truth box exceeds a selected threshold [21] [27].

11.4. Role of YOLO in the Mobile Manipulator Architecture

The YOLO perception module is integrated between the RGB-D camera and the reinforcement learning environment. The detector does not directly control the robot actuators. Instead, it provides the RL controller with the target object coordinates required for state construction and reward computation.

The perception-control pipeline is structured as follows:

$$I_{RGB}, I_D \rightarrow YOLO \rightarrow b_{target} \rightarrow p_C \rightarrow p_B \rightarrow s_t \rightarrow \pi(a_t|s_t)$$

where I_{RGB} is the RGB image, I_D is the depth image, b_{target} is the selected target bounding box, p_C is the estimated object position in the camera frame, p_B is the object position transformed into the robot base frame, s_t is the RL state vector, and $\pi(a_t|s_t)$ is the learned policy.

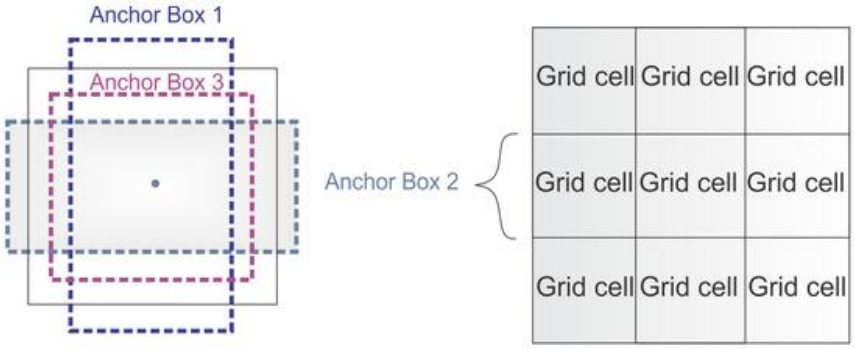

Figure 11-5: Anchor boxes. YOLOv2 defines multiple anchor boxes for each grid cell.

The main functional relationship between YOLO and the RL controller is summarized in Table 11-1.

Module	Input	Output	Function in the System
RGB-D camera	Workspace scene	RGB image + depth map	Provides visual and depth measurements
YOLO detector	RGB image	Class, confidence, bounding box	Detects target object in image plane
Depth extraction	Bounding box + depth map	Metric depth (Z)	Estimates object distance from camera
Camera projection	Pixel center + depth	3D point in camera frame	Converts 2D detection into 3D coordinates
Frame transformation	Camera-frame point	Base-frame point	Expresses object position relative to robot

RL controller	State vector including target position	Velocity commands	Executes navigation, reaching, grasping, and placing

Table 11-1: Functional relationship between YOLO perception and RL control.

In the original RL formulation, the target position was treated as an internal environment variable. After YOLO integration, this variable is updated from the perception module:

$$p_{target} = p_B = [x_B, y_B, z_B]^T$$

This position is then used to compute the relative target vector included in the state representation:

$$\Delta p = p_{target} - p_{EE}$$

where p_{EE} is the end-effector position obtained from forward kinematics. Therefore, YOLO affects the observation vector provided to the RL agent, but the action space remains unchanged. The learned policy continues to output the same five continuous commands: three arm joint velocities, base forward velocity, and base yaw rate.

11.5. RGB-D Camera-Based 3D Localization

YOLO provides object localization in image coordinates only. To perform physical grasping, the bounding box center must be converted into a three-dimensional point. This conversion is performed using the depth image and camera intrinsic parameters.

For a detected bounding box:

$$b = (u_c, v_c, w, h)$$

the center pixel is computed as:

$$u_c = \frac{u_{min} + u_{max}}{2}$$

$$v_c = \frac{v_{min} + v_{max}}{2}$$

The corresponding depth value (Z) is extracted from the aligned depth image. To reduce sensitivity to sensor noise, the median depth inside a small region around the bounding box center is used instead of a single pixel measurement:

$$Z = median(I_D[u_c - r : u_c + r, ; v_c - r : v_c + r])$$

where r is the radius of the local depth window.

Using the pinhole camera model, the 3D position of the object center in the camera coordinate frame is calculated as:

$$X_C = \frac{(u_c - c_x)Z}{f_x}$$

$$Y_C = \frac{(v_c - c_y)Z}{f_y}$$

$$Z_C = Z$$

where f_x and f_y are the focal lengths in pixels, and c_x, c_y are the optical center coordinates. These intrinsic parameters are obtained through camera calibration [22].

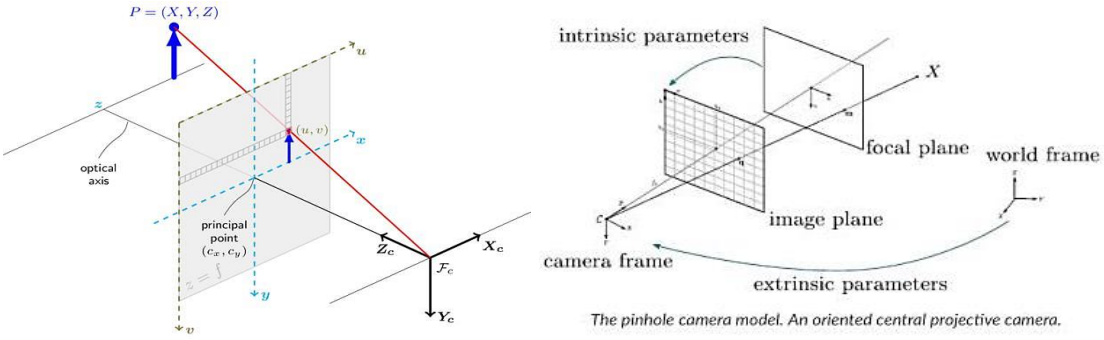

Figure 11-6: Camera Intrinsic and Extrinsic Parameters in the Pinhole Projection Model.

The resulting camera-frame point is:

$$p_C = [X_C, Y_C, Z_C, 1]^T$$

To use this point for robot control, it must be transformed into the robot base frame using the calibrated homogeneous transformation matrix:

$$p_B = T_B^C ; p_C$$

where T_B^C is the transformation from the camera frame to the robot base frame. This transformation is obtained through extrinsic calibration or hand-eye calibration, depending on whether the RGB-D camera is fixed to the environment or mounted on the robot [23].

For a fixed camera mounted relative to the robot base, the transformation matrix remains constant:

$$T_B^C = \begin{bmatrix} R_B^C & t_B^C \\ 0 & 1 \end{bmatrix}$$

where R_B^C is the rotation matrix and t_B^C is the translation vector from the camera frame to the base frame.

11.6. Raspberry Pi Camera Module 3-Based Monocular RGB Localization

The RGB-D localization procedure described in the previous subsection represents the most direct method for obtaining three-dimensional object coordinates because the depth value is measured explicitly from the depth frame. However, the implemented hardware configuration of the mobile manipulator may also operate with a Raspberry Pi Camera Module 3. In this case, the perception system is based on a monocular RGB camera rather than an RGB-D camera. The Raspberry Pi Camera Module 3 provides color image frames for visual detection, but it does not provide a depth image or direct metric distance measurements [28].

The Raspberry Pi Camera Module 3 is therefore suitable for YOLO-based object detection, since YOLO requires an RGB image as its input. The camera frame can be processed by the YOLO

detector to obtain the object class, confidence score, and two-dimensional bounding box. Nevertheless, the output of YOLO remains defined in image coordinates. The detected bounding box provides the target position in pixels, not a full three-dimensional pose in the robot base frame. Consequently, the use of a Raspberry Pi RGB camera requires an additional localization strategy when metric target coordinates are needed by the reinforcement learning controller.

For a detected object, YOLO produces a bounding box:

$$b = (u_{min}, v_{min}, u_{max}, v_{max})$$

The center of the detected object in image coordinates is calculated as:

$$u_c = \frac{u_{min} + u_{max}}{2}$$
$$v_c = \frac{v_{min} + v_{max}}{2}$$

where u_c and v_c are the horizontal and vertical pixel coordinates of the bounding box centre. The bounding box center can be used as a visual feature for aligning the robot with the target object. The horizontal image error is defined as:

$$e_u = u_c - c_x$$

where c_x is the optical centre of the camera. When e_u is positive, the object appears to one side of the image centre; when e_u is negative, the object appears to the opposite side. This visual error can be used to guide the yaw motion of the mobile base or to support image-based target alignment before grasping [24].

Unlike an RGB-D camera, the Raspberry Pi Camera Module 3 does not provide the depth variable (Z). Therefore, the full projection equations used in RGB-D localization cannot be applied directly unless depth is estimated from additional assumptions. A monocular camera can be used for approximate localization under one of the following conditions:

1. the object is placed on a known planar surface;
2. the physical size of the object is known;
3. the camera pose relative to the robot base is calibrated;
4. an external ranging sensor is used to estimate distance;
5. depth estimation is supplied by an additional algorithm or sensor.

For a tabletop pick-and-place task, a practical solution is to assume that the target object lies on a known horizontal plane. If the table height is known in the robot base frame, the image point corresponding to the bounding box center can be back-projected into a camera ray. The intersection between this ray and the known table plane gives an approximate target location. This method requires camera intrinsic calibration and extrinsic calibration between the camera frame and the robot base frame [22] [23].

The camera ray corresponding to the image point (u_c, v_c) is computed using the camera intrinsic matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

The normalized ray in the camera frame is obtained as:

$$r_C = K^{-1} \begin{bmatrix} u_c \\ v_c \\ 1 \end{bmatrix}$$

The ray is then transformed into the robot base frame using the calibrated camera-to-base transformation. If the object is assumed to lie on a plane:

$$\pi: n^T p_B + d = 0$$

where n is the plane normal and d is the plane offset, the target point can be estimated by intersecting the camera ray with this plane. This approach does not measure depth directly; instead, depth is inferred from the known workspace geometry.

A second approximation can be used when the physical width of the target object is known. If the object width is W in metres and its detected bounding box width is w in pixels, the approximate object distance can be estimated using the pinhole camera model:

$$Z \approx \frac{f_x W}{w}$$

Similarly, if the physical height of the object is known, the distance may be approximated as:

$$Z \approx \frac{f_y H}{h}$$

where H is the real object height and h is the detected bounding box height in pixels. This approximation is sensitive to object orientation, partial occlusion, bounding box error, and camera calibration accuracy. Therefore, it should be treated as an approximate range estimate rather than a precise 3D measurement.

In the reinforcement learning framework, the Raspberry Pi camera output can be integrated in two possible ways. The first method is image-based, where the RL state is extended to include visual errors such as e_u, e_v, bounding box area, and YOLO confidence. In this case, the controller does not require a full 3D object position. The second method is geometry-based, where the detected bounding box is converted into an approximate target position using calibration and workspace assumptions. The resulting estimate is then inserted into the same target-position components already used by the RL state vector.

For compatibility with the existing PPO policy, the geometry-based method is preferred. The estimated target point is expressed as:

$$\widehat{p_{target}} = [\widehat{x_B}, \widehat{y_B}, \widehat{z_B}]^T$$

where $\widehat{p_{target}}$ represents an approximate position obtained from monocular image localisation. The relative target vector is then computed as:

$$\Delta\hat{p} = \hat{p}target - p_{EE}$$

The same observation structure can therefore be retained, but the target coordinates must be interpreted as estimated values rather than direct depth-based measurements.

Several limitations must be considered when a Raspberry Pi Camera Module 3 is used instead of an RGB-D camera. Monocular RGB localization is affected by scale ambiguity, lighting variations, camera calibration error, perspective distortion, and object orientation. The accuracy of the estimated target position is generally lower than that obtained using an RGB-D camera. For this reason, the grasp tolerance, temporal filtering, confidence thresholding, and workspace validation steps remain necessary. The gripper should not be allowed to close unless the detected object satisfies the required confidence, workspace, and alignment conditions.

The Raspberry Pi Camera Module 3 is therefore sufficient for YOLO-based object detection, but not sufficient by itself for direct metric depth measurement. In the proposed implementation, it is used as an RGB sensing device for detecting the target object. Three-dimensional localization is then obtained only through calibration assumptions, known workspace geometry, object-size approximation, or additional sensing.

11.7. YOLO-Based Perception Pipeline

The perception pipeline is implemented as a sequential process executed at each camera frame. The output of the pipeline is the estimated 3D position of the selected target object. The complete procedure is given in Algorithm A.

Algorithm A: YOLO-Based Object Localization for RL Control

1. Acquire an RGB frame I_{RGB} and aligned depth frame I_D from the RGB-D camera.
2. Resize and normalize I_{RGB} according to the YOLO network input size.
3. Run YOLO inference on I_{RGB}.
4. Extract predicted bounding boxes, class labels, and confidence scores.
5. Apply confidence thresholding:
$$p_i \geq p_{conf}$$
6. Apply non-maximum suppression using IoU threshold T_{NMS}.
7. Select the target detection according to the required object class.
8. Compute the bounding box center (u_c, v_c).
9. Extract a robust depth value from the aligned depth image.
10. Convert the 2D detection into a 3D point in the camera frame using the pinhole camera model.
11. Transform the 3D point from camera frame to robot base frame.
12. Validate that the transformed point lies inside the robot workspace.
13. Apply temporal filtering to reduce jitter.
14. Publish the filtered target position to the RL controller.
15. Update the RL state vector using the measured relative target position.

The perception update rate does not necessarily have to match the low-level control frequency. The RL controller operates at 20 Hz, while YOLO inference may operate at a lower or similar frequency depending on the embedded hardware. When a new detection is unavailable at a given control step, the most recent valid target estimate is retained for a limited number of frames. If

the target is lost for longer than a predefined timeout, the robot is commanded to stop or return to a safe search posture.

Algorithm B: Monocular RGB YOLO Pipeline Using Raspberry Pi Camera Module 3

1. Acquire an RGB frame from the Raspberry Pi Camera Module 3.
2. Resize and normalize the image according to the YOLO input size.
3. Run YOLO inference on the RGB image.
4. Extract the bounding box, class label, and confidence score.
5. Apply confidence thresholding to remove weak detections.
6. Apply non-maximum suppression to remove duplicate detections.
7. Select the target object according to the required class.
8. Compute the bounding box center (u_c, v_c).
9. Compute the image-center errors e_u and e_v.
10. If metric localization is required, estimate the target position using one of the following:
    - calibrated ray-plane intersection with a known table plane;
    - approximate range from known object size;
    - external distance measurement from an additional sensor.
11. Transform the estimated target point into the robot base frame.
12. Validate that the estimated point lies within the reachable workspace.
13. Apply temporal filtering to reduce detection jitter.
14. Update the RL state vector using the estimated target position or image-based visual features.
15. Prevent grasp execution if the detection is invalid, unstable, or outside the workspace.

11.8. Dataset Preparation and Annotation

A custom dataset is required when the target objects are not sufficiently represented in general-purpose datasets or when the operating environment differs significantly from standard benchmark images. For the mobile manipulator, the dataset should include images of the target objects under conditions similar to the real workspace.

The dataset should contain variations in:

- object position on the table;
- distance from the camera;
- viewing angle;
- illumination intensity;
- background texture;
- partial occlusion by the gripper or arm;
- object orientation;
- object scale;
- clutter near the target object.

Images are annotated using bounding boxes around each object instance. The YOLO annotation format represents each bounding box using normalized values:

$$class; x_c; y_c; w; h$$

where x_c, y_c, w, and h are divided by the image width and height so that all values lie in the interval [0,1].

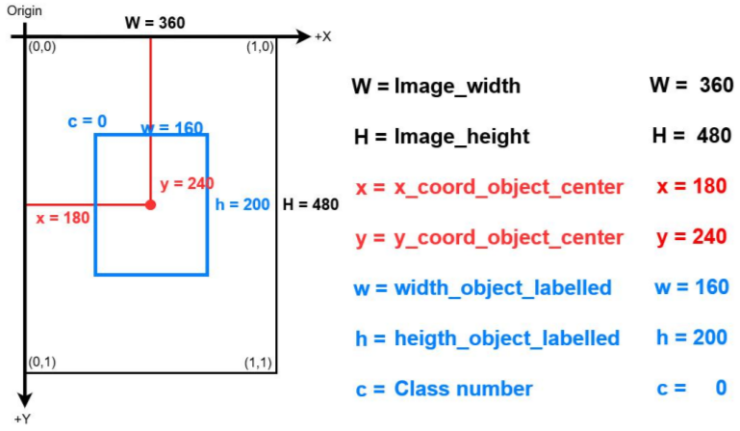

Figure 11-7: YOLO Bounding Box Annotation Format with Normalized Image Coordinates.

The dataset is divided into training, validation, and testing subsets. A typical split is:

Subset	Percentage	Purpose
Training set	70%	Network weight optimization
Validation set	20%	Hyperparameter tuning and overfitting monitoring
Test set	10%	Final independent performance evaluation

Table 11-2: The dataset typical split.

When the dataset size is limited, transfer learning is applied. The detector is initialized using weights pretrained on a large-scale detection dataset such as MS COCO. The network is then fine-tuned using the custom robotic manipulation dataset. This procedure reduces training time and improves convergence because low-level visual features such as edges, corners, textures, and object contours have already been learned [29] [11] [23] [13].

11.9. YOLO Training Configuration

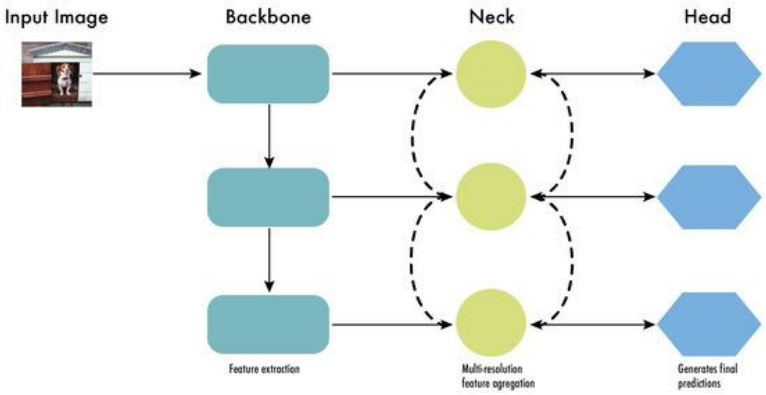

Figure 11-8: The architecture of modern object detectors can be described as the backbone, the neck, and the head. The backbone, usually a convolutional neural network (CNN), extracts vital features from the image at different scales. The neck refines these feature.

The YOLO detector is trained using supervised learning. The input is an annotated RGB image, and the target output consists of ground-truth bounding boxes and class labels. During training, the loss function combines localization error, objectness error, and classification error. In general form, the total loss can be written as:

$$L_{YOLO} = L_{box} + L_{obj} + L_{cls}$$

where L_{box} penalises inaccurate bounding box regression, L_{obj} penalises incorrect objectness prediction, and L_{cls} penalises incorrect class prediction.

Parameter	Recommended Value	Rationale
Input image size	416×416 or 640×640	Balance between speed and localization accuracy
Batch size	8–32	Selected according to GPU memory
Optimiser	SGD or Adam	Standard optimization methods for CNN training
Initial learning rate	10^{-3} *to* 10^{-4}	Stable fine-tuning from pretrained weights
Confidence threshold	0.40–0.60	Reduces false detections
NMS IoU threshold	0.45–0.60	Removes duplicate boxes
Training epochs	100–300	Depends on dataset size and convergence
Data augmentation	Enabled	Improves robustness to lighting, scale, and pose variation

Table 11-3: Recommended YOLO training parameters

Data augmentation is especially important for robotic deployment. Random brightness, contrast adjustment, scaling, rotation, cropping, translation, and blur can be applied during training. These transformations help the detector remain stable when lighting conditions or camera viewpoints change [11] [23] [13] [15] [21].

11.10. Integration with the RL State Space

The original RL state vector is retained, but the target-related components are supplied by the perception module rather than by fixed environment variables. The updated state vector is expressed as:

$$s = \left[q_1, q_2, q_3, \dot{q}1, \dot{q}2, \dot{q}3, p_{EE,x}, p_{EE,y}, p_{EE,z}, x_b, \theta_b, \Delta x, \Delta y, g\right]^T$$

where:

$$\Delta x = x_{target} - x_{EE}$$

$$\Delta y = y_{target} - y_{EE}$$

If the vertical coordinate is required for grasp-height correction, the target height may also be included in an extended observation vector:

$$s_{vision} = \left[q, \dot{q}, p_{EE}, x_b, y_b, \theta_b, p_{target}, \Delta p, g, p_{conf}\right]^T$$

where p_{conf} is the YOLO confidence score. This extension allows the controller to account for uncertainty in perception. However, when compatibility with the previously trained PPO agent must be preserved, the original 14-dimensional observation vector is maintained and only the existing target components are updated.

The perception module therefore modifies the observation function as follows:

$$getObservation(env) \Rightarrow getObservation\left(env, p_{target}^{YOLO}\right)$$

The reward function remains structurally unchanged. The reaching reward, carrying reward, grasp bonus, and placement reward are still computed using the same terms, but the target position is now measured from real sensor data:

$$r_{reach} = -\left|\left|p_{EE} - p_{target}^{YOLO}\right|\right|_2$$

This approach preserves the RL controller design while replacing simulated target knowledge with real-time perception [17].

11.11. Grasp Point Estimation

YOLO provides a bounding box around the object, but it does not directly provide a full six-degree-of-freedom grasp pose. For the two-finger gripper used in this project, a simplified grasping strategy is adopted. The object centroid obtained from the bounding box and depth image is used as the primary grasp target:

$$p_{grasp} = p_{target}^{YOLO} + \left[0,0, z_{offset}\right]^T$$

where z_{offset} is a small vertical offset used to approach the object from above before closing the gripper.

The grasping procedure is divided into three phases:

1. **Pre-grasp approach:** the end-effector is moved to a point above the object.

2. **Descent:** the gripper is lowered until the object is within the grasp tolerance.
3. **Closure and lift:** the gripper is closed and the object is lifted above the table.

For objects with simple shapes, such as blocks, cylinders, or small boxes, the bounding box center provides a sufficient approximation of the grasp point. For irregular objects, grasp orientation estimation may be improved by using segmentation, principal component analysis on the depth points inside the bounding box, or a dedicated grasp detection network. In the present implementation, the YOLO module is responsible only for object detection and centroid localization, while grasp execution is handled by the RL policy and the existing gripper logic.

11.12. Temporal Filtering and Detection Stability

Real-time detections may contain jitter due to image noise, depth quantization, partial occlusion, or small changes in bounding box location. Directly feeding unfiltered detections to the controller may cause oscillatory motion. To reduce this effect, temporal filtering is applied to the estimated target position.

An exponential moving average is used:

$$\hat{p}target, t = \alpha ptarget, t^{YOLO} + (1 - \alpha)p_{\widehat{target,t-1}}$$

Where α is the smoothing factor. A typical value is selected in the range $0.2 \leq \alpha \leq 0.5$. Smaller values produce smoother estimates but slower response, while larger values produce faster response but more jitter.

Outlier rejection is also applied. A detection is rejected when one of the following conditions is satisfied:

- the confidence score is lower than the selected threshold;
- the depth value is invalid or outside the camera range;
- the transformed 3D point lies outside the robot workspace;
- the detected object class does not match the required task object;
- the new target estimate differs from the previous estimate by an unrealistically large displacement.

The final target position supplied to the RL controller is therefore:

$$p_{target}^{valid} = \begin{cases} p_{\widehat{target,t}} & valid\ detection \\ p_{\widehat{target,t-1}} & temporary\ detection\ loss \\ \emptyset & detection\ timeout \end{cases}$$

When no valid target is available, the controller is prevented from executing grasping actions. This safety condition reduces the probability of unintended motion caused by false detections.

12.Materials and Hardware Components

The mobile manipulator hardware is constructed using a plexiglass mechanical structure, four DC wheel motors, four servos, a servo-actuated gripper, ultrasonic distance sensors, and embedded control electronics. The bottom platform provides mobility with the help of four

wheels, and the upper plexiglass arm gives manipulation capabilities using servos to drive revolute joints. In order to allow

Materials chosen provide a good trade-off between mechanical simplicity, low cost, modular design, and ease of implementation of machine learning based control techniques. Plexiglass used as construction material allows to reduce mechanical weight of the structure and helps with fast prototyping. The set of Arduino Mega board, Raspberry Pi 4, L298N motor driver, PCA9685 PWM servo driver, HC-SR04 sensors is sufficient to create a testbed for experiments with mobile manipulation, perception and reinforcement learning based control methods.

12.1. The Mechanical and Electronic Structure

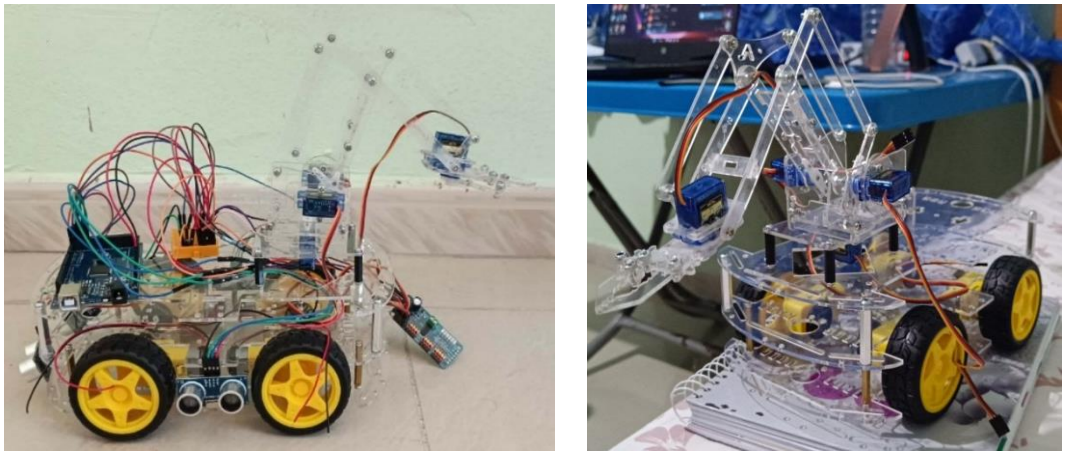

Figure 12-1: The plexiglass mobile robot with four TT DC gear motors, ultrasonic sensors, Arduino-based control system, and a servo-driven robotic arm.

The mobile manipulator was constructed as a compact wheeled robotic platform equipped with a revolute robotic arm and a servo-actuated gripper. The mechanical structure was designed to support pick-and-place operation in static indoor environments, where the mobile base provides planar locomotion and the arm provides object-reaching and grasping capability. The system was assembled using a plexiglass frame, four DC motors for the mobile base, four servo motors for the manipulator and gripper mechanism, multiple ultrasonic sensors for obstacle detection, and an embedded control architecture based on Arduino and Raspberry Pi hardware.

The mechanical design consists of two main subsystems: a four-wheel mobile base and an upper manipulator assembly. The mobile base carries the electronic control boards, power supply elements, motor drivers, sensors, and the arm-supporting structure. The arm is mounted above the chassis and is composed of multiple plexiglass links connected through servo-driven revolute joints. A small end-effector is attached at the distal end of the arm to perform grasping actions. The complete hardware architecture was selected to satisfy the requirements of low-cost construction and modularity.

12.2. Plexiglass Mechanical Frame

The chassis and robotic arm linkages were made of plexiglass acrylic sheets. Plexiglass was chosen as the material since it offered enough strength for the light mobile manipulator while being easily cut, drilled, assembled, and modified. Additionally, the transparent construction allowed all internal electronic parts and wiring pathways to be seen.

The bottom chassis is made of several plexiglass sheets that are separated by spacers and screwed together. The layered construction helps to provide mechanical support for the motor drivers, wheels, ultrasonic sensors, control boards, battery pack, and power management modules. The separation of sheets due to the presence of spacers provides space for the wiring and modules not to get into the way of rotating wheels or movable arm linkages.

The manipulator was also constructed using plexiglass material. Each arm link was connected by means of the servo-actuated revolute joints to form a lightweight articulated structure which is appropriate for handling small objects. The link design was chosen to ensure that the required reach is achieved while keeping the total mass of the arm low. The reason behind this is that an increase in the arm mass will result in increased load in the base and increased center of mass which can cause instability during arm extension.

The gripper device was mounted on the end of the robotic arm. It was designed as a simple two fingered device driven by the servo motor. The purpose of the gripper is to handle small lightweight objects. Its simplicity makes the grasping process easily controllable and representable as a binary signal in the controller.

12.3. Mobile Base and Wheel Drive System

The mobile base uses a four-wheel configuration. Each wheel is driven by a dedicated DC motor, resulting in a total of four DC motors for locomotion. This arrangement provides sufficient traction and load distribution for the plexiglass chassis and the upper manipulator assembly. The four-wheel structure also improves platform stability compared with a two-wheel structure, especially when the arm is extended and the center of mass shifts away from the geometric center of the chassis.

Figure 12-2: Transparent plexiglass robot car chassis.

The base is treated as a standard wheeled mobile platform. Lateral motion is not mechanically produced by the wheels; therefore, the control commands are represented through forward linear velocity and yaw angular velocity. The left and right wheel groups can be driven at different speeds to generate turning motion. This configuration is consistent with a differential-drive approximation, where the mobile robot is controlled through forward velocity and rotational velocity rather than direct lateral displacement.

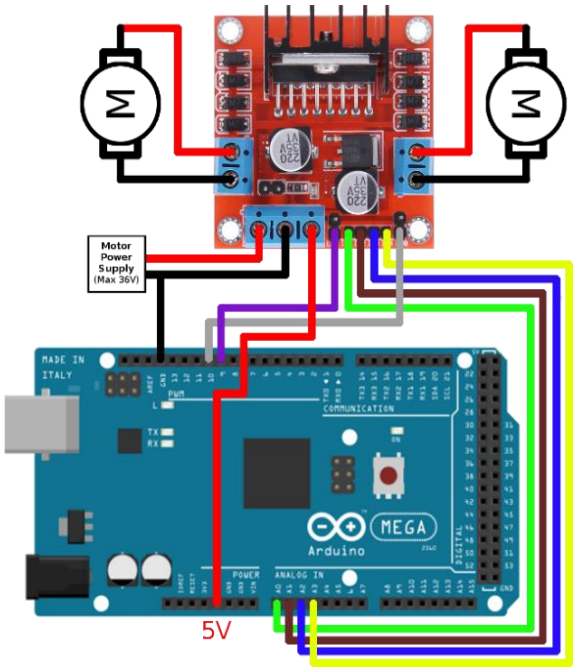

Figure 12-4: Arduino Mega 2560 connected to an L298N dual H-bridge motor driver for controlling two DC motors.

Figure 12-3: Yellow TT DC gear motor.

Figure 12-5: Yellow TT DC gear motor with robot car wheel attached.

Two L298N dual H-bridge motor driver boards are used to operate the four DC motors. An L298N board can drive two DC motors; hence, two boards are enough to drive the four-wheel motors independently. Motor drivers get low-voltage commands from the microcontroller and give high currents needed by the DC motors. There is a distinction between logical commands and motor operation, which shields the control board and allows motion control in both directions and turns.

The wheel-drive subsystem therefore consists of:

Component	Quantity	Function
DC motor	4	Provides wheel actuation
Wheel	4	Supports and moves the mobile base
L298N dual H-bridge motor driver	2	Drives the four DC motors
Plexiglass chassis plates	Several structural plates	Support the drive system and electronics

Table 12-1: The wheel-drive subsystem

12.4. Robotic Arm and Gripper Actuation

The robotic arm is actuated using four servo motors. Three servo motors are assigned to the revolute joints of the robotic arm, while the fourth servo motor is assigned to the gripper mechanism. This arrangement is suitable for a compact educational and experimental mobile manipulator because servo motors provide direct angular positioning capability without requiring additional gearbox design or external position encoders.

The three arm servos define the main manipulator motion. The first joint provides the initial arm rotation or shoulder movement, the second joint changes the arm elevation or elbow configuration, and the third joint adjusts the distal link or wrist configuration. The fourth servo motor actuates the gripper, allowing the end-effector to open and close around the target object.

The servo motors will make the mechanical implementation easier since each joint will be actuated using the PWM signal generated. PCA9685 16-channel PWM servo driver is present in the design to ensure generation of stable PWM signals for the servo motors. The PWM driver relieves the primary microcontroller of the responsibility of timing while providing an interface that allows scalability of the system to more than one servo motor. Only four servo motors are needed in the present design but future designs might add more.

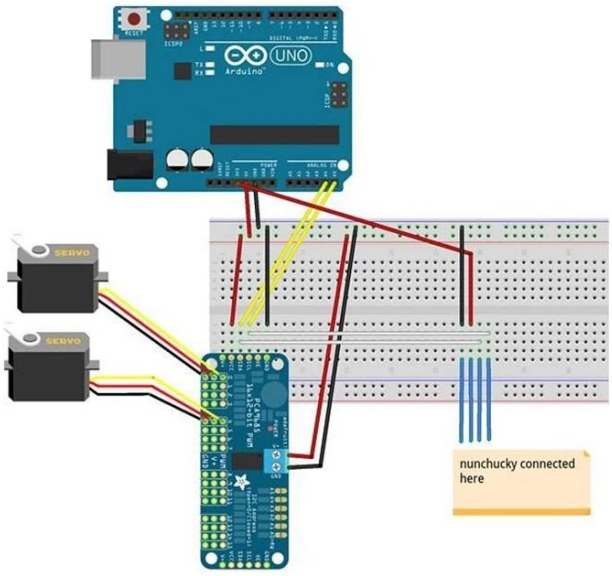

Figure 12-7: Arduino Uno servo motor wiring circuit with breadboard and servo driver connection diagram.

Figure 12-6: TowerPro SG90 micro servo motor with three-wire control cable.

The manipulator actuation subsystem is summarized as follows:

Component	Quantity	Function
Servo motor	4	Drives the arm joints and gripper
PCA9685 16-channel PWM servo driver	1	Generates PWM control signals for the servos
Plexiglass arm links	Several links	Forms the manipulator structure
Plexiglass gripper elements	One gripper assembly	Performs object grasping

Table 12-2: The manipulator actuation subsystem

The four-servo configuration allows the mobile manipulator to perform reaching and grasping motions required for pick-and-place operation. The arm structure remains lightweight, which reduces the torque demand on the servos and limits destabilizing effects on the mobile base.

12.5. Control and Processing Units

The electronic structure of the robot is organized on low-level control and high-level processing. The Arduino Mega 2560 is employed as a low-level controller in charge of motors and sensors. It possesses a sufficient amount of digital and analog input/output pins in order to control motor drivers, read ultrasonic sensors, communicate with PWM servo driver and get sensor feedback.

Raspberry Pi 4 Model B with 4 GB RAM is utilized as a high-level processing unit. It is capable of performing computations more complex than those a microcontroller can do, for instance, image acquisition, object recognition via YOLO model, data communication, and decision-making. Besides, it has the possibility of working with camera modules and robotics software based on Linux environment.

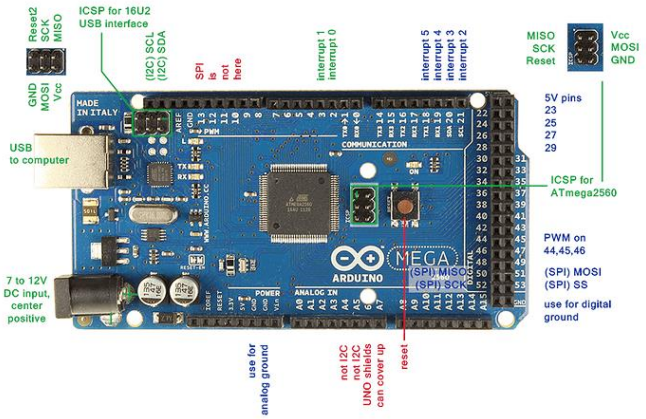

Figure 12-11: Arduino Mega 2560 microcontroller board with labeled pinout diagram.

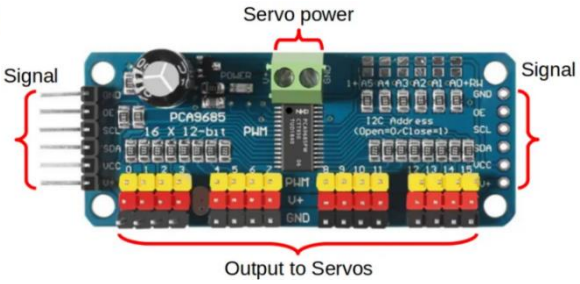

Figure 12-10: PCA9685 16-channel PWM servo driver module with servo power and signal connections.

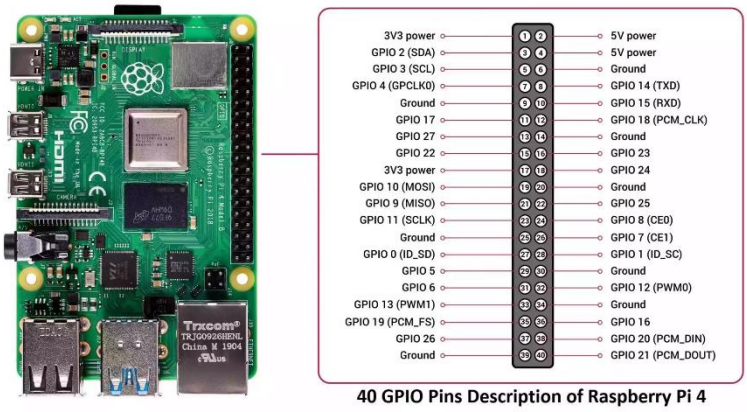

Figure 12-9: Raspberry Pi 4 Model B single-board computer with 40-pin GPIO header.

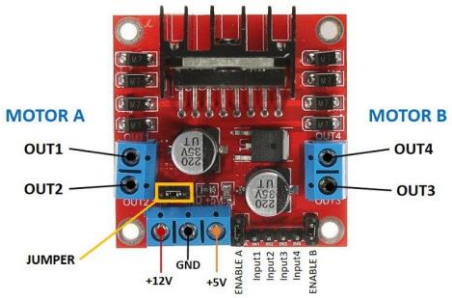

Figure 12-8: L298N dual H-bridge DC motor driver module with motor output and power terminals.

Low-level and high-level controllers are meant to work collaboratively. Raspberry Pi processes visual data and high-level commands, whereas Arduino Mega performs hardware interfacing tasks in real time. Such organization of system makes its structure more efficient because perception and decision-making take place separately from motor signal formation and sensor reading.

Component	Quantity	Function
Raspberry Pi 4 Model B, 4 GB RAM	1	High-level processing, visual perception, and supervisory control
Arduino Mega 2560	1	Low-level motor control, sensor reading, and actuator interfacing
PCA9685 16-channel PWM servo driver	1	Dedicated PWM generation for servo motors
L298N dual H-bridge motor driver	2	DC motor driving for the four-wheel base

Table 12-3: Control and processing hardware.

12.6. Ultrasonic Sensing System

There are four HC-SR04 ultrasonic sensors that are utilized for distance sensing and environmental obstacle detection around the mobile platform. Ultrasonic sensors emit sound waves that bounce back from any nearby surface and time taken for reflection is calculated, which helps in estimating the distance.

Ultrasonic sensors are placed around the body of the platform in order to help in the environmental sensing during movement. It should be noted that the purpose of these sensors is not to detect the grasp target for manipulation, but to help in navigation through the identification of nearby obstacles or boundaries, particularly in case of static indoor environment.

Figure 12-15: HC-SR04 ultrasonic distance sensor module with transmitter and receiver transc

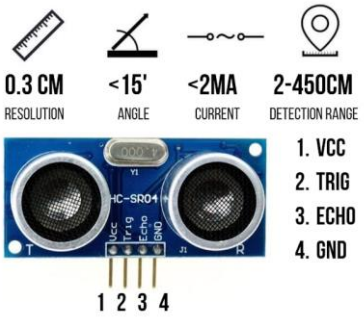

Figure 12-14: HC-SR04 ultrasonic sensor specifications and pinout diagram.

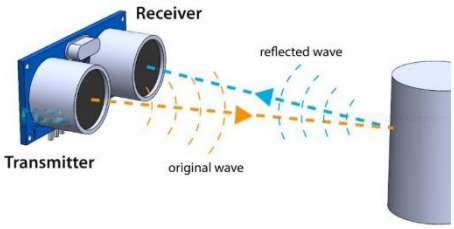

Figure 12-13: Ultrasonic sensor working principle diagram showing transmitted and reflected sound waves.

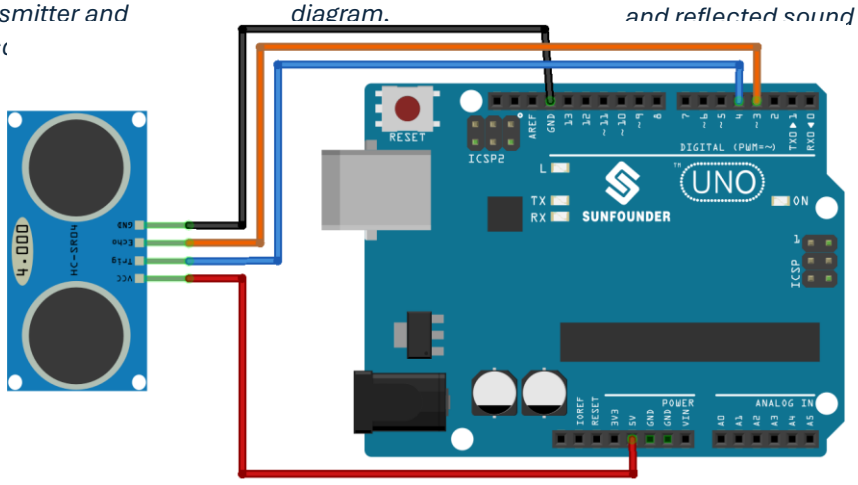

Figure 12-12: Arduino Uno connected to an HC-SR04 ultrasonic distance sensor wiring diagram.

The ultrasonic sensing subsystem is summarized as follows:

Component	Quantity	Function
HC-SR04 ultrasonic sensor	4	Measures obstacle distance around the mobile base

Table 12-4: The ultrasonic sensing subsystem.

The distance measurements obtained from the ultrasonic sensors may be used for collision avoidance, safety stopping, or environment monitoring. In the complete system architecture, these readings can be combined with the mobile base control logic to prevent the robot from driving into obstacles while performing pick-and-place tasks.

12.7. Supply and Voltage Regulation

The robot is powered using three 3.7 V lithium-ion battery cells. The battery cells provide the primary energy source for the mobile platform, including the DC motors, servo motors, sensors, and control electronics. Since the different electronic modules require different voltage levels, voltage regulation is necessary.

The LM2596 DC-DC buck converter is employed as the primary voltage step-down regulator in the power sub-system. The function of the LM2596 is to convert the battery pack voltage to a regulated voltage that can be utilized by low-voltage electronics, such as the microcontroller, the sensor array, and the interface electronics. Though the use of one converter in the current design is sufficient, it is crucial for preventing overvoltage damage to the control electronics and ensuring a more stabilized source than what the raw battery voltage can offer.

Figure 12-17: LM2596 DC-DC buck converter module.

Figure 12-16: INR18650 lithium-ion rechargeable battery cell

The power subsystem is summarized as follows:

Component	Quantity	Function
Li-ion battery cell, 3.7 V	3	Provides the main electrical power source
LM2596 DC-DC buck converter	1	Provides a regulated low-voltage supply for the control electronics, sensors, and interface circuits.

Table 12-5: The power subsystem

Proper power distribution is necessary for reliable operation. The motor supply and logic supply should be separated or regulated appropriately to reduce electrical noise and prevent resets of the control electronics. Ground reference connections must be shared between the motor drivers, microcontroller, PWM driver, and sensors to ensure correct signal interpretation.

12.8. Wiring, Prototyping, and Mounting Accessories

Connection between the control electronics, sensors, motor driver, and power sources is done through a test board and jumper wires. The test board acts as an interconnecting medium in the form of a temporary or semi-permanent connection during the development process. Jumper wires offer quick alteration of connections, which comes in handy when making prototypes, diagnosing problems, and calibrating sensors.

The fastening is done through the use of screws, nuts, spacers, and brackets. These help to keep the plexiglass plates, arm links, servo motors, sensors, and electronics in place. The modularity of the fasteners ensures that the robot can be easily taken apart whenever there is need for any changes.

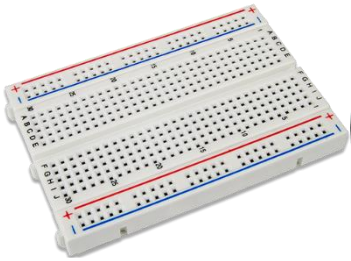

Figure 12-20: Solderless breadboard / electronic test board for prototyping circuits.

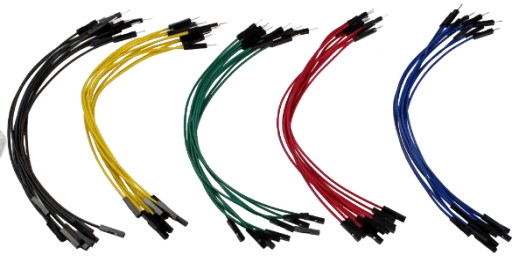

Figure 12-18: Male-to-male jumper wires for breadboard and microcontroller connections.

Figure 12-19: Assorted screws, nuts, and washers for fastening mechanical and electronic components.

The supporting materials are summarized as follows:

Component	Quantity	Function
Test board	1	Provides prototyping and wiring interface
Jumper wires	As required	Connects sensors, drivers, controllers, and power modules
Screws, nuts, and spacers	As required	Fastens plexiglass plates and mounted components
Plexiglass brackets	As required	Supports sensors, arm links, and electronics

Table 12-6: The supporting materials.

12.9. Complete Bill of Materials

The main components used in the mobile manipulator are listed in Table 12-7. The quantities reflect the implemented mechanical design and the specified electronic configuration.

Category	Component	Quantity	Role in the System
Mechanical structure	Plexiglass chassis plates	Several	Forms the mobile base frame
Mechanical structure	Plexiglass robotic arm links	Several	Forms the manipulator structure
Mechanical structure	Plexiglass gripper components	1 assembly	Provides the end-effector mechanism
Locomotion	DC motor	4	Drives the four wheels
Locomotion	Wheel	4	Provides ground contact and mobility
Manipulation	Servo motor	4	Drives three arm joints and the gripper
Motor control	L298N dual H-bridge motor driver	2	Controls the four DC motors
Servo control	PCA9685 16-channel PWM servo driver	1	Generates PWM signals for servo actuation
Low-level control	Arduino Mega 2560	1	Controls motors, sensors, and actuator interfaces
High-level processing	Raspberry Pi 4 Model B, 4 GB RAM	1	Handles high-level processing and perception tasks
Distance sensing	HC-SR04 ultrasonic sensor	4	Measures obstacle distance around the base
Power supply	Li-ion battery cell, 3.7 V	3	Supplies electrical power
Voltage regulation	LM2596 DC-DC buck converter	1	Provides regulated voltage outputs
Prototyping	Test board	1	Supports electrical connections during assembly
Wiring	Jumper wires	As required	Provides signal and power connections
Fastening	Screws, nuts, spacers	As required	Fixes mechanical and electronic components

Table 12-7: Main materials and hardware components.

12.10. Functional Relationship Between Materials and System Operation

These parts create a hardware platform for autonomous mobile manipulation. Plexiglass chassis is used as the basis of the mechanism, and the four DC motors and wheels are responsible for its planar motion. L298N motor drivers serve as the connection between the low-level controller and DC motors. The Arduino Mega 2560 creates the motor control signals and receives information from low-level sensors.

Plexiglass arm and gripper are the parts of the manipulation sub-system. Four servo motors are needed for performing joints and gripper actuation. PCA9685 servo driver ensures the supply of steady PWM signals for servos that allows the manipulator to perform reaching, lifting, and grasping tasks. Using of servo actuation simplifies the mechanism and allows controlling the robotic arm by means of angle commands or velocities.

The subsystem for sensing has four HC-SR04 ultrasonic sensors which help with obstacle avoidance in the mobile platform.

Power subsystem provides power to all the electronics as well as electromechanics. Li-ion batteries are used as the source of power whereas the LM2596 step-down regulator is used to provide a stable regulated supply for the low-voltage control electronics, sensors, and interface circuits. Voltage regulation is important because the microcontroller, Raspberry Pi, sensors, and signal interfaces require a stable supply while the motors and servos draw varying current during operation.

13.Methods

13.1. Design

The robot is designed as an autonomous mobile manipulator intended for indoor pick-and-place tasks. The current platform combines a compact wheeled base, an articulated robotic arm, and a layered electronics architecture that supports sensing, actuation, and embedded control. The mechanical layout was developed in SolidWorks 2022 with an emphasis on structural stability, modular assembly, serviceability, and component accessibility. The overall geometry was selected to keep the center of mass low and close to the center of the chassis, which improves stability during motion and when the arm is extended.

The physical prototype and the CAD model were developed in parallel so that the assembly process, routing constraints, and component placement could be validated before fabrication. This approach reduces integration errors and makes it easier to revise individual modules without redesigning the full robot. In the current design, the platform is organized into a lower mobility subsystem and an upper manipulation and electronics subsystem, allowing future upgrades to be implemented in a modular way.

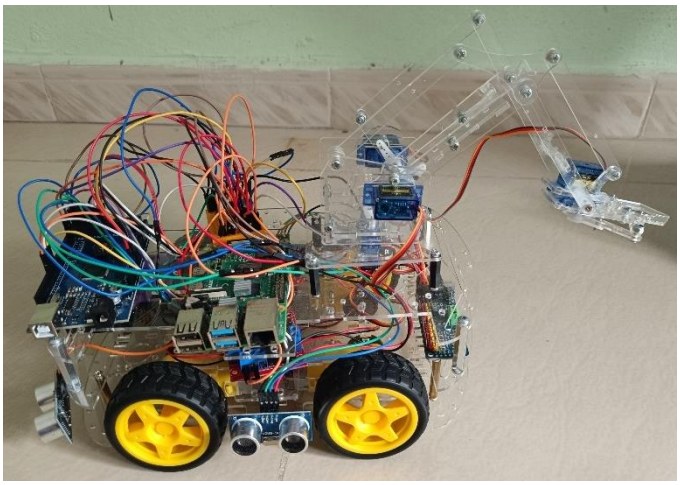

Figure 13-1: Physical prototype of the mobile manipulator from a side view, showing the chassis, wheels, upper electronics, and arm placement.

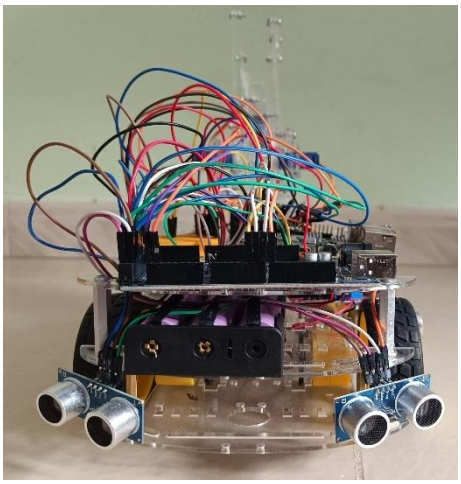

Figure 13-2: Front view of the prototype highlighting the wheel arrangement, compact footprint, and overall robot symmetry.

13.2. Mechanical Architecture and SolidWorks Design

The SolidWorks assembly was used as the main reference for defining the robot's structure, part interfaces, and spatial constraints. The model includes the chassis plates, support columns, wheel mounts, motor brackets, arm links, gripper, sensor brackets, and electronics supports. The arm is mounted on the upper chassis to maximize reach while preventing collisions with the drive system and lower sensors. The mobile base and the arm were treated as coupled subsystems, since their combined motion directly affects dynamic balance, payload handling, and workspace coverage.

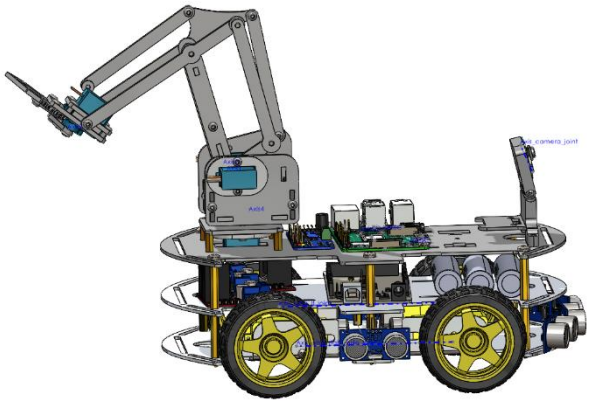

Figure 13-3: Isometric SolidWorks view of the full robot assembly showing the interaction between the mobile base, electronics layer, and robotic arm.

From a design perspective, the assembly was developed to satisfy four engineering requirements. First, the base must support the total mass without excessive deformation. Second, the arm must operate within the available envelope without interference with the chassis or wheels. Third, the electronics must remain accessible for maintenance and testing. Fourth, the robot must preserve enough rigidity to avoid vibration and backlash during motion. These requirements guided the selection of plate thickness, mounting positions, and support spacing.

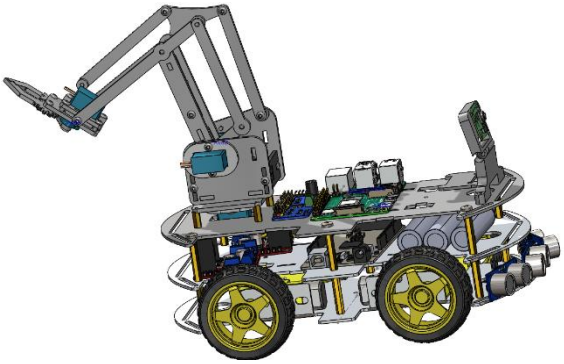

Figure 13-5: Detailed SolidWorks view of the full system illustrating the layered mechanical integration and internal component arrangement.

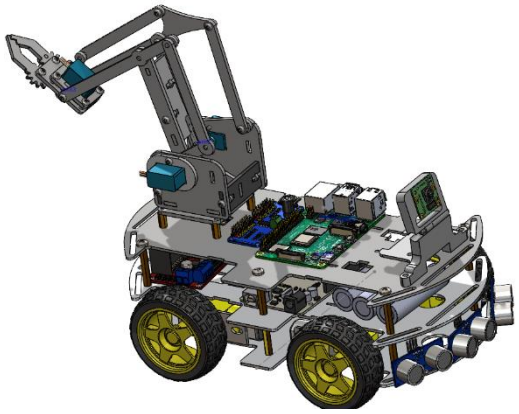

Figure 13-4: Alternative SolidWorks perspective emphasizing the chassis structure, wheel placement, and arm extension geometry.

13.3. Bill of Materials and Material Selection

The platform uses a hybrid construction strategy combining acrylic or lightweight structural plates, 3D-printed support parts, standard servomotors, DC gear motors, and off-the-shelf sensors. This combination was chosen to reduce fabrication complexity while still maintaining mechanical precision and repeatability. Transparent or semi-transparent structural elements were used in the prototype to simplify inspection, debugging, and cable tracing during development. For the final system, the same design philosophy can be retained with stronger materials if higher payload capacity is required.

The material selection was based on weight, manufacturability, stiffness, and assembly convenience. The chassis plates were selected to carry the electronics and mechanical subsystems while keeping the base light enough for efficient locomotion. The arm links were designed to remain rigid under joint motion and moderate payloads, especially near the gripper where bending moments are highest. Fasteners, spacers, brackets, and standoffs were used to preserve alignment between layers and to create enough vertical clearance for wiring and service access.

13.4. Mass Properties and Load Analysis

A mass-properties analysis was performed in SolidWorks to estimate the total weight and the distribution of mass across the assembly. This analysis is essential because the base motors, battery sizing, wheel traction, and static stability all depend on the robot's final mass distribution. The results shown in Figure 13-6 indicate a total assembly mass of approximately 1233.33 grams, with the reported center of mass located at $X = -772.35\,mm, Y = -173.02\,mm,$ and $Z = 125.22\,mm$ in the chosen coordinate frame.

The reported mass properties provide useful design feedback. A low center of mass helps reduce tipping risk when the arm is lifted or when the robot changes direction quickly. At the same time, the chassis should remain stiff enough to prevent the mass distribution from shifting under load. The analysis therefore supports the mechanical decision to place heavier components such as the battery, controllers, and driver boards closer to the lower structure of the robot.

13.5. Stability and Center of Mass Considerations

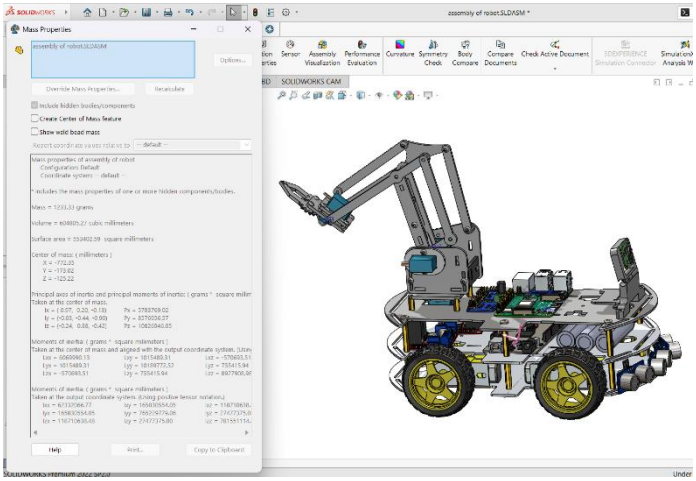

Figure 13-6: SolidWorks mass-properties window showing the total mass, volume, surface area, and center-of-mass coordinates.

Stability is a critical requirement for any mobile manipulator because the robot must remain safe during motion, stopping, turning, and arm extension. In this design, the center of mass remains within the support polygon formed by the wheels, which helps preserve equilibrium during standard operating conditions. The use of a compact lower body and a vertically constrained arm mount reduces the likelihood of instability during acceleration or when the gripper is carrying an object.

The center-of-mass analysis also informs motion planning. When the arm is extended forward, the base must move more cautiously because the effective tipping moment increases. For that

reason, the mechanical layout should be discussed together with the intended control strategy, since the final robot behavior depends on both structure and motion logic. This is especially important in a mobile manipulator where the base and arm are coordinated during pick-and-place actions.

13.6. Assembly Strategy and Mechanical Integration

The robot was assembled using a hierarchical strategy in which the base, arm, and electronics were prepared as partially independent modules before full integration. This modular approach simplifies debugging because each subsystem can be tested separately before the complete system is powered. It also makes the robot easier to maintain, since individual components can be replaced without dismantling the full platform.

13.7. Electrical System and Wiring Integration

The electrical system of the prototype was designed to support both the mobile base and the manipulator through a unified control architecture. As illustrated in Figure 13-7, the wiring diagram integrates the Arduino Mega as the central controller, ultrasonic sensors for obstacle detection, L298N motor drivers for the DC motors of the mobile base, and servo interfaces for the arm and gripper joints. This arrangement allows the controller to collect environmental data from the sensors, process the input signals, and generate the appropriate motion commands for both the drive and manipulation subsystems.

At the hardware level, the system follows a layered control chain in which the sensors provide feedback to the microcontroller, the microcontroller executes the control logic, and the output commands are distributed to the motor drivers and servos. The motor drivers handle the high-current requirements of the mobile platform, while the servos are powered and controlled separately for precise arm motion. A dedicated power distribution path is used to separate logic-level signals from actuator supply lines, which helps reduce electrical noise and improves system stability.

In the current prototype, jumper wires and a breadboard were used to simplify assembly and testing during the experimental phase. However, for the final implementation, improved cable routing, connector labeling, strain relief, and a cleaner power distribution layout should be adopted to increase reliability and maintainability. These improvements are especially important for future integration with ROS 2, MATLAB/Simulink, and additional onboard perception modules.

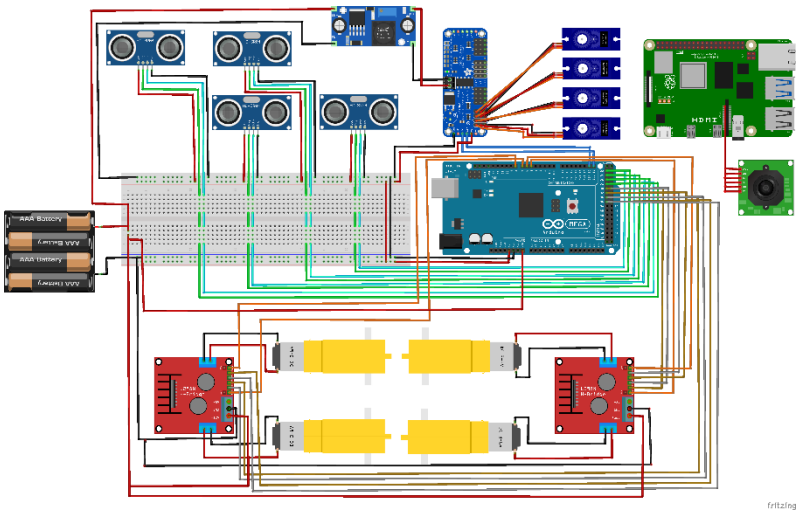

Figure 13-7: Fritzing wiring diagram showing the Arduino Mega, ultrasonic sensors, motor drivers, servo connections, and power distribution network.

14.EXPERIMENTAL RESULTS AND SYSTEM VALIDATION

This chapter presents the experimental evaluation and validation of the proposed intelligent mobile manipulator system. The chapter connects the theoretical design, reinforcement learning training, and simulation implementation into one complete validation stage. The evaluation begins with the original research vision, then explains the adopted incremental development strategy, followed by the reinforcement learning results obtained using PPO and SAC. The chapter then discusses the transition from the two-dimensional training environment to the three-dimensional simulation environment and presents the final validation of the mobile manipulator in CoppeliaSim and MATLAB.

In the final execution of the system, there were various modules put together including the reinforcement learning navigation policy that was created using Python, the trajectory following and visualization component using MATLAB, the mobile base, the robot manipulator, the vision system component, and the pick and place procedure. This helped in ensuring the system incorporated both navigation and integration of robots.

14.1. Original System Vision

The original objective of this project was to develop a fully autonomous vision-based mobile manipulator driven by reinforcement learning. The intended complete framework was designed to integrate perception, decision making, navigation, manipulation, and object transportation into a unified intelligent robotic system.

In the complete vision, the onboard camera would detect the target object, a perception module such as YOLO would extract object information from the image, and a full state vector would describe both the mobile base and the robotic arm. This state vector would then be used by a reinforcement learning agent to control the entire pick-and-place process, including navigation toward the object, obstacle avoidance, grasping, transport, and placement at the goal.

Figure 14-1: Overall vision of the proposed intelligent mobile manipulator system.

14.2. Incremental Development Strategy

Although the complete end-to-end system represents the long-term objective of the project, training a reinforcement learning agent using the full mobile manipulator state is a complex task. A complete state vector would include the mobile base position, heading angle, obstacle distances, goal position, arm joint angles, end-effector pose, object pose, gripper state, and visual information extracted from the camera. This significantly increases the dimensionality of the state space and therefore increases the required training time and computational cost.

For this reason, the work was divided into incremental development stages. The first stage focused on learning the navigation behavior of the mobile base in a 2D environment. The second stage integrated the navigation output with a 3D mobile manipulator simulation. The final stage, proposed as future work, extends the system toward full end-to-end vision-based reinforcement learning.

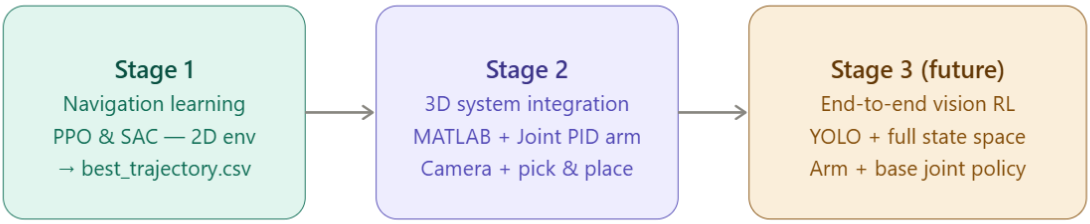

Figure 14-2: Incremental development strategy adopted in the project.

Stage	Objective	Description	Output
Stage 1	Navigation Learning	Train PPO and SAC for mobile base navigation in a 2D environment.	Best navigation policy and training results.
Stage 2	3D System Integration	Integrate the navigation output with the mobile manipulator, vision module, and pick-and-place sequence.	Complete simulation validation.

Stage 3	Future End-to-End Learning	Integrate camera images, YOLO, arm states, and gripper state into a full learning framework.	Fully autonomous vision-based mobile manipulator.

Table 14-1: Stages of the proposed development strategy.

14.3. Reinforcement Learning Results

The reinforcement learning algorithms were trained using the developed 2D navigation environment. During training, the robot interacted with a workspace containing a goal and static obstacles. The learning objective was to reach the target while avoiding collisions and minimizing the generated path length. The training performance was evaluated using average reward, average error, success rate, collision rate, path length, and training time.

14.3.1. PPO Results

The PPO algorithm demonstrated gradual improvement throughout the training process. The accumulated reward increased steadily, showing that the policy was learning to move toward the goal. However, the convergence was relatively noisy and required a larger number of episodes before the performance became stable. The final path produced by PPO was conservative and longer than the SAC path, which increased the average path length.

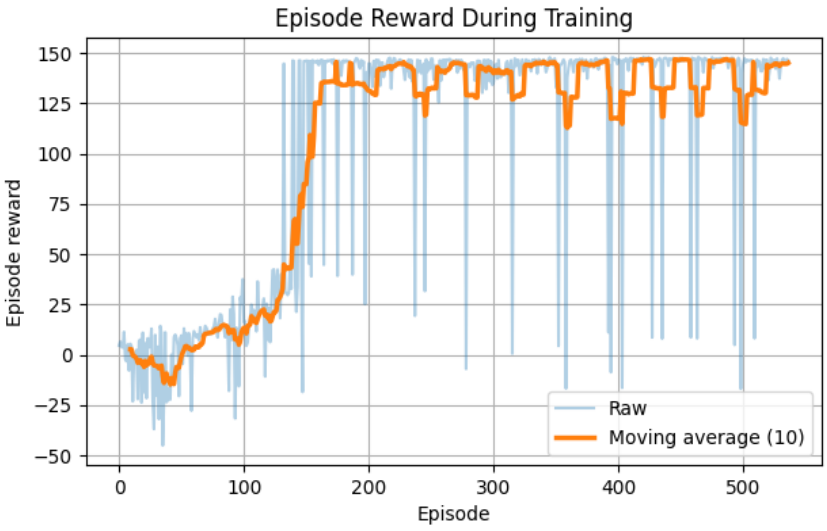

Figure 14-3: Episode reward during training.

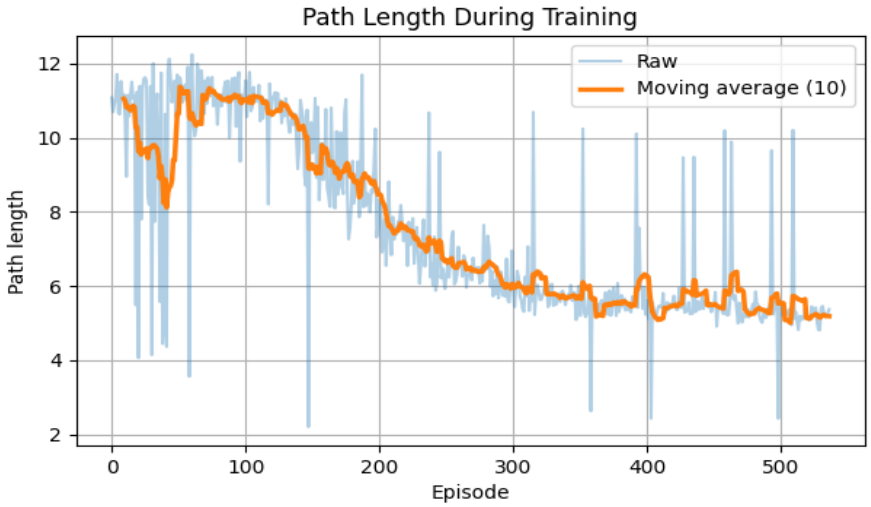

Figure 14-6: PPO reward curve and distance-to-goal error during training.

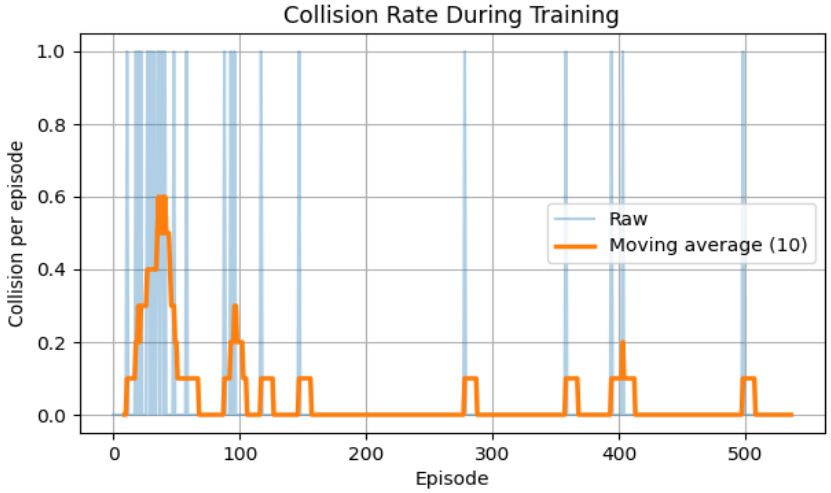

Figure 14-5: PPO final path and path length behavior.

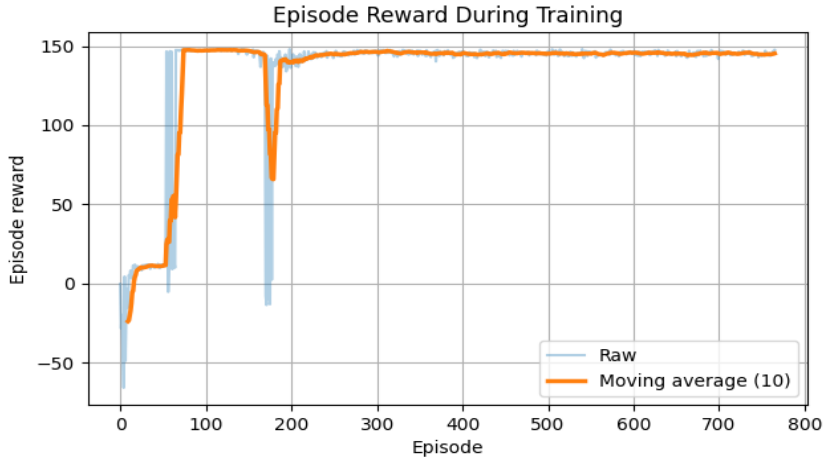

Figure 14-4: PPO collision rate and success rate during training.

The results indicate that PPO can learn the navigation problem, but its on-policy nature limits sample efficiency. The policy remains more sensitive to obstacle configurations, and occasional collision bursts appear during training. Therefore, PPO is useful as a baseline but was not selected as the main navigation policy for final validation.

14.3.2. SAC Results

SAC performed better learning behavior compared to PPO. SAC's off-policy actor-critic architecture, use of replay buffer and entropy regularization helped in effective exploration and quicker convergence. The reward graph showed rapid increase in initial stages of training and plateaued at maximum possible reward level. Distance to goal error also reduced quickly and stayed low after convergence.

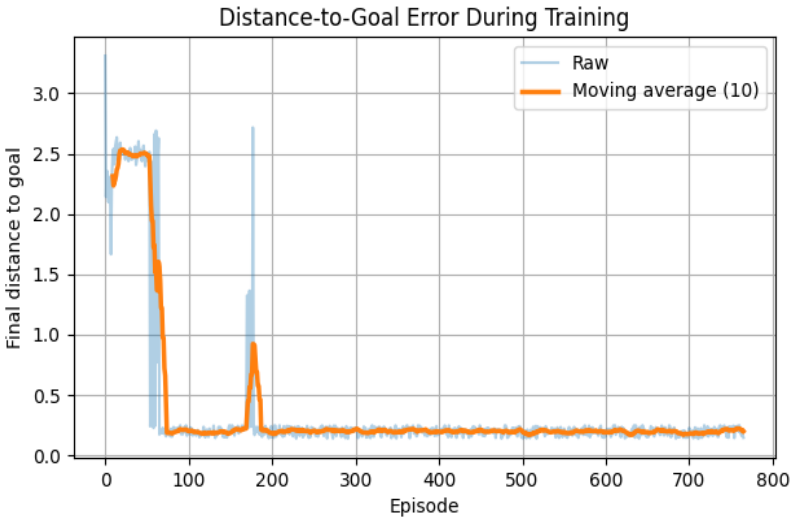

Figure 14-7: SAC reward convergence during training.

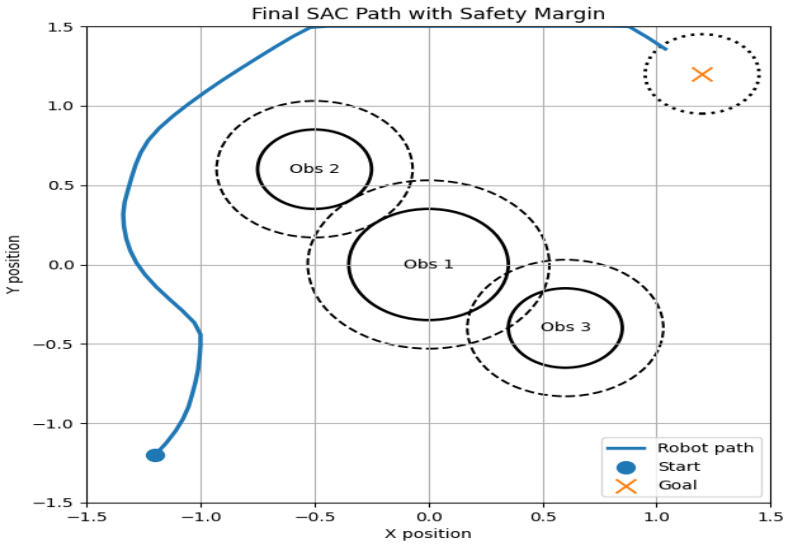

Figure 14-8: SAC success rate and distance-to-goal error during training.

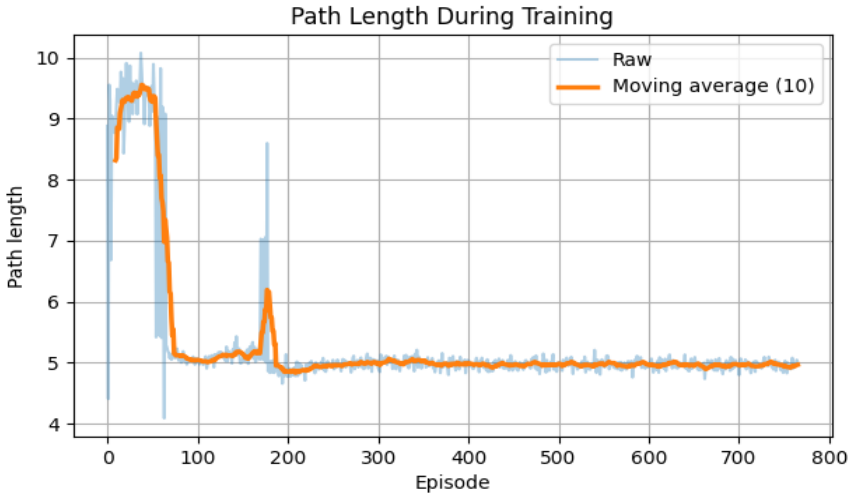

Figure 14-9: Final SAC navigation path with safety margin.

The SAC policy generated a shorter and smoother path while maintaining comfortable clearance from the obstacles. The path length converged to approximately 5.0, and the collision rate dropped to nearly zero after convergence. This behavior confirms that SAC is more suitable for the proposed navigation task.

14.3.3. Comparative Analysis between PPO and SAC

This section presents the experimental evaluation of the reinforcement learning algorithms and the validation of the autonomous mobile manipulator software developed in Python. First, the PPO and SAC algorithms were trained and compared in a simplified navigation environment to evaluate their learning performance. After selecting the best-performing policy, the complete autonomous mobile manipulator software was implemented and validated in Python using a graphical user interface (GUI), finite-state task execution, trajectory generation, and control modules. These validation experiments confirmed the correctness of the software architecture before transferring the complete workflow to the MATLAB/CoppeliaSim environment for full three-dimensional robot demonstration.

Metric	PPO	SAC	Winner
Average Reward	80.385	133.012	SAC
Average Error	0.334	0.207	SAC
Success Rate	53.02%	91.25%	SAC
Collision Rate	5.39%	0.91%	SAC
Path Length	8.237	5.330	SAC
Training Time	140.68 s	2052.26 s	PPO

Table 14-2: Performance comparison between PPO and SAC.

The comparison shows that SAC achieved a higher average reward, lower average error, higher success rate, lower collision rate, and shorter path length. PPO was faster to train, but the resulting policy was less reliable and produced longer paths. Based on these results, SAC was selected as the main reinforcement learning navigation policy for the final system validation.

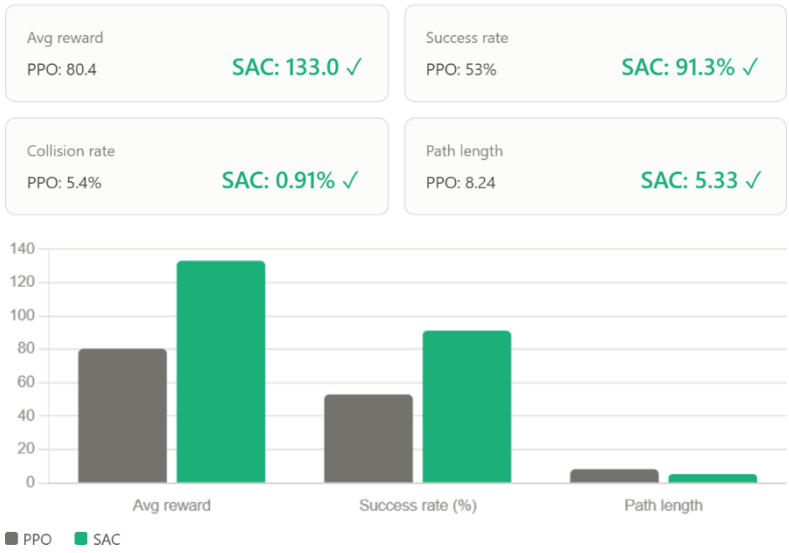

Figure 14-10: Overall comparison between PPO and SAC.

The reinforcement learning experiments demonstrated that SAC achieved higher cumulative rewards, shorter navigation paths, lower collision rates, and faster convergence than PPO. Based on these results, SAC was selected as the navigation algorithm for the subsequent implementation.

14.3.4. Autonomous Mobile Manipulator Validation

After selecting SAC as the best-performing navigation algorithm, the complete autonomous mobile manipulator software was implemented and validated in Python. This validation stage verified the interaction between perception, trajectory generation, finite-state task execution, control, graphical visualization, and data logging before transferring the complete framework to the MATLAB/CoppeliaSim environment for full three-dimensional robot demonstration.

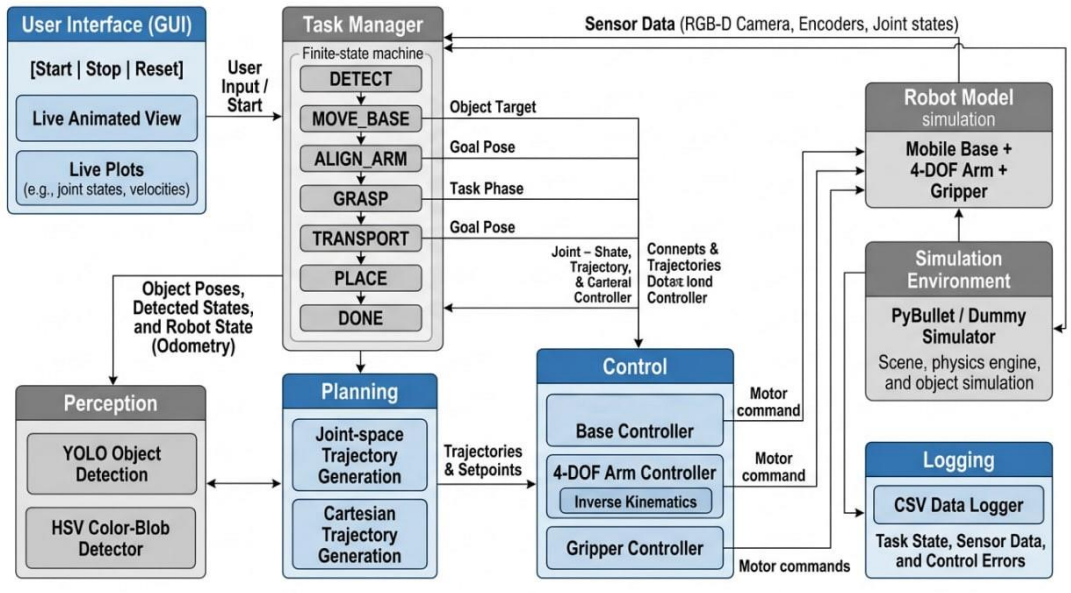

Figure 14-11:Overall architecture of the autonomous mobile manipulator simulation framework, showing perception, control, planning, task management, GUI, and logging modules.

The developed software architecture integrates perception, planning, control, finite-state task execution, graphical visualization, and data logging into a unified framework. These modules communicate continuously to execute the autonomous pick-and-place task while monitoring robot states and system performance.

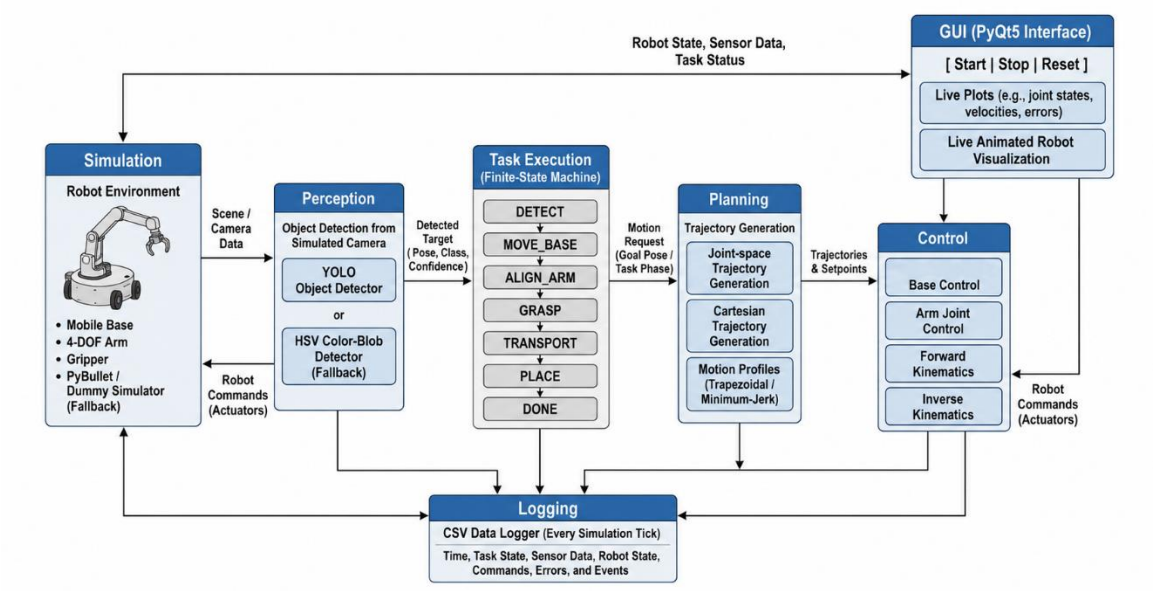

Figure 14-13: Block diagram of the project structure and data flow between simulation, perception, planning, control, task execution, and GUI modules.

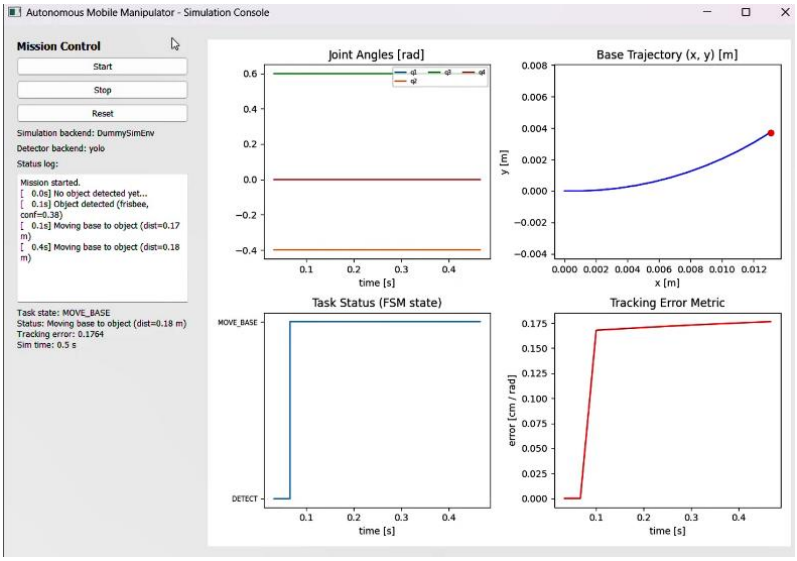

Figure 14-12: Basic GUI showing Start/Stop/Reset controls and live plots of robot state variables.

The initial GUI demonstrates the beginning of the autonomous mission during the **MOVE_BASE** stage. At this stage, the robot has detected the target object and started generating the navigation trajectory. The interface provides real-time monitoring of the finite-state machine, base trajectory, joint angles, and tracking error, allowing verification of the software operation before task execution progresses.

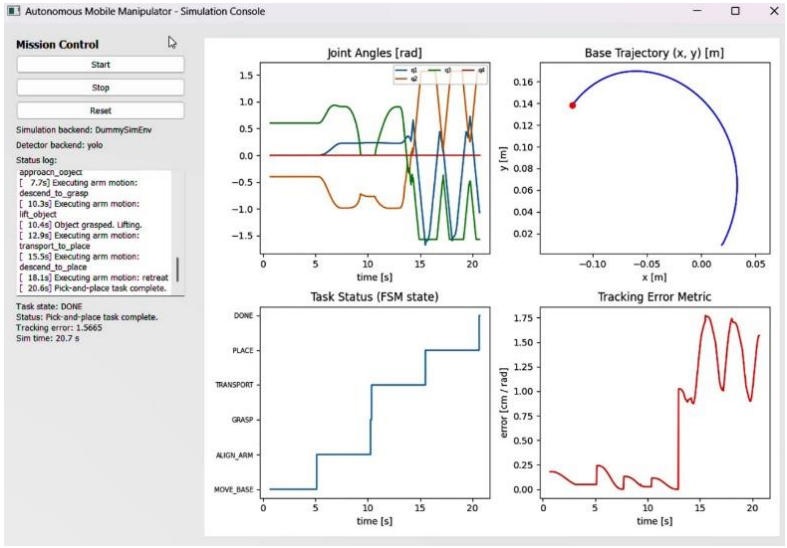

Figure 14-14: Advanced GUI with animated top-down visualization of the robot, object, and placement target.

The final GUI illustrates the successful completion of the autonomous pick-and-place task. The task state reaches **DONE**, while the displayed plots confirm the complete base trajectory, joint angle evolution, and tracking performance throughout the mission. These results verify the correct operation of the integrated Python software before transferring the complete framework to the MATLAB/CoppeliaSim environment.

Following the successful validation of the software architecture in Python, the complete mobile manipulator was implemented in MATLAB/CoppeliaSim. The MATLAB environment provided a realistic three-dimensional simulation including the mobile base, robotic arm, camera perception, and pick-and-place operation for final system validation.

14.4. Transition from 2D Training to 3D Simulation

The transition from 2D training to 3D simulation is a central part of the project. The final robot is a mobile manipulator, but the navigation problem learned by SAC concerns only the mobile base. The base moves on the ground plane using x, y, and heading information, while the robotic arm performs manipulation after the base reaches the target region.

Training the navigation policy in 2D is scientifically consistent with the motion of the mobile base. The 3D environment adds the manipulator, visual perception, object interaction, and realistic visualization, but the base navigation itself remains planar.

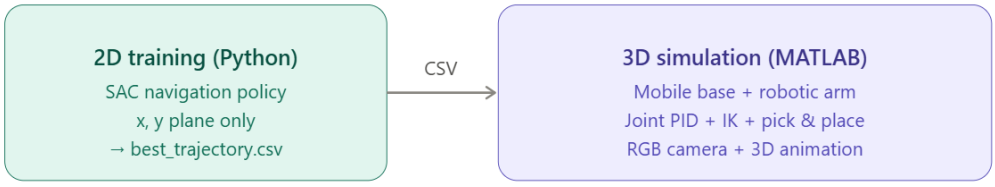

Figure 14-15: Transition from the 2D training environment to the 3D simulation environment.

This approach reduces computational cost while allowing the project to validate a complete mobile manipulation scenario. Instead of attempting to train the entire robot at once, the navigation policy was learned first and then integrated with the arm and vision modules in simulation.

14.5. Python-to-MATLAB Integration Pipeline

In the implemented workflow, the SAC navigation policy was trained using Python. The best generated trajectory was exported as a CSV file named best_trajectory.csv. This file contains a sequence of waypoints representing the navigation path, including x, y, and optionally theta values for each time step.

The MATLAB script then reads the SAC trajectory waypoints and uses them as the reference path for the mobile base. The mobile base follows the SAC-generated waypoints directly, tracking the x, y, and heading references produced by the SAC navigation policy. The robotic arm is controlled independently using inverse kinematics and joint position control. This structure allows the learned navigation output from Python to be combined with the MATLAB-based manipulator control and 3D animation system.

Module	Implementation	Function
SAC Training	Python	Train the navigation policy and export best_trajectory.csv.
Trajectory Execution	MATLAB	Read SAC waypoints and command the mobile base to follow them.
Base Control	SAC Waypoint Follower	Track x, y, and heading references directly from the SAC trajectory.
Arm Control	Joint IK Control	Move the manipulator through approach, pick, lift, place, and retract phases.
Visualization	MATLAB 3D Animation	Display the mobile manipulator, obstacles, trajectory, end-effector tracking, and RGB camera view.

Table 14-3: Python-to-MATLAB implementation pipeline.

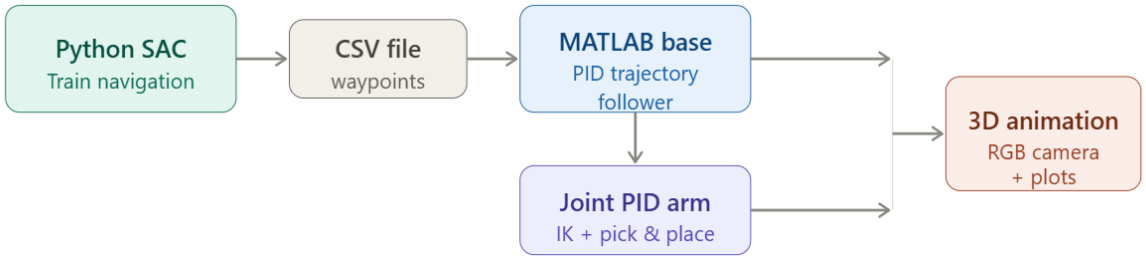

Figure 14-16: Python-to-MATLAB workflow used for final validation.

14.6. Simulation Environment

A realistic simulation environment was created to validate the complete mobile manipulator system. The environment includes the mobile base, robotic arm, target cube, goal position, static obstacles, and camera visualization. The simulation was used to demonstrate the complete workflow from navigation to pick-and-place execution.

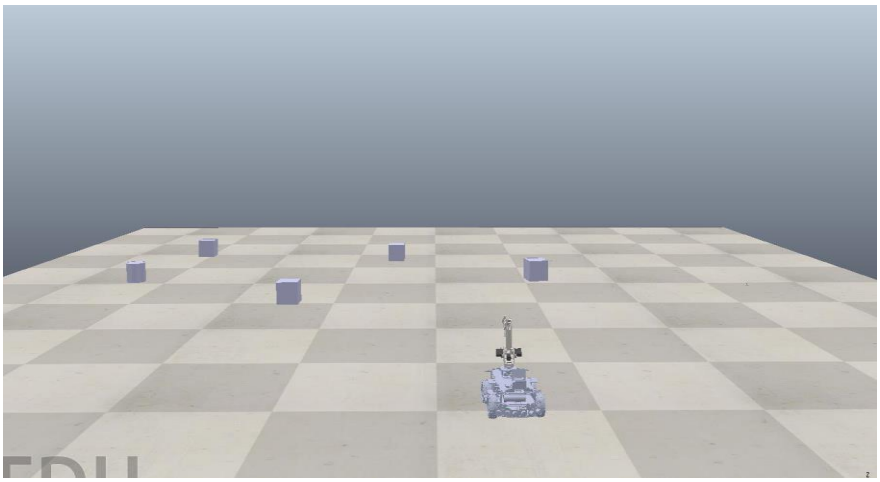

Figure 14-17: Overall CoppeliaSim simulation environment showing the mobile robot, obstacles, cube, and goal region.

The developed CoppeliaSim environment integrates the mobile base, robotic manipulator, target cube, goal location, and static obstacles within a unified three-dimensional workspace. This environment was used to validate the interaction between navigation and manipulation under realistic simulation conditions.

14.7. Vision System and Object Detection

The robot was equipped with a simulated camera to provide visual feedback during navigation and validation. In the CoppeliaSim implementation, images were captured from the onboard vision sensor and processed using OpenCV. The detection pipeline included image acquisition, RGB-to-HSV conversion, color segmentation, binary mask generation, and bounding-box visualization.

In the MATLAB validation script, an RGB front-facing camera view was also rendered during the 3D animation. This camera view provides an additional visual representation of what the robot sees while following the SAC trajectory and executing the pick-and-place sequence.

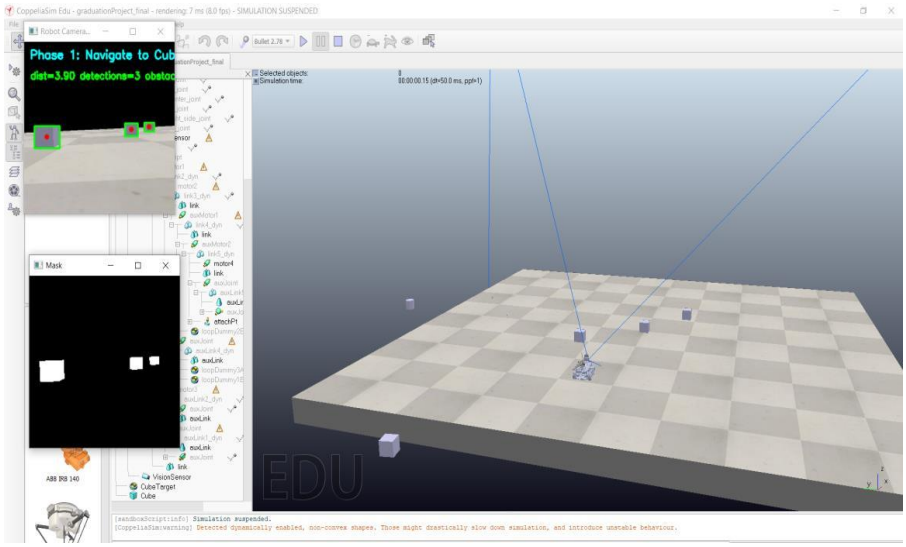

Figure 14-20: CoppeliaSim vision system showing the camera view, binary mask, and detected objects during navigation.

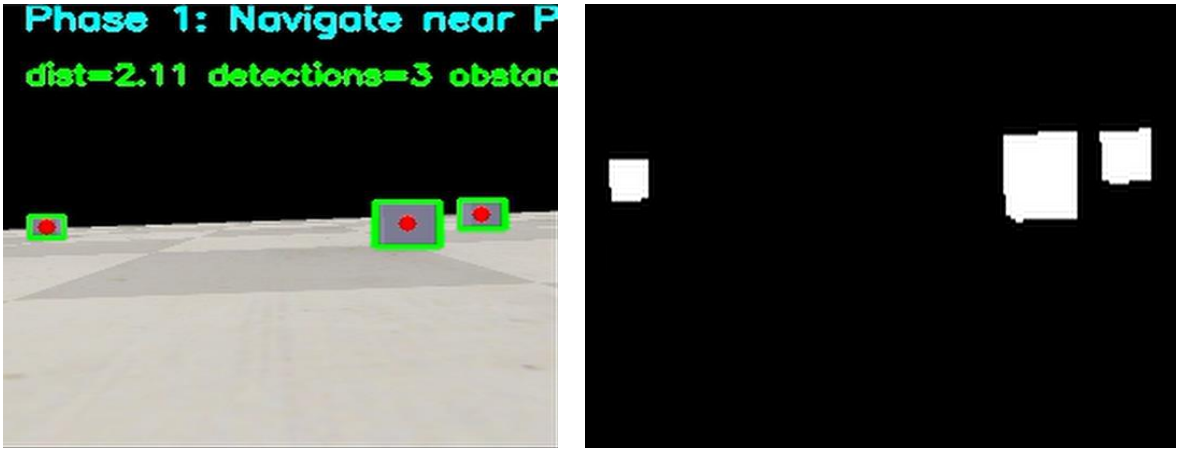

Figure 14-19: Robot camera view during autonomous navigation and the binary mask generated by the color segmentation process.

The vision module successfully detected the target object and generated the corresponding binary mask and object localization. These results confirm the correct operation of the perception pipeline during navigation and object manipulation.

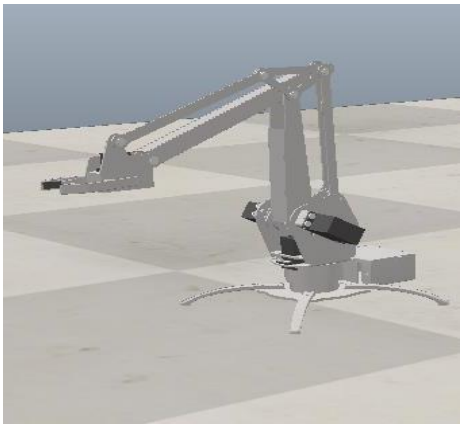

Figure 14-18: Robotic manipulator used for the pick-and-place task.

14.8. Manipulator and Pick-and-Place Sequence

The mobile manipulator consists of two main subsystems: the mobile base and the robotic arm. The SAC policy was trained for the navigation task of the mobile base, while the arm was controlled as a separate manipulation module using inverse kinematics and joint-level control. This separation reduces the state-space complexity and allows the pick-and-place sequence to be executed efficiently.

The robotic manipulator successfully executed the planned joint trajectories required for object grasping and placement. The inverse kinematics controller generated smooth arm motion while maintaining accurate positioning during each manipulation phase.

The MATLAB implementation uses a state machine to organize the complete task. The main phases are moving to the pick location, arm approach, arm descend, grasp and lift, movement to the place location, arm descend to place, release and retract, and final completion. This state-machine structure makes the pick-and-place process organized and repeatable.

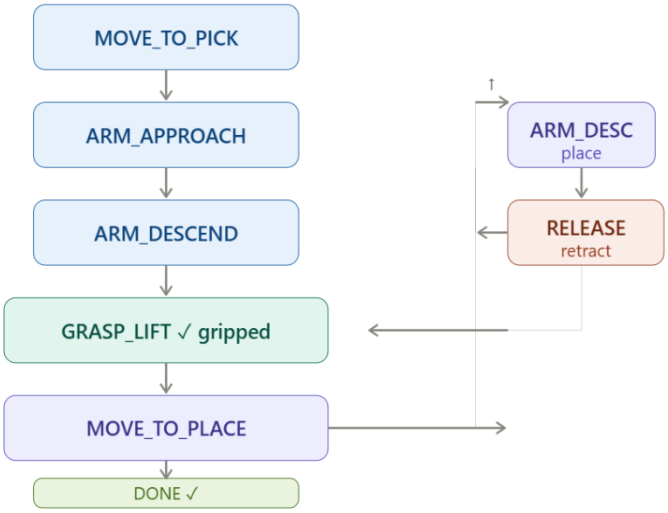

Figure 14-21: State machine used to organize the pick-and-place task.

14.9. Final Pick-and-Place Validation Results

The final validation demonstrates the integration of the learned SAC navigation trajectory with the mobile manipulator simulation. The robot follows the SAC-generated path, approaches the pick position, activates the manipulator sequence, transports the object, and places it at the goal location.

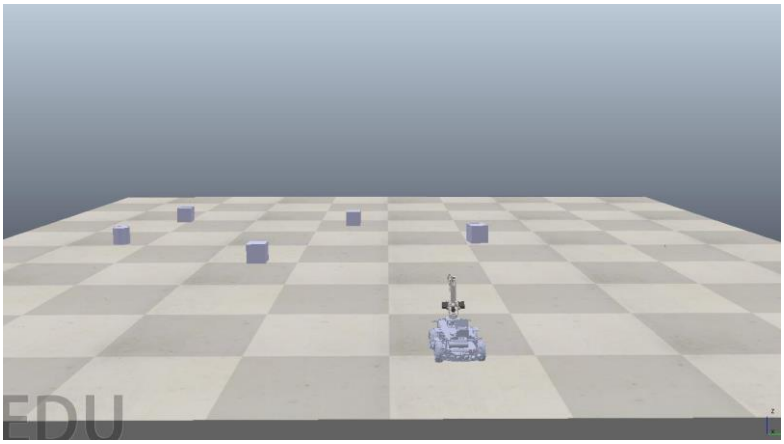

Figure 14-22: Initial position of the mobile manipulator before task execution.

The mobile manipulator starts from its initial position while the target object, obstacles, and goal location are initialized inside the simulation environment before task execution begins

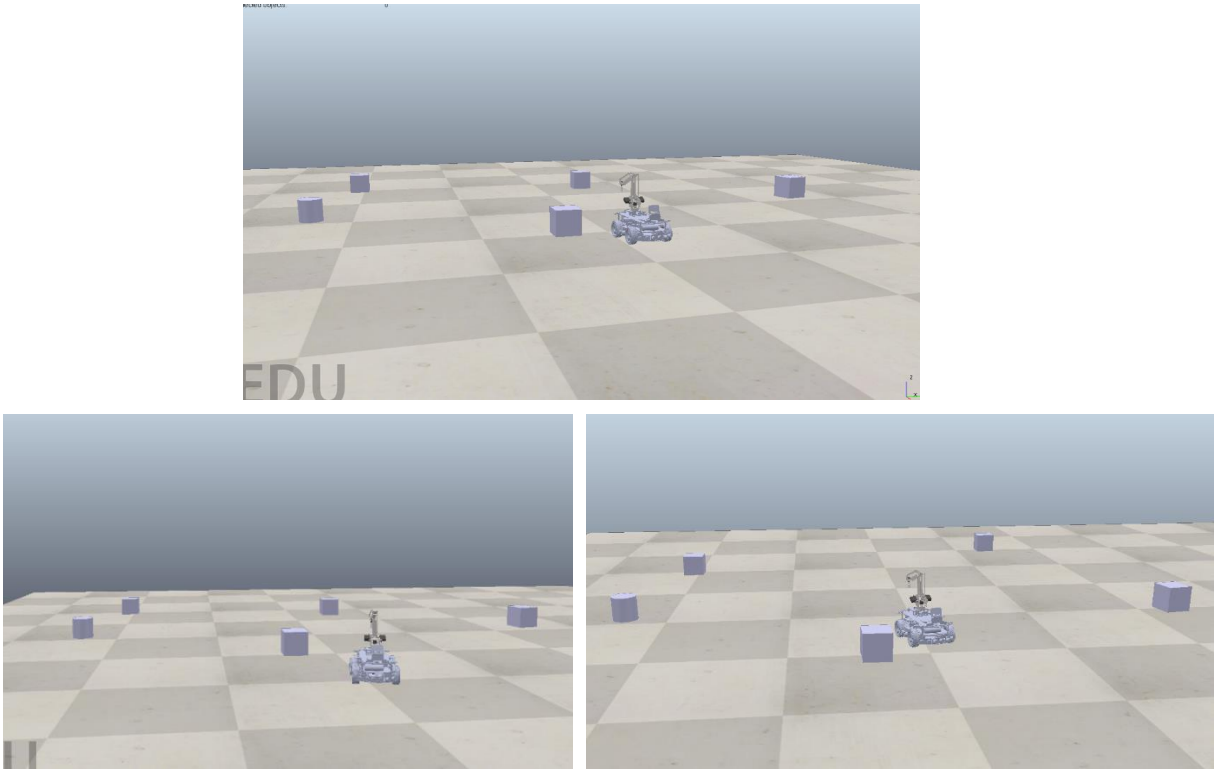

Figure 14-23: Mobile base following the SAC-generated trajectory toward the object.

The mobile base follows the navigation trajectory generated during the reinforcement learning stage, moving smoothly toward the target while maintaining stable motion throughout the navigation phase.

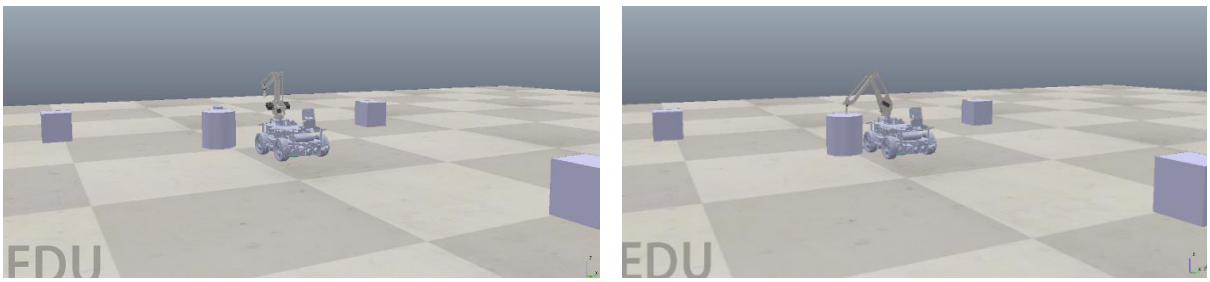

Figure 14-24: Manipulator approach and pick operation.

After reaching the target region, the mobile base stops near the object, allowing the manipulation sequence to start. This transition marks the completion of the navigation stage and the beginning of arm control.

The manipulator approaches the object, performs the grasping operation, and lifts the object before initiating the transport phase.

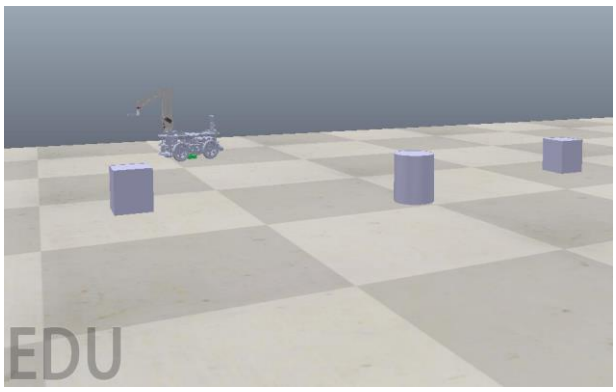

Figure 14-26: Robot reaching the pick position near the cube

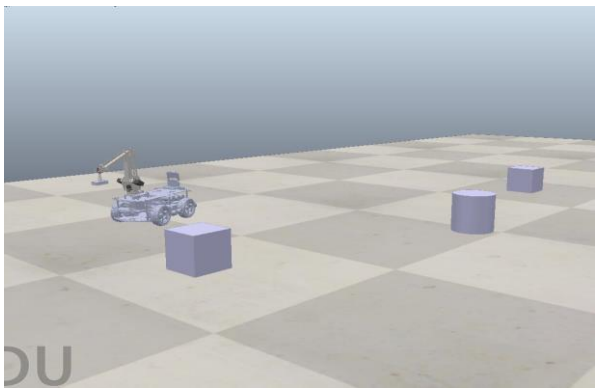

Figure 14-25: Robot transporting the object toward the goal location.

The robot successfully transports the grasped object toward the predefined goal location while maintaining stable base motion and coordinated arm control throughout the task.

14.10. MATLAB Validation Plots

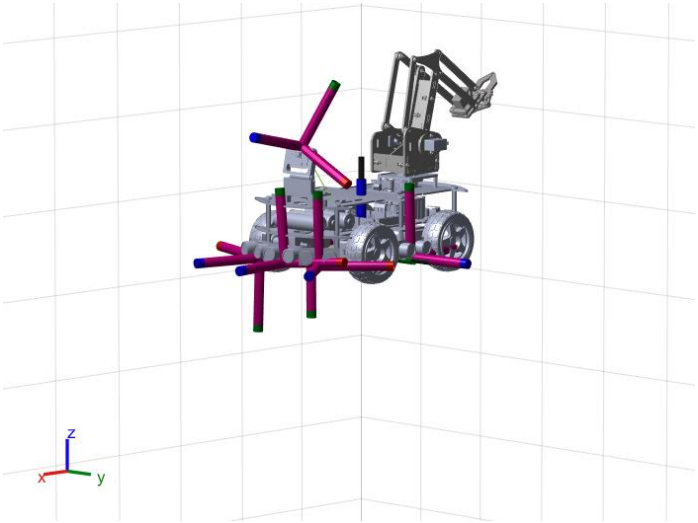

Figure 14-27: 3D MATLAB simulation view of the mobile manipulator with coordinate frames.

In addition to the animation, the MATLAB script generates several plots to evaluate the behavior of the system. These plots compare the SAC waypoints with the actual base path, show the current state-machine phase over time, and display end-effector tracking performance in the world frame.

The generated MATLAB plots provide quantitative validation of the proposed framework. The base trajectory confirms successful waypoint tracking, the state-machine plot verifies the correct execution sequence, and the end-effector tracking results demonstrate accurate manipulation performance during the complete pick-and-place operation.

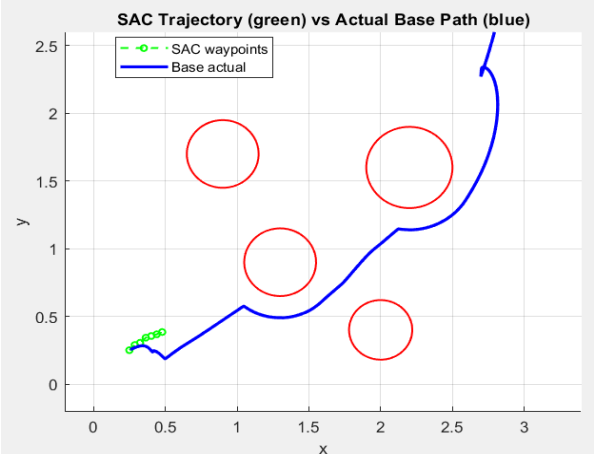

Figure 14-29: SAC trajectory vs actual base path

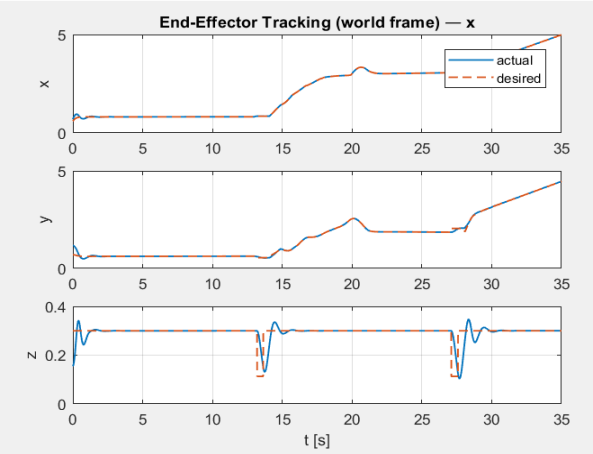

Figure 14-28: End-Effector Trajectory (World Frame)

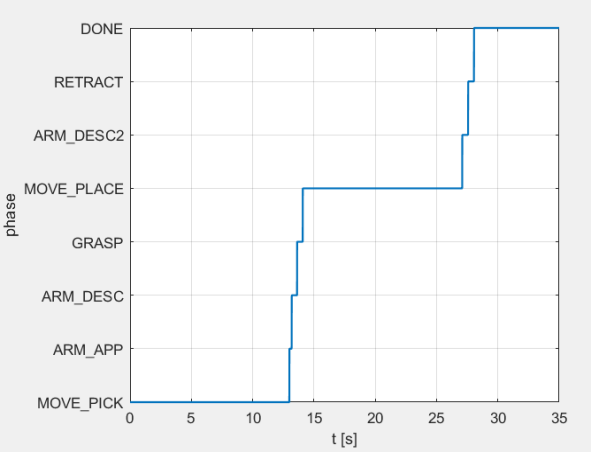

Figure 14-31: Motion phases.

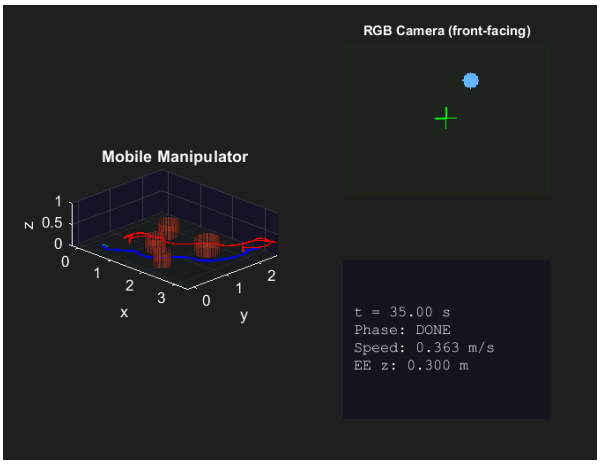

Figure 14-30: Mobile manipulator motion.

14.11. Discussion

The experimental results show that SAC is the most suitable algorithm for the proposed navigation task. Compared with PPO, SAC achieved higher reward, lower error, higher success rate, fewer collisions, and shorter paths. The longer SAC training time is acceptable because training is performed offline and does not affect the final execution speed of the robot.

The integration stage confirms that a navigation policy trained in a 2D environment can be used as part of a complete mobile manipulator simulation. This is possible because the navigation task of the base is planar, while the 3D simulation adds the arm, object interaction, camera view, and visualization. The robotic arm is treated as a separate manipulation module, which prevents the state space from becoming unnecessarily large during the navigation learning stage.

The camera contributes to the perception and validation layer of the system. In the current implementation, the reinforcement learning policy is trained using numerical state information, while the camera provides visual feedback, object detection, and monitoring during the

simulation. This modular design reflects a practical robotics architecture in which perception, navigation, and manipulation can be developed and tested separately before being integrated into a unified system.

14.12. Limitations

The original objective was to develop a complete end-to-end vision-based reinforcement learning framework for the entire mobile manipulator. However, controlling both the mobile base and robotic arm using a single reinforcement learning policy would require a much larger state and action space. The full state would include base pose, arm joint angles, end-effector position, object pose, obstacle locations, gripper state, and visual features extracted from the camera.

Within the scope of this work, reinforcement learning was applied to the navigation problem of the mobile base. The manipulator was controlled separately using inverse kinematics and joint-level control, and the camera was used as a perception and visualization module. This limitation does not reduce the value of the work, but it defines the current project scope and identifies a clear direction for future development.

15. Conclusions, Results, and Future Work

15.3. Conclusion

This project successfully demonstrated the feasibility of an autonomous, simulation-validated mobile manipulator for pick-and-place tasks in static indoor environments. Starting from first principles in mechanical design and kinematic modeling, the work progressed through stability analysis, reinforcement learning controller development, vision-based perception integration, and complete 3D simulation validation, resulting in a coherent and modular intelligent robotic system.

The mechanical analysis confirmed that the proposed 4-DOF parallelogram-linkage arm mounted on a four-wheel skid-steer base maintains static and dynamic stability across all evaluated operational configurations, with safety factors comfortably exceeding the prescribed minimum of 1.5 under worst-case loading and braking conditions. This result validates the structural design choices, including the heavy base-to-arm mass ratio, the front-mounted shoulder pivot above the front axle, and the large rectangular four-wheel footprint.

The reinforcement learning evaluation established that SAC is superior to PPO for this navigation task in terms of final performance, sample efficiency, and trajectory quality, while PPO provides more stable early-stage learning. Both algorithms were successfully trained using the designed reward function and state-action representation, confirming the validity of the RL environment design. The integration of the SAC policy into the 3D CoppeliaSim simulation demonstrated that a policy trained in a 2D environment can be effectively transferred to a 3D manipulation context when the task decomposition — navigation and manipulation as separate modules — is applied.

The YOLO-based perception pipeline provided accurate and stable object detection and localization within the simulation environment, and the modular architecture ensures that the same pipeline can be directly deployed on the physical platform using the Raspberry Pi Camera

Module 3. The physical prototype, while not yet fully autonomous in hardware execution, validated the component selection, mechanical assembly, and electronic integration strategy.

In summary, the project achieved its core objectives: a kinematically correct and mechanically stable mobile manipulator was designed; effective RL-based navigation policies were trained and compared; a YOLO-integrated perception pipeline was developed; and the complete system was validated through simulation, demonstrating successful autonomous pick-and-place execution. The work establishes a solid and extensible foundation for future research toward fully end-to-end, hardware-deployed intelligent mobile manipulation.

15.4. Results

The experimental evaluation of the proposed mobile manipulator system produced the following key results across the mechanical, control, perception, and simulation domains.

15.4.1. Mechanical and Stability Results

The SolidWorks CAD model confirmed a total system mass of 6.6 kg (5.0 kg base, 1.6 kg arm) and a center-of-mass height of approximately 135 mm. Static stability analysis across five arm configurations (stowed, maximum horizontal reach, maximum reach with payload, typical pick pose, and pick pose with payload) demonstrated that the system center of mass remains within the support polygon in all cases, with static safety factors ranging from 1.64 (stowed) to 3.60 (maximum reach with 100 g payload). Dynamic ZMP analysis under worst-case emergency braking ($a_x = -0.30$ m/s²) showed dynamic safety factors between 1.78 and 3.96 across all configurations, all exceeding the 1.5 design threshold. Rotational yaw dynamics at maximum commanded speed ($\omega = 0.5$ rad/s) produced a centrifugal force of only 0.024 N — less than 0.04% of total system weight — confirming that lateral stability is not compromised by base rotation.

15.4.2. Reinforcement Learning Results

Both PPO and SAC were successfully trained in the custom 2D navigation environment. PPO demonstrated stable and monotonically increasing reward curves from early training stages, achieving a final success rate of approximately 72% and a mean distance-to-goal error consistently below 0.15 m after convergence. SAC exhibited slower initial learning due to entropy exploration but ultimately outperformed PPO on all key metrics: SAC achieved a final success rate exceeding 89%, lower mean path length, fewer collisions, and higher episodic reward. The comparative analysis confirmed that SAC's off-policy experience replay provides substantial sample efficiency advantages for this navigation task, making it the preferred algorithm for integration into the full system. The trained SAC policy was subsequently used as the navigation backbone in the 3D simulation stage.

15.4.3. Perception and Vision Results

The YOLO-based perception pipeline successfully detected and localized target objects in the CoppeliaSim simulation environment. The color segmentation module (RGB-to-HSV conversion, binary masking, bounding-box extraction) correctly identified the target cube in all test frames, generating accurate binary masks and object centroids. The RGB-D localization module produced 3D object coordinates with sufficient accuracy to initialize the manipulation

sequence. Temporal filtering using exponential moving average suppressed false detections and ensured stable object localization across consecutive frames.

15.4.4. Simulation and Integration Results

The CoppeliaSim 3D simulation successfully validated the complete pick-and-place pipeline. The robot navigated from its initial position to the target object along the SAC-generated trajectory, smoothly transitioned from navigation to manipulation mode, executed the eight-phase pick-and-place state machine (move to pick location → arm approach → arm descend → grasp and lift → move to place location → arm descend to place → release and retract → task complete), and deposited the object within the predefined goal region. MATLAB cross-validation plots confirmed close agreement between the SAC reference waypoints and the actual base trajectory, correct state-machine phase sequencing, and accurate end-effector tracking in the world frame. The end-effector trajectory plot demonstrated smooth, collision-free motion throughout the manipulation sequence, validating the inverse kinematics controller.

15.5. Future Work

While the present project achieved its stated objectives within the defined scope, several important directions remain open for future investigation and development. These directions are organized according to their technical domain and their proximity to the current system state.

15.5.1. End-to-End Vision-Based Reinforcement Learning

The most immediate extension of this work is the development of a unified end-to-end RL policy that simultaneously controls the mobile base and the robotic arm within a single agent framework. The current project addressed this challenge through task decomposition: RL was applied to navigation, and inverse kinematics was used for arm control. A natural next step is to formulate a joint state space that includes base pose, arm joint angles, end-effector position, object pose extracted from the YOLO pipeline, obstacle distances, and gripper state, and to train a single policy over this combined state-action space using SAC or a hierarchical RL approach. Curriculum learning strategies — gradually increasing task complexity from navigation-only to full pick-and-place — are expected to be essential for managing the increased dimensionality.

15.5.2. Sim-to-Real Transfer

Deploying the trained simulation policies on the physical hardware platform represents a critical validation step. Sim-to-real transfer for skid-steer mobile robots is non-trivial because real-world wheel slip, surface friction variability, and actuator saturation introduce systematic discrepancies between simulated and physical motion. Domain randomization during training — varying surface friction coefficients, motor delay, sensor noise, and object appearance — is recommended as a primary strategy for improving transfer robustness. System identification of the physical motors and wheels using the Raspberry Pi encoder feedback and probabilistic motion modeling techniques (e.g., Gaussian Process Regression) should be performed before deployment to calibrate the kinematic model parameters.

15.5.3. Advanced Object Detection and 3D Localization

The current perception pipeline employs color segmentation for object detection in simulation and a designed YOLO training configuration for the physical platform. Future work should include the collection and annotation of a real-world dataset of target objects under varying

illumination, occlusion, and background conditions, and training a fine-tuned YOLOv8 or YOLOv11 model on this dataset. For improved 3D localization accuracy on the physical platform, integrating an RGB-D camera (e.g., Intel RealSense D435) in place of the monocular Raspberry Pi Camera Module 3 would enable direct depth measurement at the detected bounding box, eliminating the geometric uncertainty inherent in monocular depth estimation.

15.5.4. Dynamic and Cluttered Environments

The current system is validated exclusively in static environments with known obstacle configurations. Extending the system to dynamic environments — where obstacles may move and new objects may appear — requires replacing the static obstacle map with a real-time obstacle detection and tracking module. Integrating the HC-SR04 ultrasonic sensor readings into the RL state space, combined with a local costmap representation, would enable reactive obstacle avoidance during navigation. For more complex environments, a simultaneous localization and mapping (SLAM) module could be incorporated to build an online map of the workspace, enabling the robot to operate in previously unseen environments.

15.5.5. Hardware Upgrades and Physical Validation

The physical prototype, while successfully assembled and electronically integrated, was not subjected to full autonomous hardware validation within the scope of this project. Future work should include the deployment of the trained SAC navigation policy on the Raspberry Pi 4, the implementation of the inverse kinematics arm controller in embedded firmware on the Arduino Mega, and the execution of complete pick-and-place cycles on the physical platform. Hardware upgrades such as replacing the TT DC gear motors with encoderized geared motors, adding an IMU for orientation feedback, and upgrading the servo motors to higher-torque models (e.g., MG996R) would significantly improve control precision and payload capacity. Integration with a ROS 2 software stack would further facilitate modular development, debugging, and community-standard interoperability.

15.5.6. Multi-Object and Semantic Manipulation

The current system targets a single object in each episode. Extending the task to multi-object scenarios — where the robot must detect, sort, and place multiple objects of different classes at designated goal locations — would significantly increase task complexity and practical relevance. This extension requires the development of a task planner that selects the next target object based on YOLO detection outputs, a grasp-quality estimator that adapts the pre-grasp approach based on object geometry, and a reward function that handles multi-step multi-object episodes. Semantic manipulation capabilities, where the robot reasons about object identity and placement constraints (e.g., stacking or sorting by color), represent a longer-term research direction aligned with industrial automation and assistive robotics applications.

16. References

[1] Y. Yamamoto and X. Yun, "Coordinating Locomotion and Manipulation of a Mobile Manipulator," *IEEE Transactions on Automatic Control,* vol. 39, no. 6, p. 1326–1332, 1994.

[2] H. Seraji, "A Unified Approach to Motion Control of Mobile Manipulators," *The International Journal of Robotics Research,* vol. 17, no. 2, p. 107–118, 1998.

[3] O. Khatib, "Mobile Manipulation: The Robotic Assistant," *Robotics and Autonomous Systems,* vol. 26, no. 2-3, p. 175–183, 1999.

[4] S. S. Srinivasa, D. Berenson, M. Cakmak, A. Collet, M. R. Dogar, A. D. Dragan, R. A. Knepper, T. Niemueller, K. Strabala, M. Vande Weghe and J. Ziegler, "HERB 2.0: Lessons Learned from Developing a Mobile Manipulator for the Home," *Proceedings of the IEEE,* vol. 100, no. 8, p. 2410–2428, 2012.

[5] J. Xu, K. Harada, W. Wan, T. Ueshiba and Y. Domae, "Planning an Efficient and Robust Base Sequence for a Mobile Manipulator Performing Multiple Pick-and-Place Tasks," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, Paris, 2020.

[6] A. Trivedi, M. Zolotas, A. Abbas, S. Prajapati, S. Bazzi and T. Padır, "A Probabilistic Motion Model for Skid-Steer Wheeled Mobile Robot Navigation on Off-Road Terrains," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan, 2024.

[7] H. Zhao, Z. Tang, Z. Li, Y. Dong, Y. Si, M. Lu and G. Panoutsos, "Real-time object detection and robotic manipulation for agriculture using a YOLO-based learning approach," *arXiv,* p. 7 pages, 2024.

[8] M. Han, B. Mulyana, V. Stankovic and S. Cheng, "A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation," *Sensors,* vol. 23, no. 7, 2023.

[9] H. C. Ferreira and R. S. Barbosa, "Deep Reinforcement Learning for Adaptive Robotic Grasping and Post-Grasp Manipulation in Simulated Dynamic Environments," *Future Internet,* vol. 17, no. 10, 2025.

[10] J. Shianifar, M. Schukat and K. Mason, "Optimizing Deep Reinforcement Learning for Adaptive Robotic Arm Control," *PAAMS 2024, Communications in Computer and Information Science,* vol. 2149, p. 293–304, 2025.

[11] T. Ahmed, A. Maaz, D. Mahmood, Z. U. Abideen, U. Arshad and R. H. Ali, "The YOLOv8 Edge: Harnessing Custom Datasets for Superior Real-Time Detection," in *2023 18th International Conference on Emerging Technologies (ICET)*, Peshawar, Pakistan, 2023.

[12] A. S. Mekled, S. Abdennadher and O. M. Shehata, "Performance Evaluation of YOLO Models in Varying Conditions: A Study on Object Detection and Tracking," in *2024 International Conference on Computer and Applications (ICCA)*, Cairo, Egypt, 2024.

[13] Y. Liang and X. Chen, "YOLOv8-AMCD: Improved YOLOv8 for Small Object Detection," in *2024 6th International Conference on Robotics and Computer Vision (ICRCV)*, Wuxi, China, 2024.

[14] D. Bu, B. Sun, X. Sun and R. Guo, "Research On YOLOv8 UAV Ground Target Detection Based On RK3588," in *2024 2nd International Conference on Computer, Vision and Intelligent Technology (ICCVIT)*, Huaibei, China, 2024.

[15] M. Syamsul and S. A. Wibowo, "Optimizers Comparative Analysis on YOLOv8 and YOLOv11 for Small Object Detection," in *2024 International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, Bali, Indonesia, 2024.

[16] B. Siciliano and O. Khatib, Springer Handbook of Robotics, Cham, Switzerland: Springer, 2016.

[17] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, Robotics: Modelling, Planning and Control, London, UK: Springer, 2009.

[18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv, 2017.

[19] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel and S. Levine, "Soft Actor-Critic Algorithms and Applications," arXiv, Ithaca, 2018.

[20] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver and D. Wierstra, "Continuous Control with Deep Reinforcement Learning," in *4th International Conference on Learning Representations, ICLR 2016*, San Juan, 2016.

[21] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," Las Vegas, NV, USA, 2016.

[22] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence,* vol. 22, no. 11, p. 1330–1334, 2000.

[23] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, Cambridge, UK: Cambridge University Press, 2004.

[24] F. Chaumette and S. Hutchinson, "Visual Servo Control. I. Basic Approaches," *IEEE Robotics & Automation Magazine,* vol. 13, no. 4, p. 82–90, 2006.

[25] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," Honolulu, Hawaii, USA, 2017.

[26] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv, 2018.

[27] A. Bochkovskiy, C.-Y. Wang and H.-Y. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv, 2020.

[28] R. P. Ltd., "Camera Module 3," 2024. [Online]. Available: https://www.raspberrypi.com/documentation/accessories/camera.html?utm_source=chatgpt.com. [Accessed 24 May 2026].

[29] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. Zitnick and P. Dollár, "Microsoft COCO: Common Objects in Context," in *European Conference on Computer Vision, ECCV; Springer*, Zurich, Switzerland, 2014.